UCS3461 Foundations of Artificial Intelligence

ADVERSARIAL SEARCH

Semester 4, B.E. Computer Science and Engineering
21 January 2026

Sakaya Milton R
SSN College of Engineering

Outline

- Games
- Minimax: Optimal Decisions in Games
- Minimax with α-β Pruning

Games

Games

- Multiagent environment
  - Actions of other agents affect the state of the world
- Agents have independent utilities (values on outcomes)
- Games: agents compete – their goals conflict
  - Adversarial search
- Agents may also cooperate – different problem

Types of Games

- Many different kinds of games!
- Axes
  - One, two, or more players?
  - Zero sum?
  - Perfect information (can you see the state)?
  - Deterministic or stochastic?
- Need algorithms for calculating a strategy (policy) which recommends a action/move in each state.

$$\text{policy} : \text{state} \rightarrow \text{action}$$

Zero-Sum Games

- Game theory – Economics
- Two agents act alternately (two-player turn-taking)
- Fully observable (perfect information)
- Deterministic
- Utilities at the end of the game are equal and opposite
  - Agents have opposite utilities (values on outcomes)
  - Adversarial, pure competition
  - Lets us think of a single value that one maximizes and the other minimizes
- Constant-sum game
  - Chess: $(0, 1)$, $(1, 0)$, $(\frac{1}{2}, \frac{1}{2})$

Games are Too Hard to Solve

Games are like real-world problems — too hard to solve

- Chess
- Branching factor $= 35$
- 50 moves by each player
- Search tree $= 35^{100} \approx 10^{154}$ nodes!
- Search graph has "only" 10^{40} nodes!!
- Optimal decision is infeasible
- Some decision is imperative, like real-world!

Two-player Game

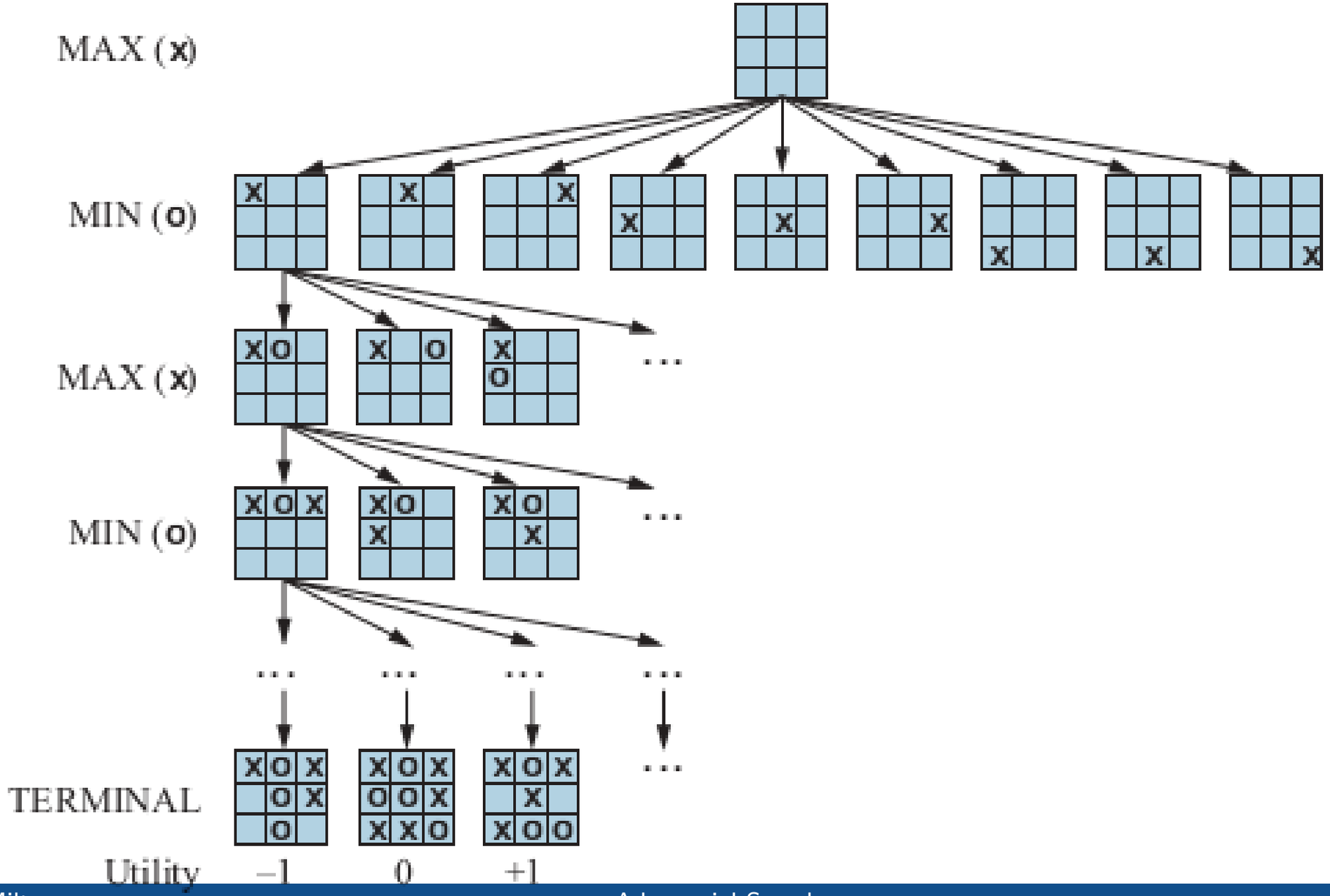
Search problem

- S_0: Initial state
- Player(s): which player has the move in state s
- Action(s): set of legal actions in state s
- Result(s, a): transition model, result of move (action) a in state s
- Terminal?(s): terminal states – true when the game is over
- Utility(s, p): utility function (objective function, payoff) – the numeric value of the game in terminal state s for player p when the game is over.
  - Zer-sum game
  - Chess: $(0, 1)$, $(1, 0)$, $(\frac{1}{2}, \frac{1}{2})$

Game Tree

- Initial state, Actions(), Result()
- Nodes $\equiv$ game states, edges $\equiv$ actions
- Search tree examines enough nodes of game tree to choose a move
- Tic-tac-toe
  - MAX moves first, MAX and MIN alternate
  - MAX places an X in an empty square
  - MIN places an O in an empty square
  - Leaves – terminal states
    - one player has 3 in a row
    - all the squares are filled
  - Utility values from the point of view of MAX
  - $9! = 362,880$ terminal nodes
- Chess: 10^{40} nodes

Tic-Tac-Toe Game Tree



Choose Good Move

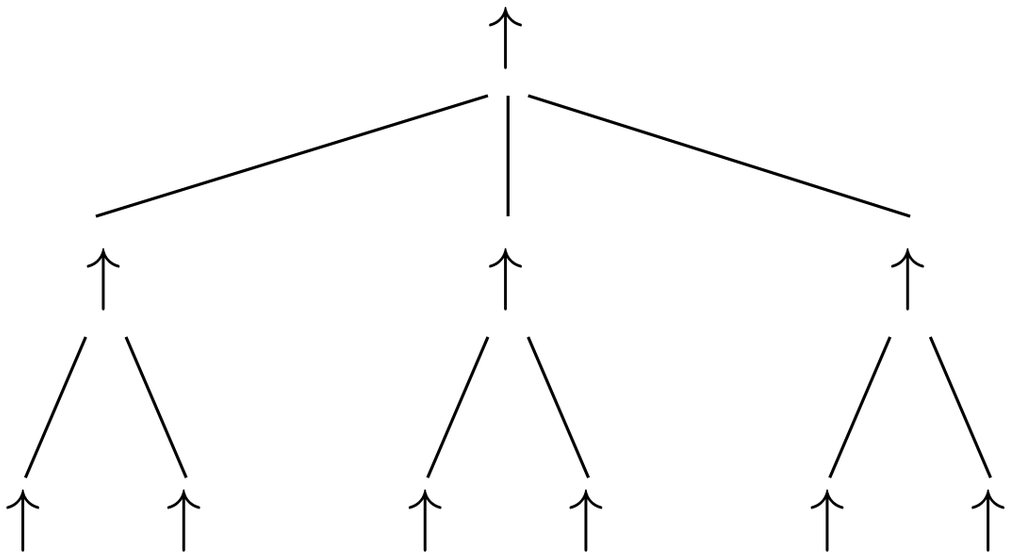
- Choose a good move when time is limited
- Pruning – ignore subtrees of search tree, which do not affect the final choice
- Evaluation function – heuristic function, to approximate the true utility without doing complete search

Optimal Decisions in Games

Optimal Decisions in Games

- In search, strategy = sequence of actions to a goal state
- In adversarial search, contingent strategy
  - MAX's move in the initial state
  - In states resulting from every possible response by MIN, MAX's moves
  - In states resulting from every possible response by MIN to those moves, MAX's moves

Value of States



$$\text{Value}(n) = \max_{c \in \text{Successors}(n)} \text{Value(c)}$$

Terminal state
$\text{Value}(n) = \text{Utility}(n)$

Maxvalue Algorithm

Algorithm: Maxvalue(n)

Input: n a node in a game tree
Output: A pair (value in node n, path that gives this value)

```
if n is a leaf node then  return evaluate(n)
max_val ← −∞
foreach child c of n do
    val ← Maxvalue(c)
    if max_val < val  then
        max_val ← val
return max_val
```

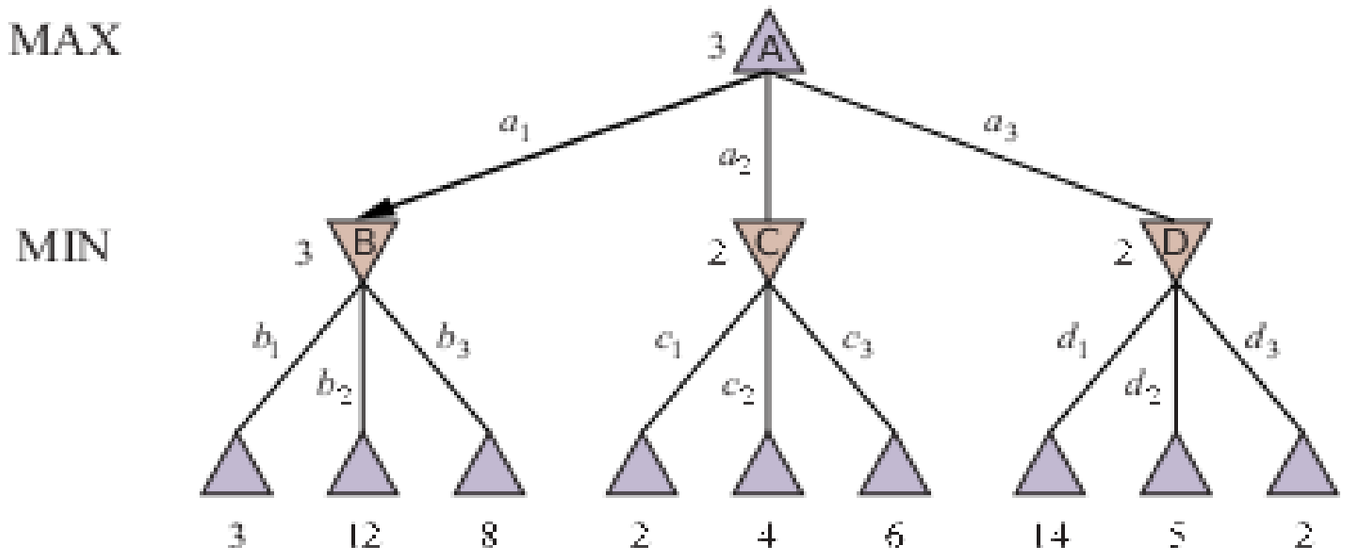
Maxvalue Algorithm

Algorithm: Maxvalue(n)

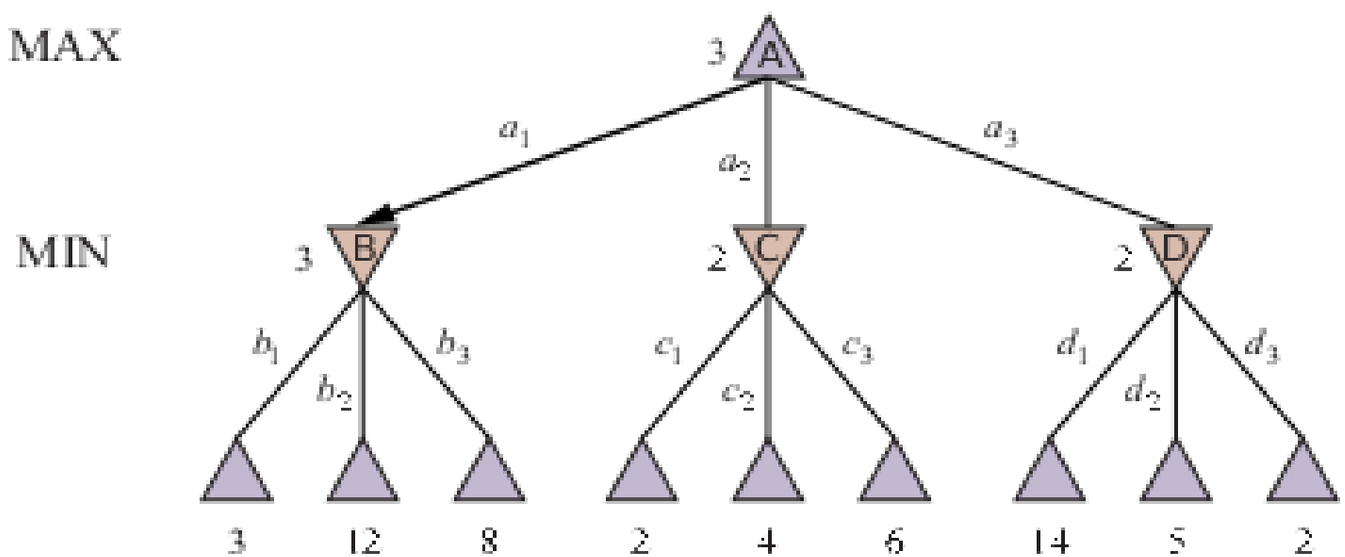
Input: n a node in a game tree
Output: A pair (value in node n, path that gives this value)

```
if n is a leaf node then  return evaluate(n), [ ]
max_val ← −∞
foreach child c of n do
    val, path ← Maxvalue(c)
    if max_val < val  then
        max_val ← val
        best ← [c] + path
return max_val, best
```

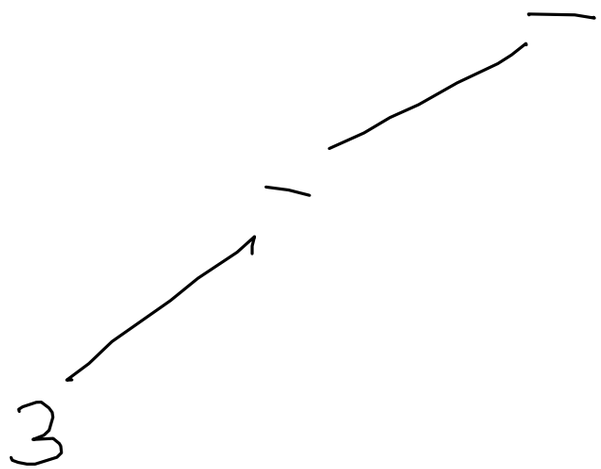
Maxvalue Example Trace

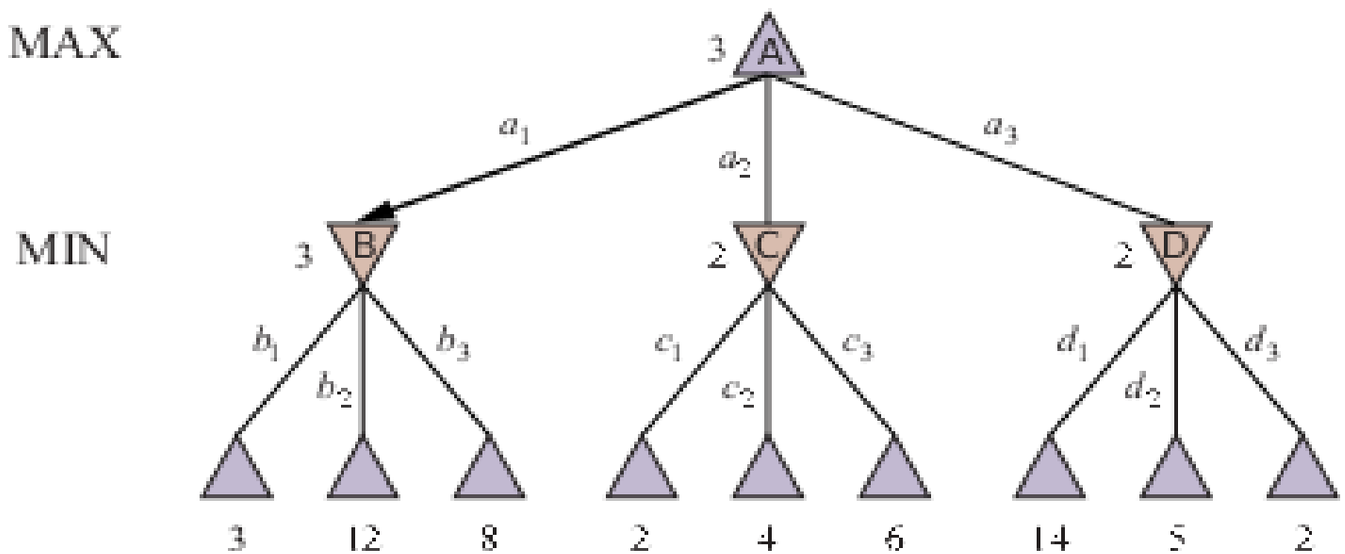


Maxvalue Example Trace

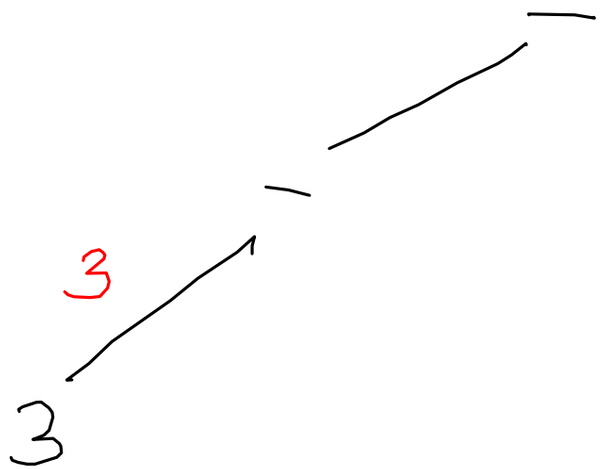


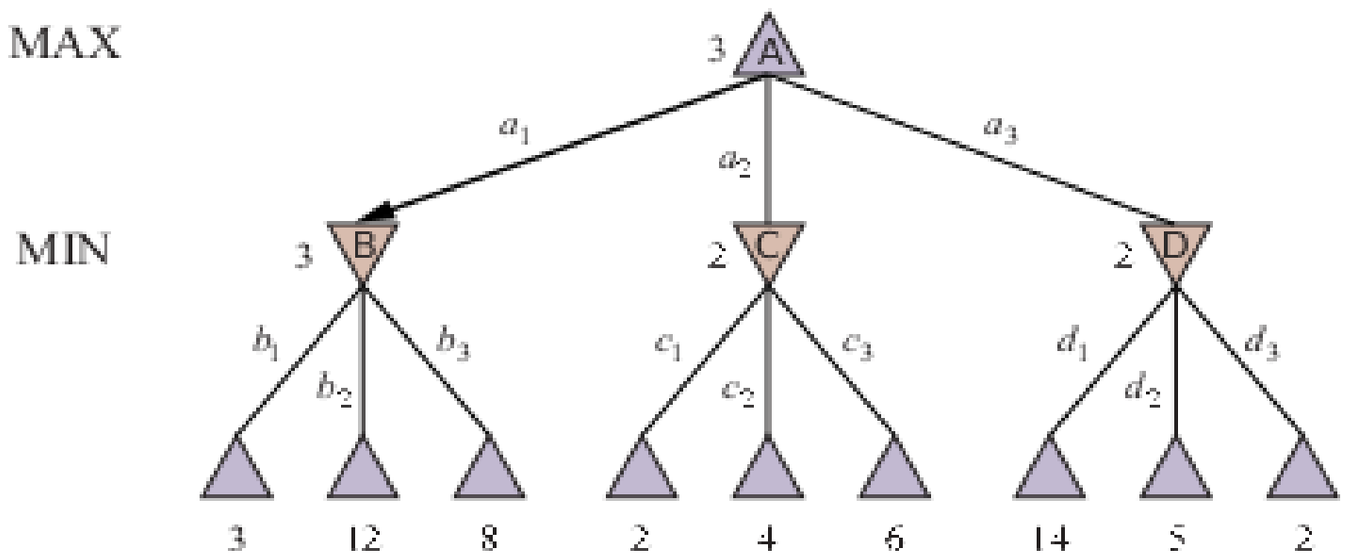
Maxvalue Example Trace



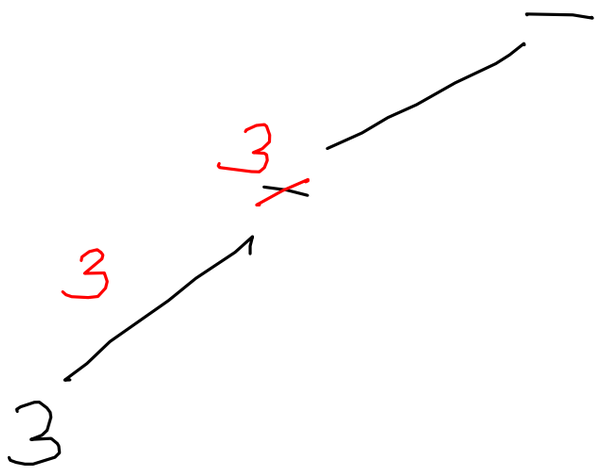


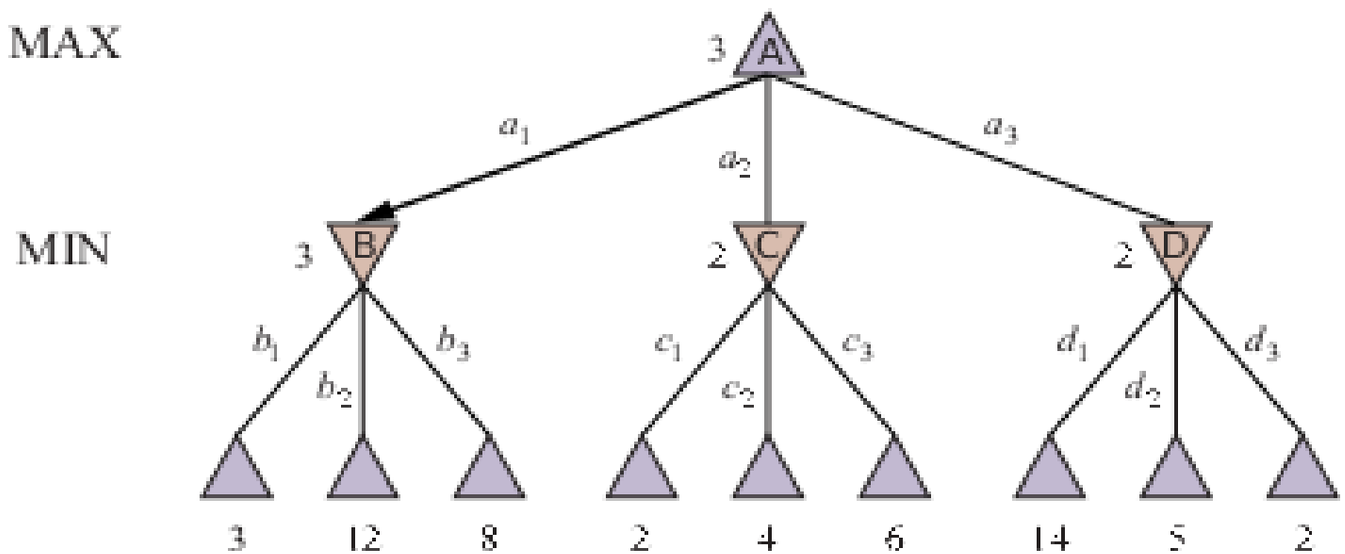
Maxvalue Example Trace



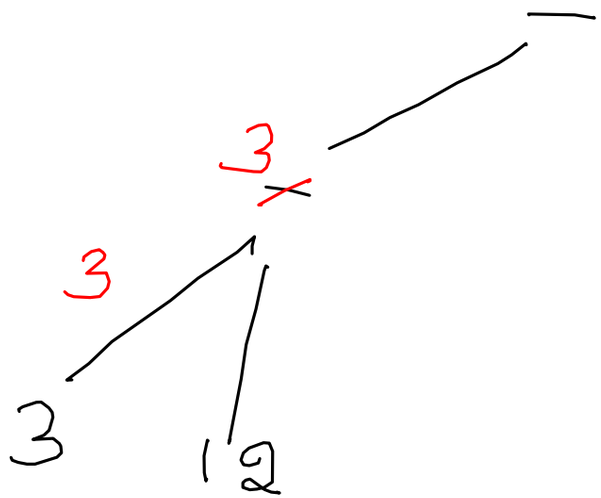


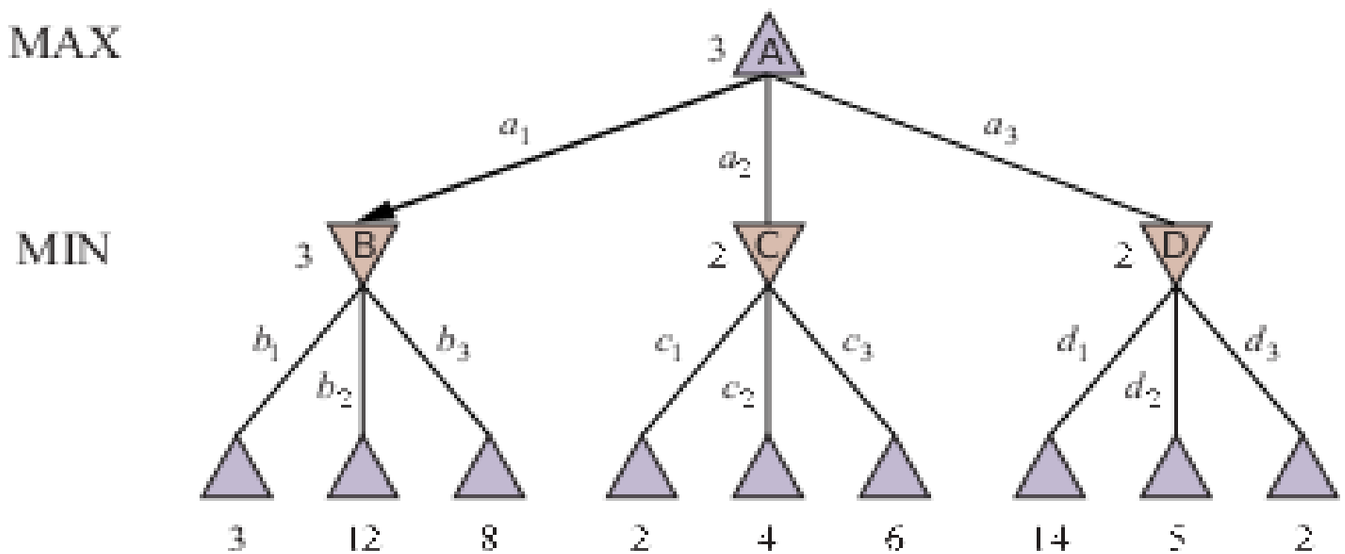
Maxvalue Example Trace



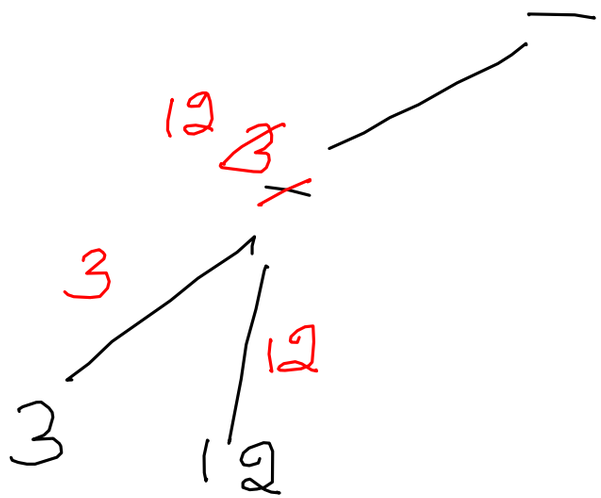


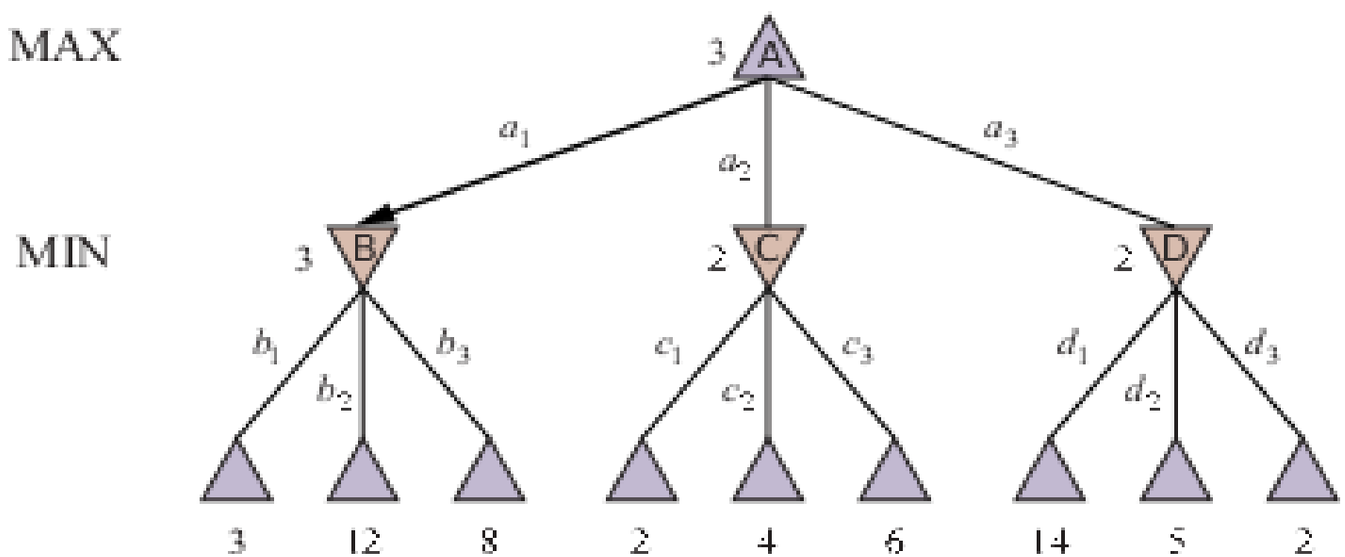
Maxvalue Example Trace



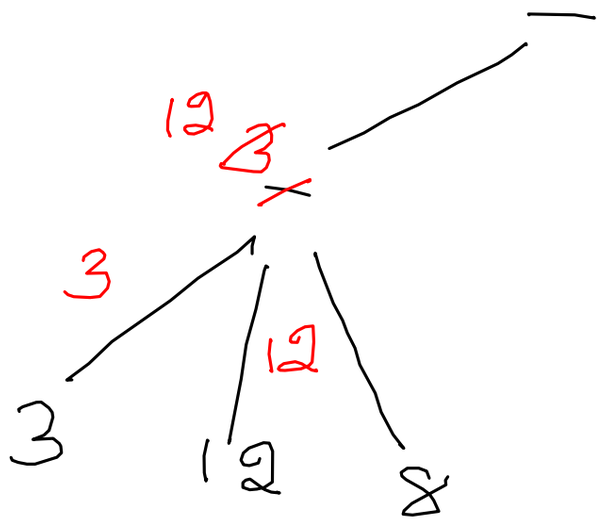


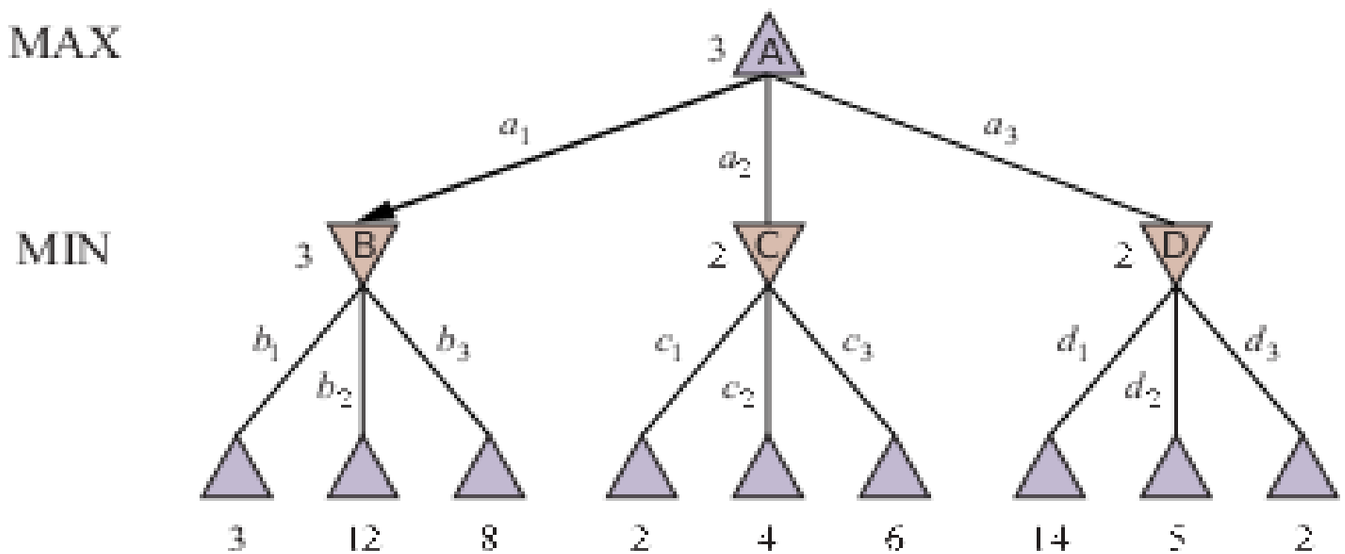
Maxvalue Example Trace



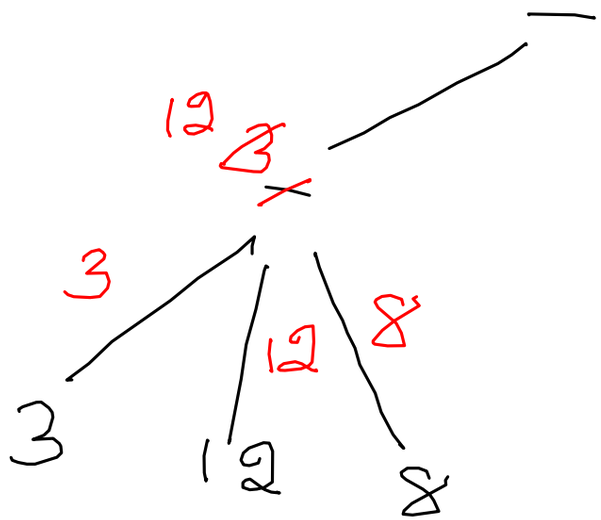


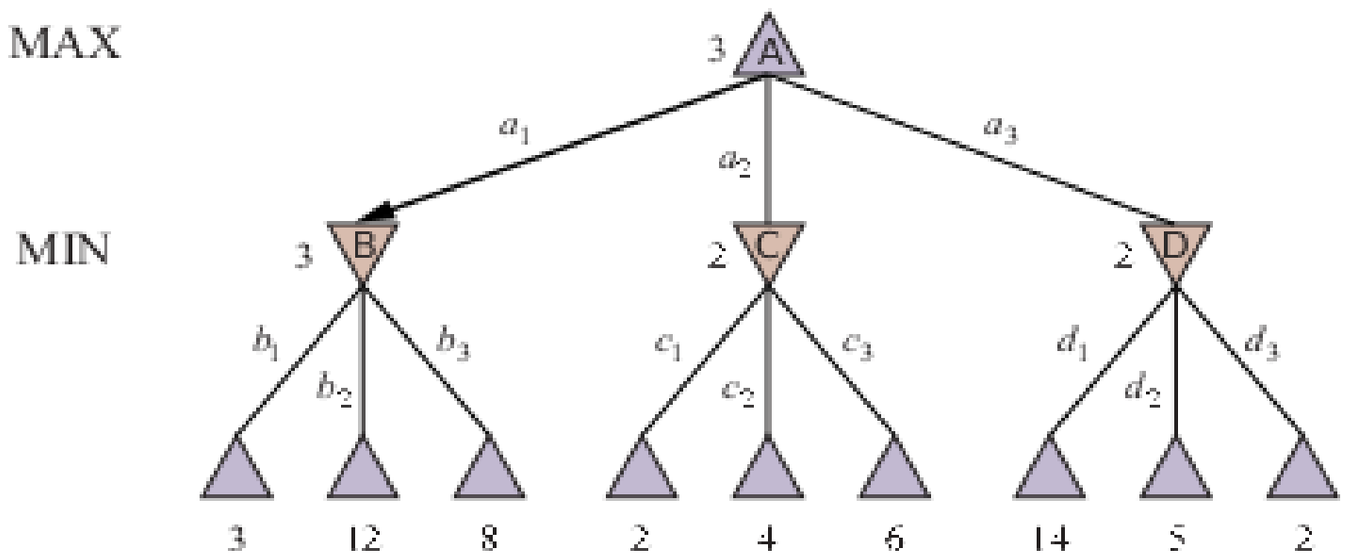
Maxvalue Example Trace



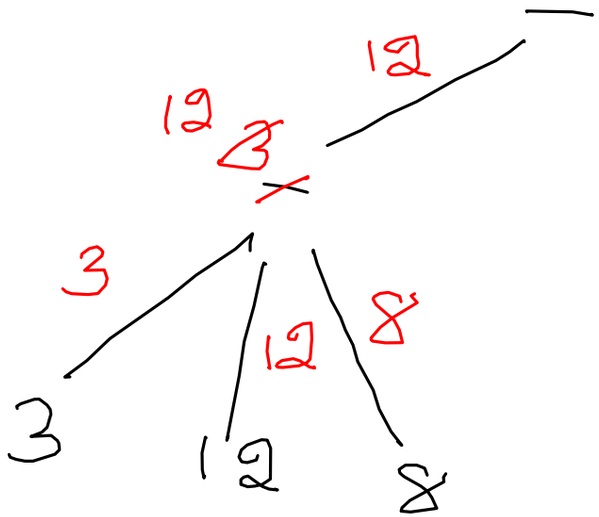


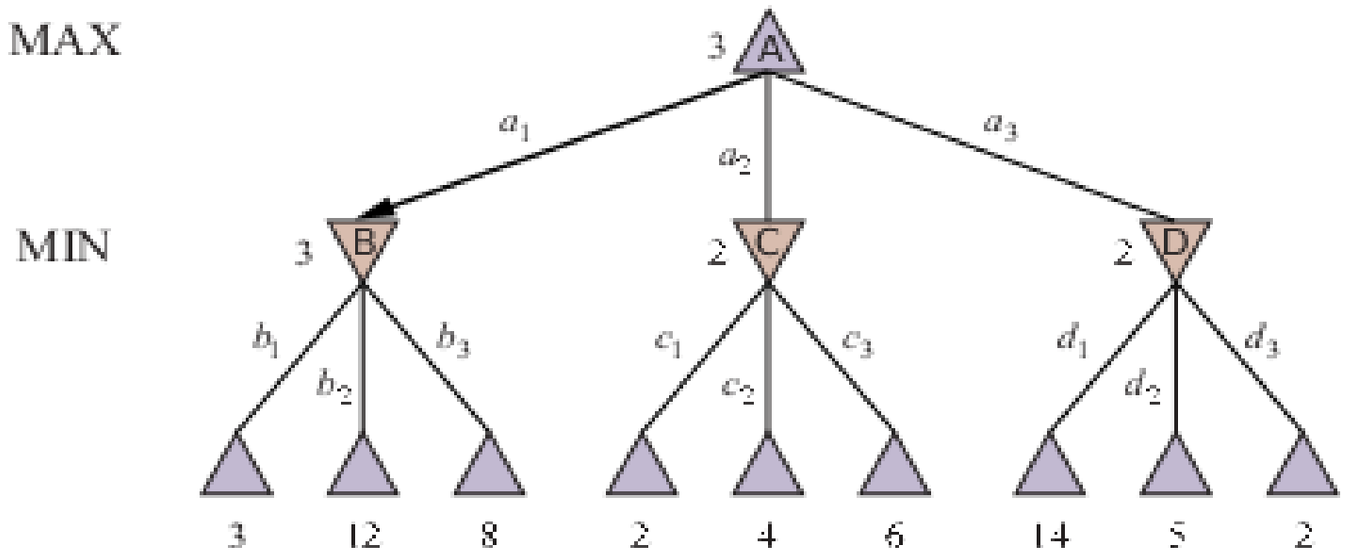
Maxvalue Example Trace



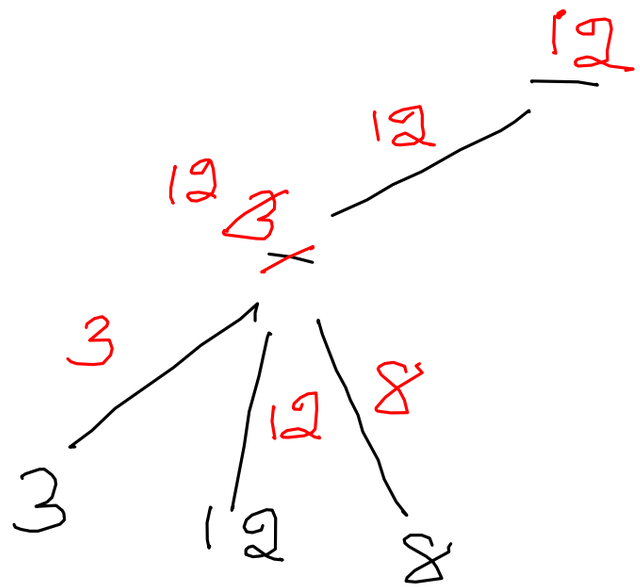


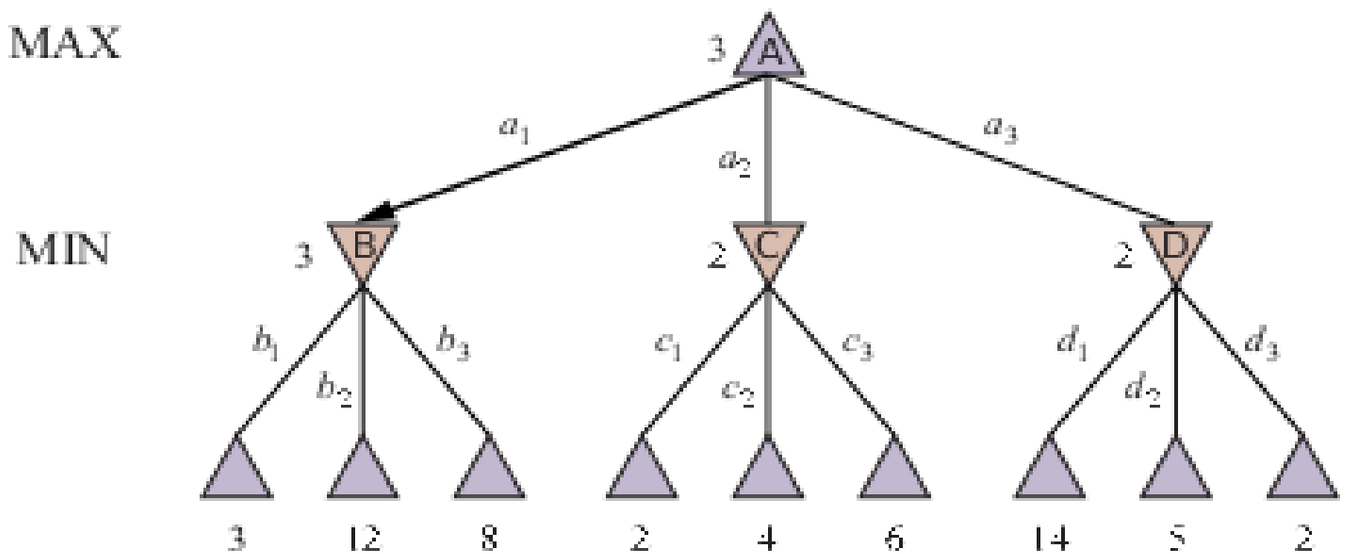
Maxvalue Example Trace



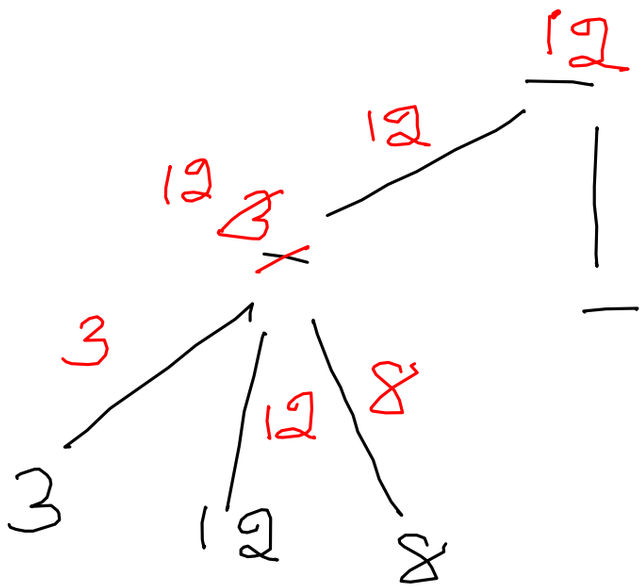


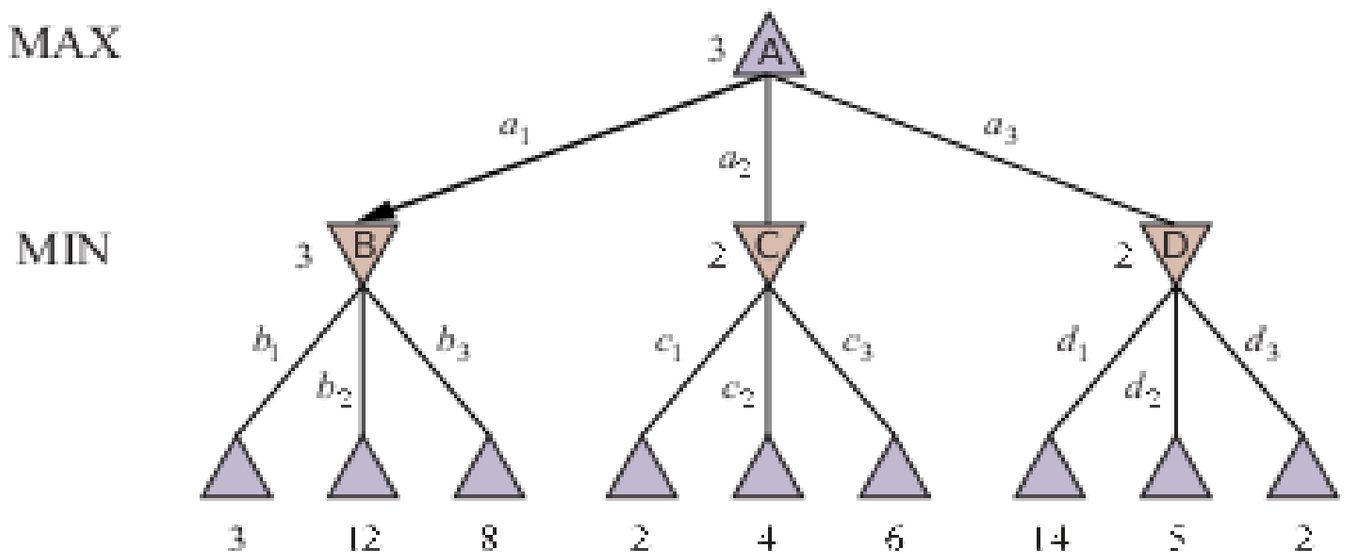
Maxvalue Example Trace



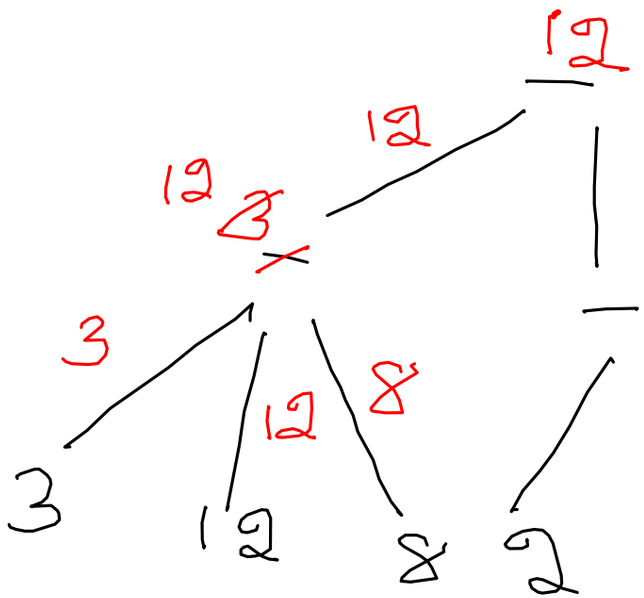


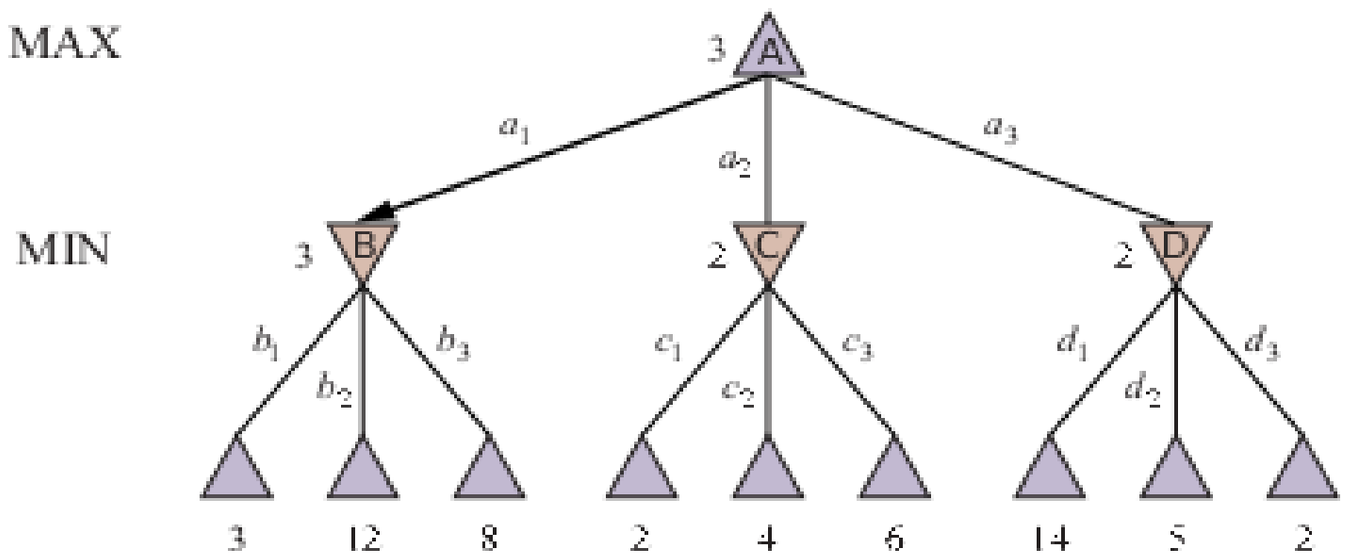
Maxvalue Example Trace



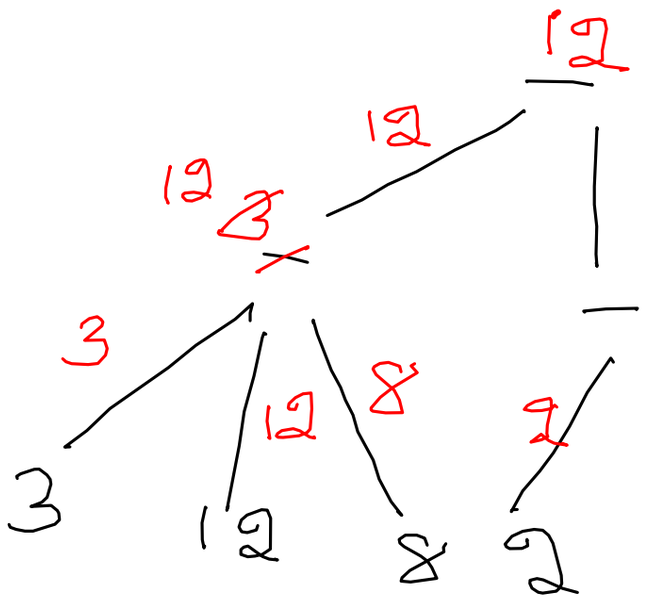


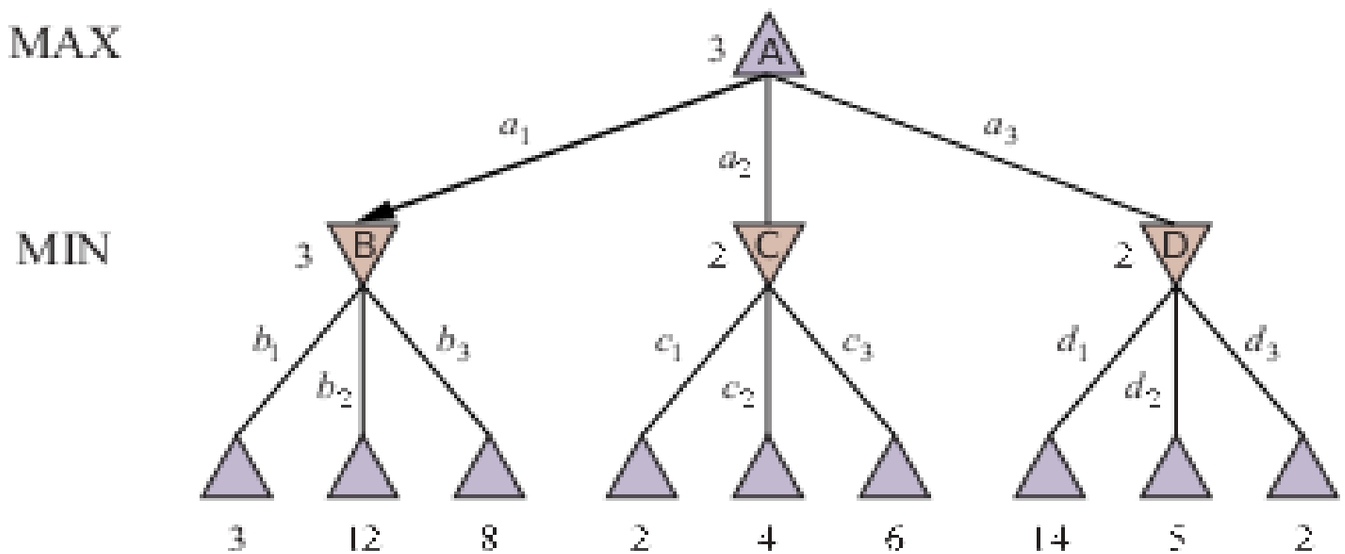
Maxvalue Example Trace



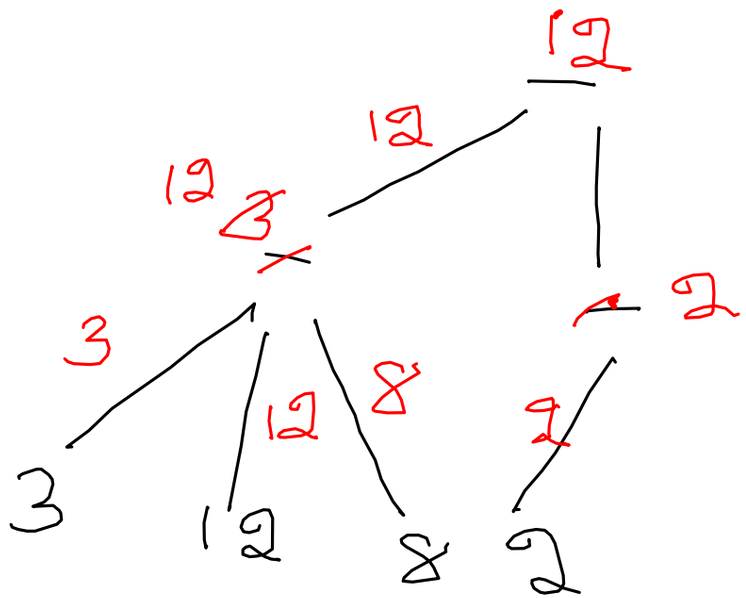


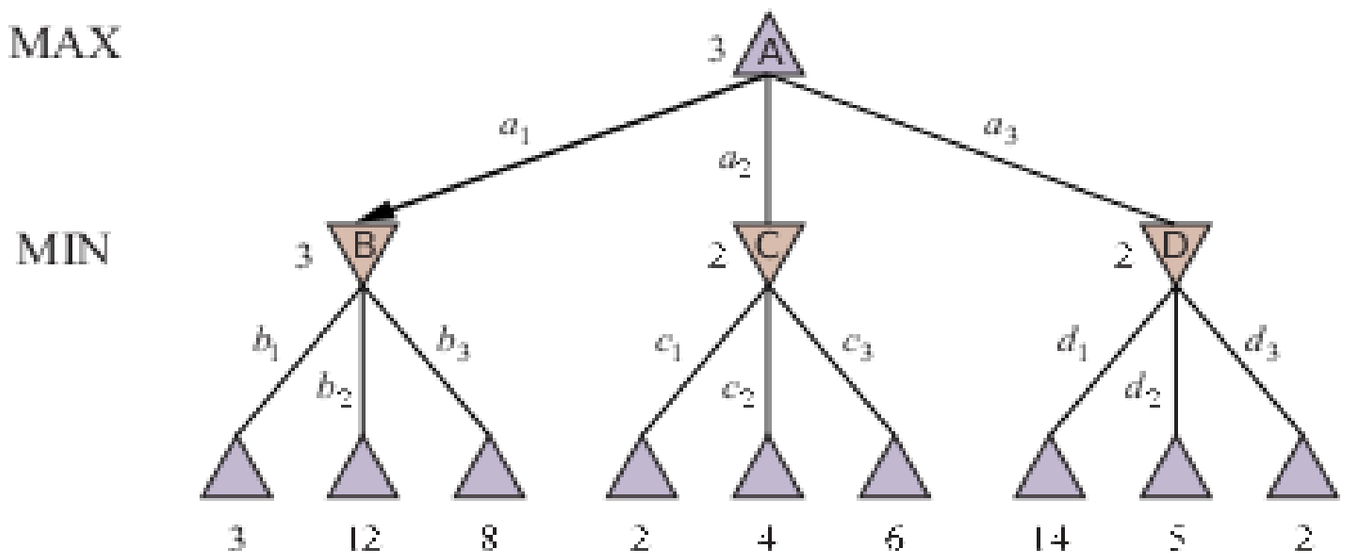
Maxvalue Example Trace



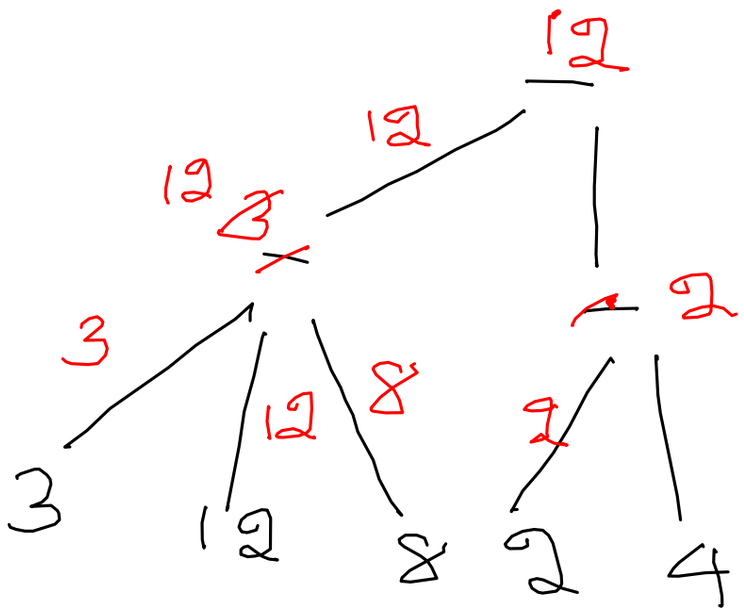


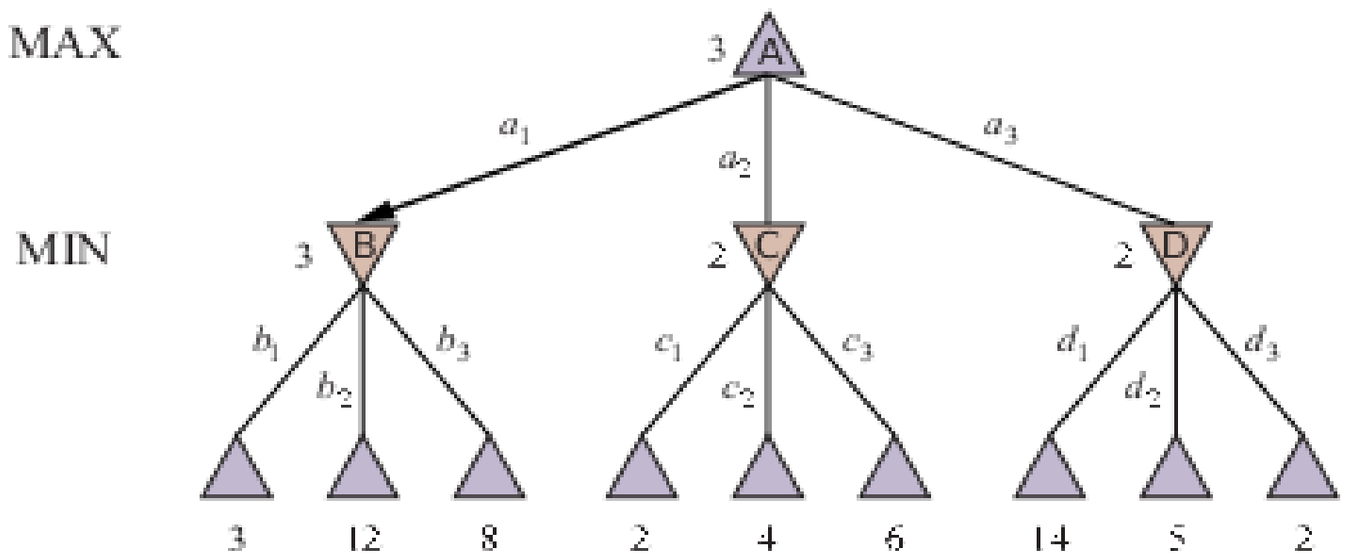
Maxvalue Example Trace



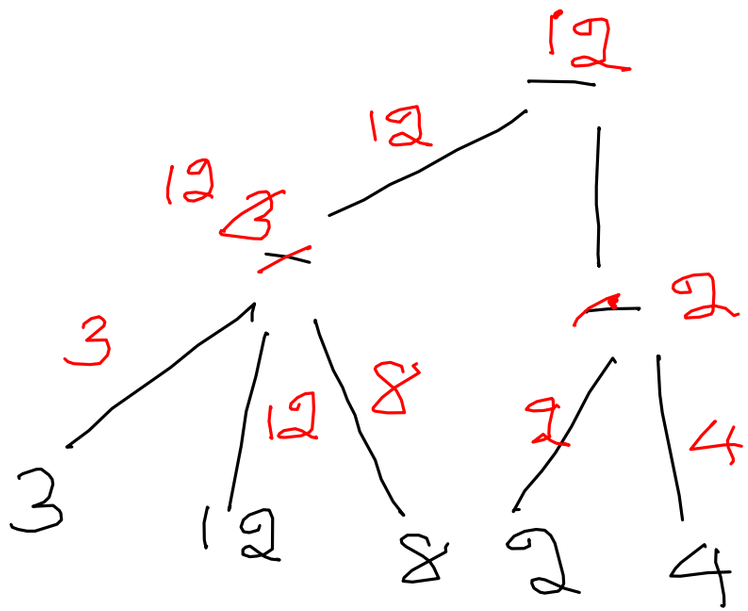


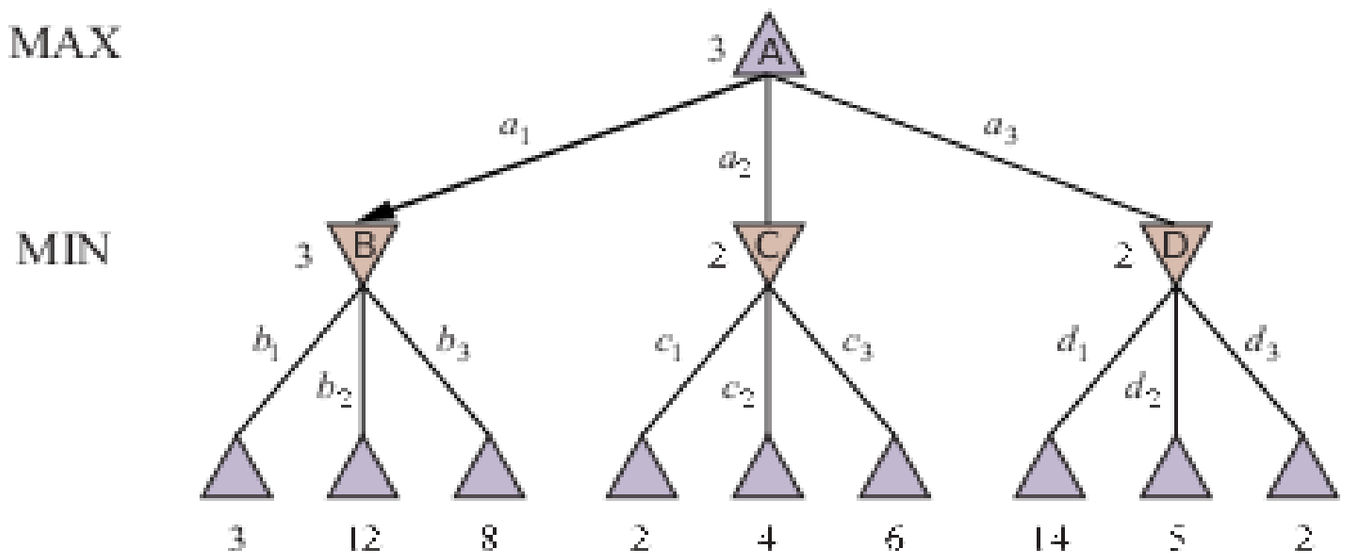
Maxvalue Example Trace



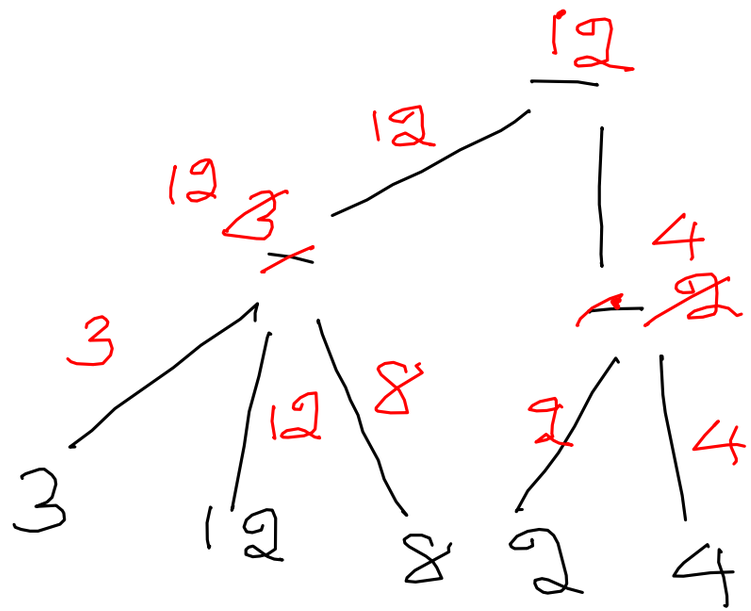


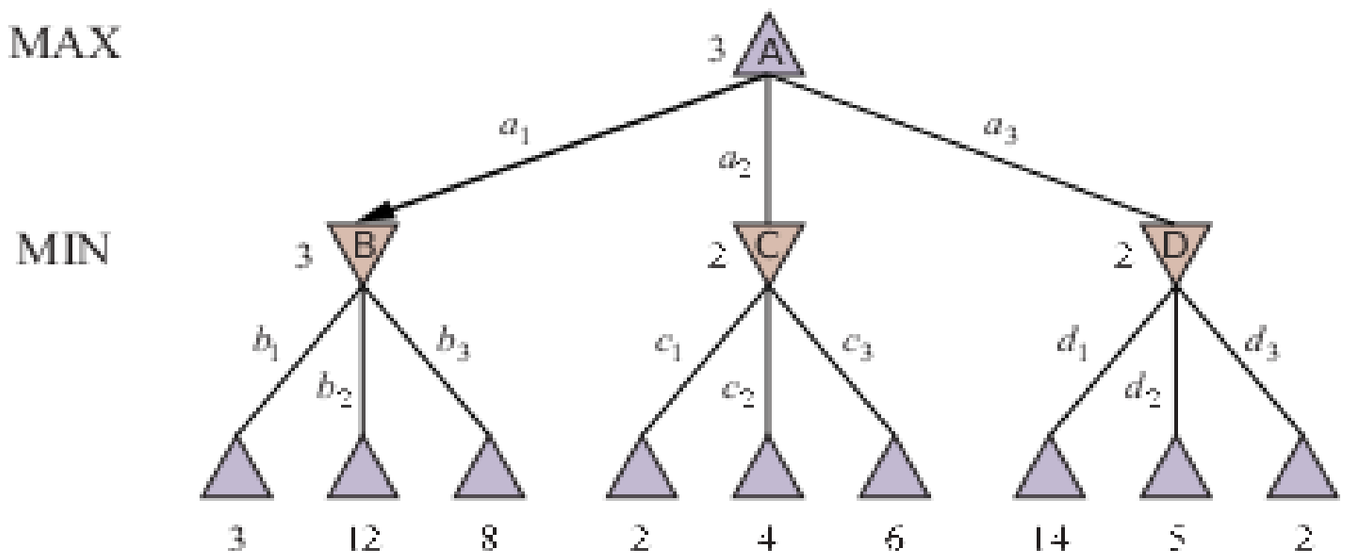
Maxvalue Example Trace



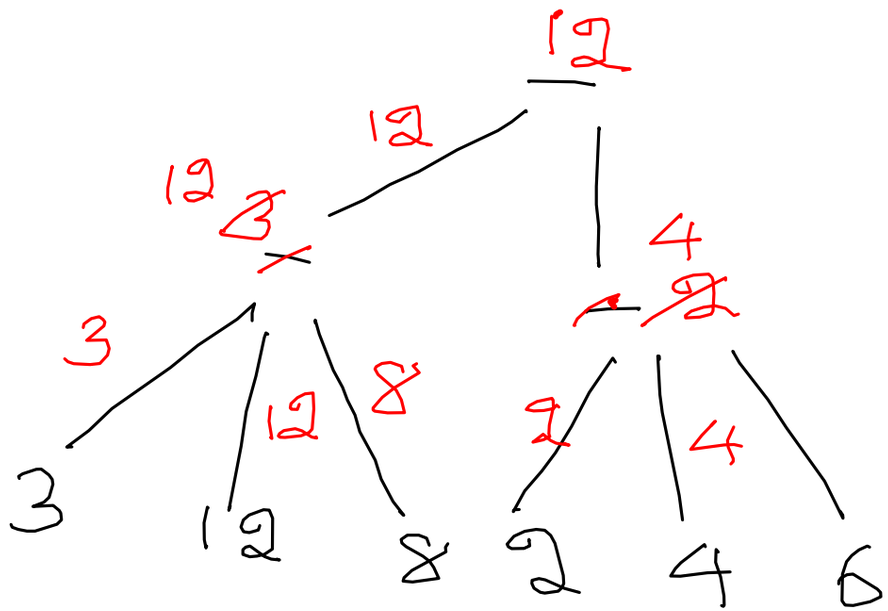


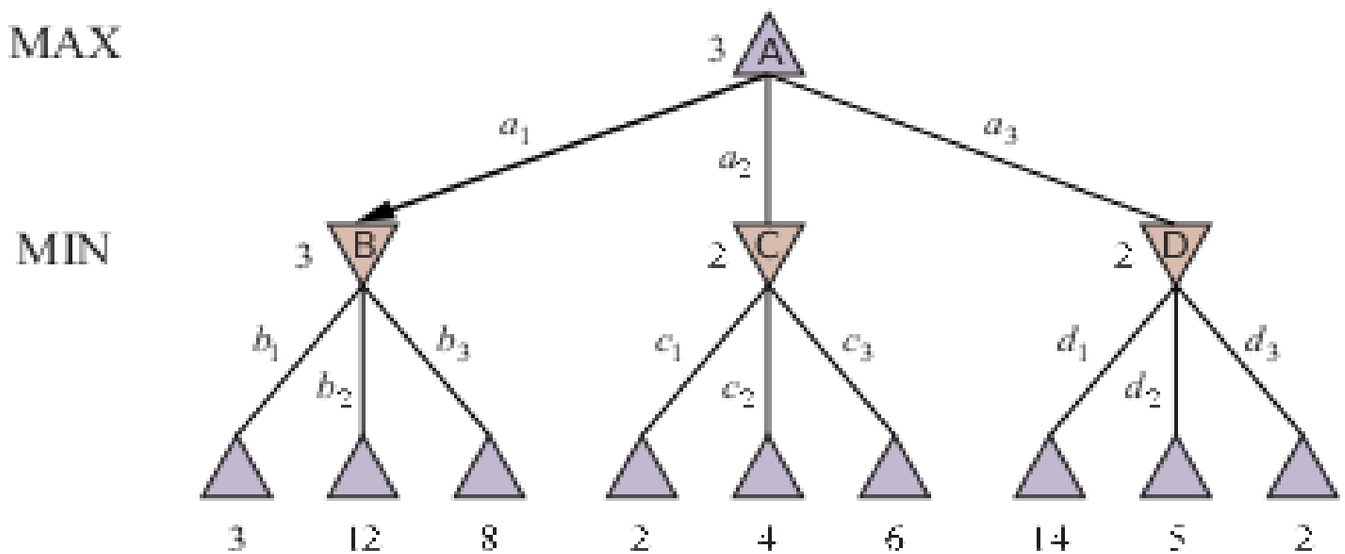
Maxvalue Example Trace



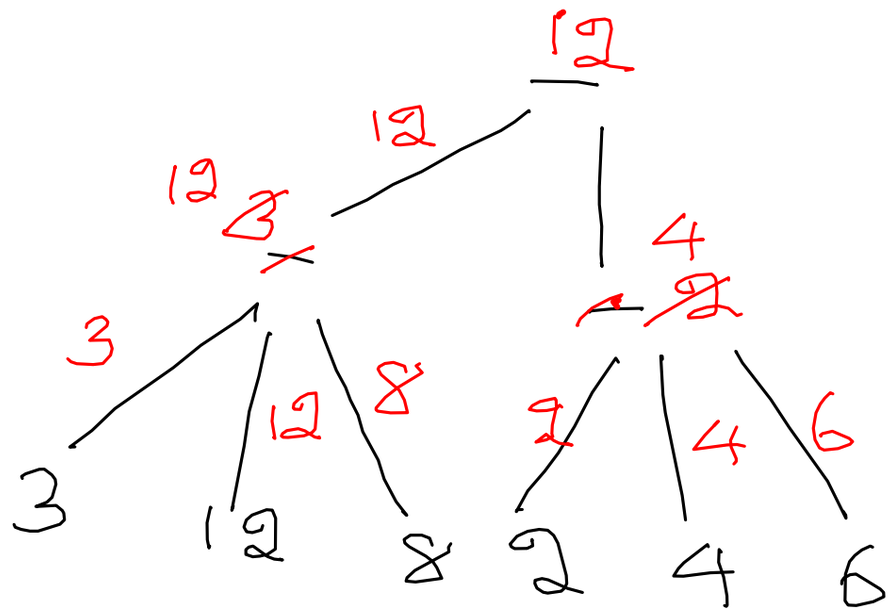


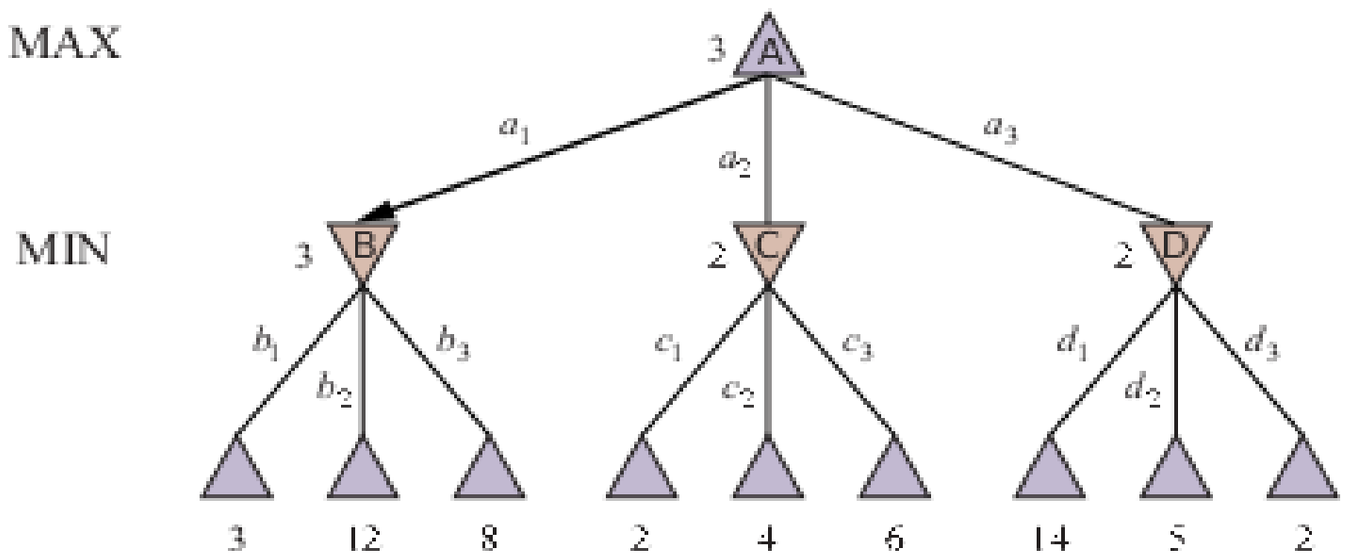
Maxvalue Example Trace



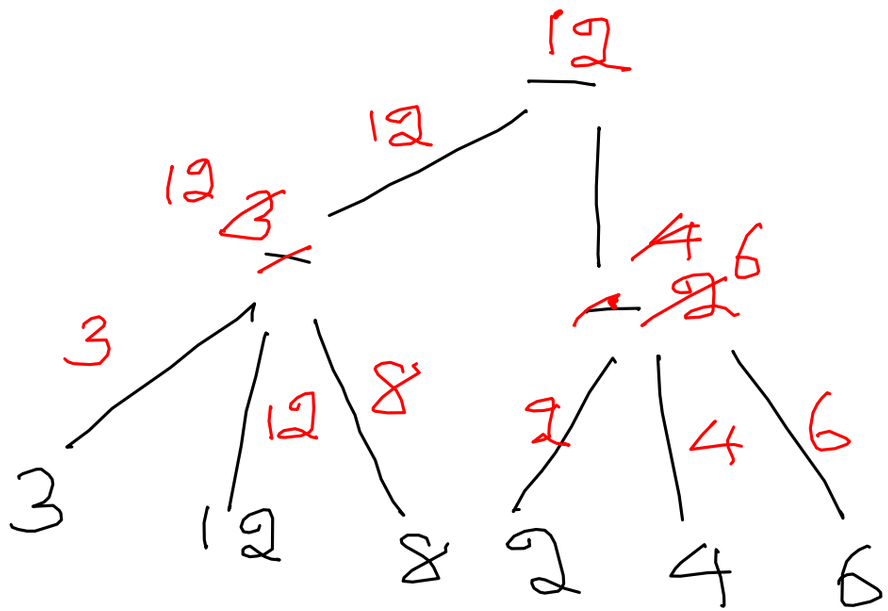


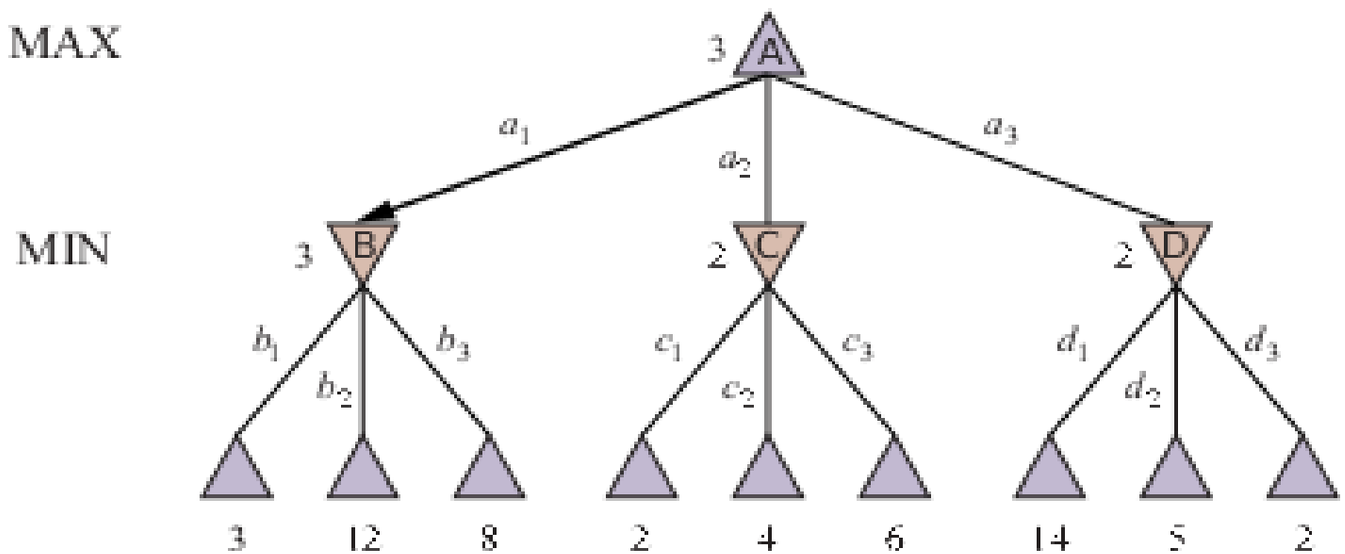
Maxvalue Example Trace



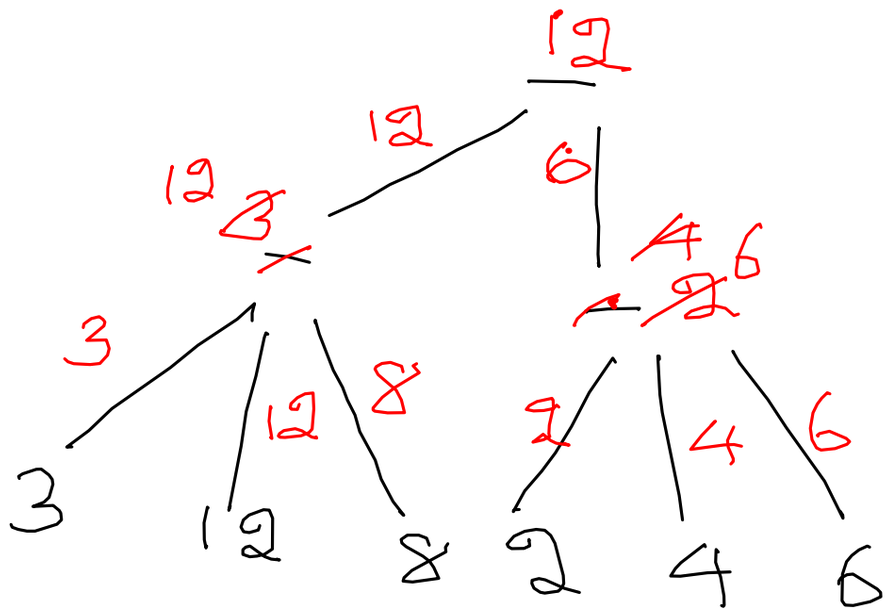


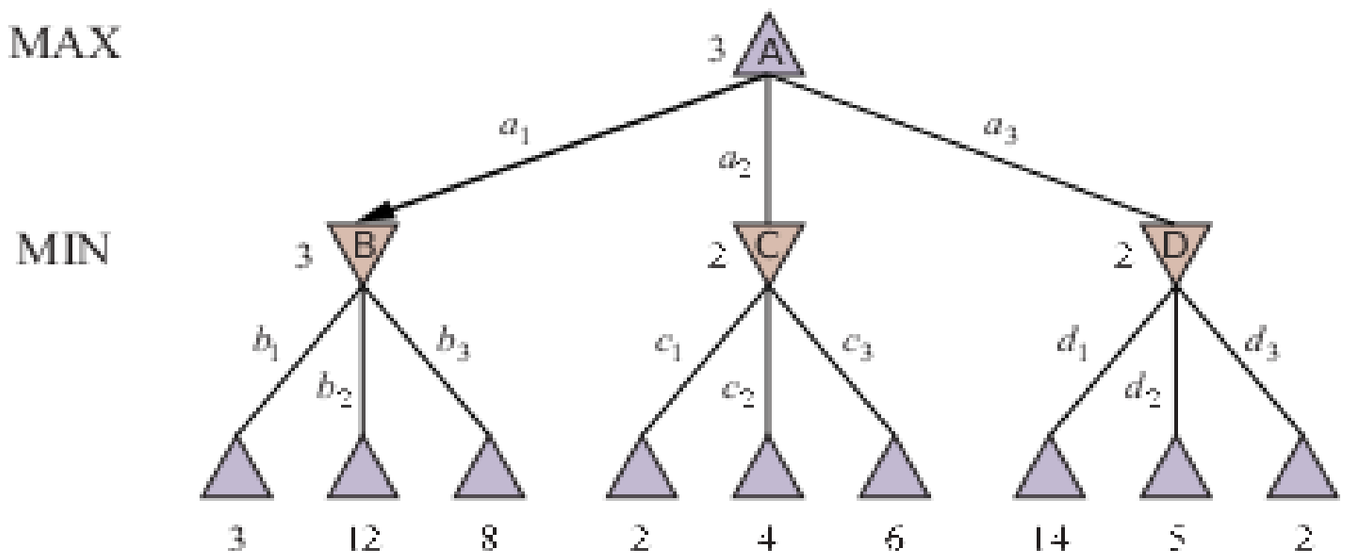
Maxvalue Example Trace



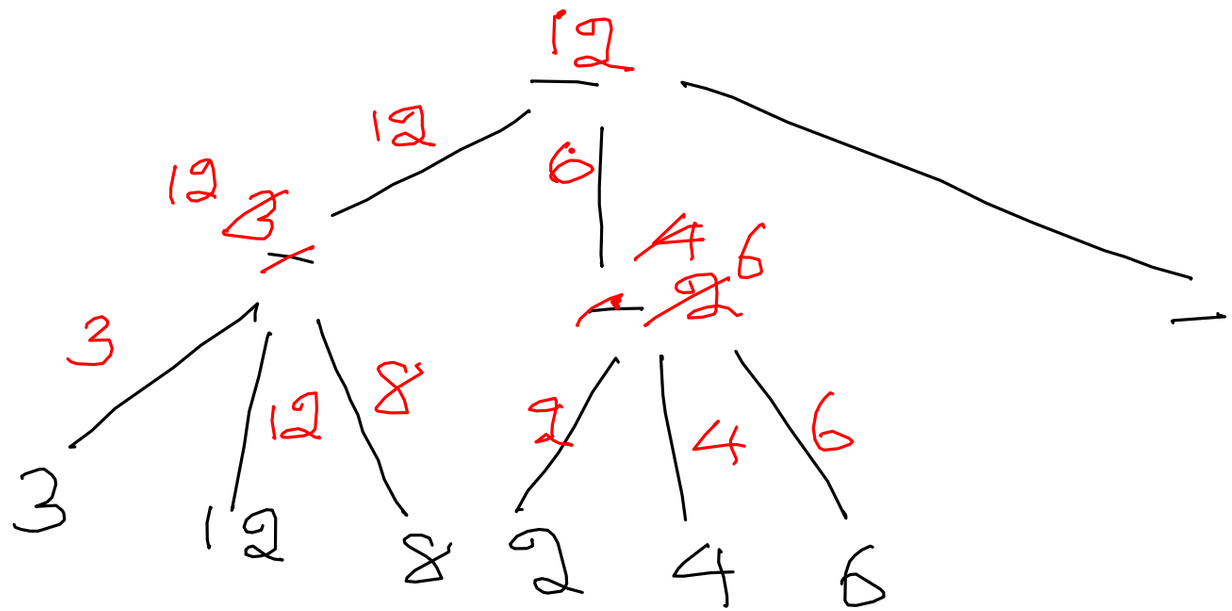


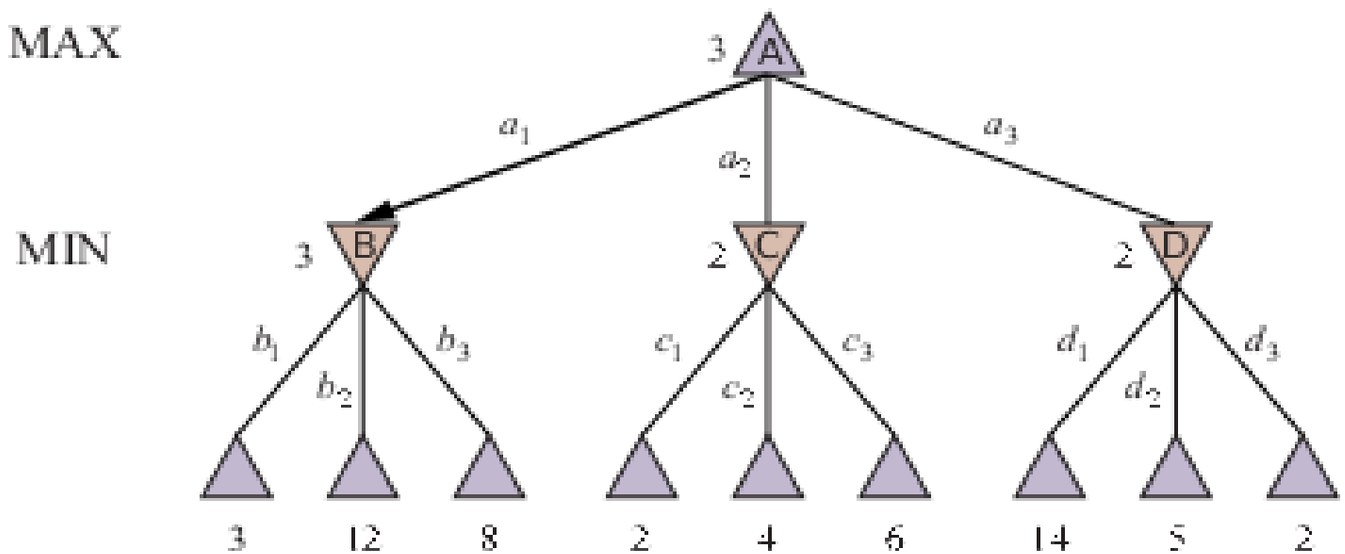
Maxvalue Example Trace



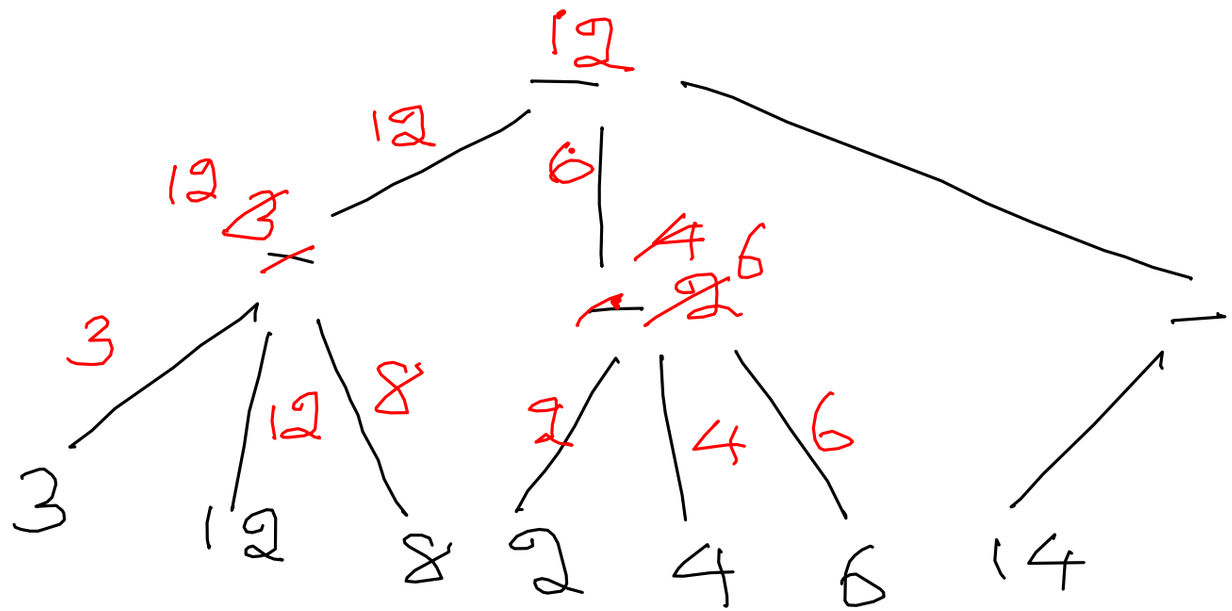


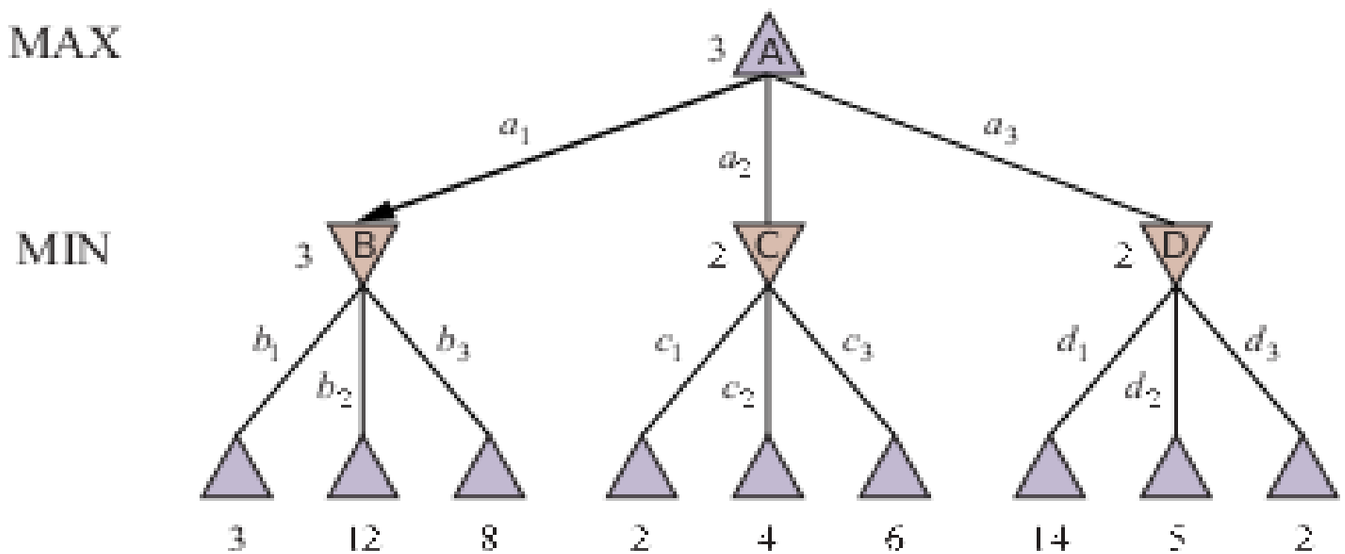
Maxvalue Example Trace



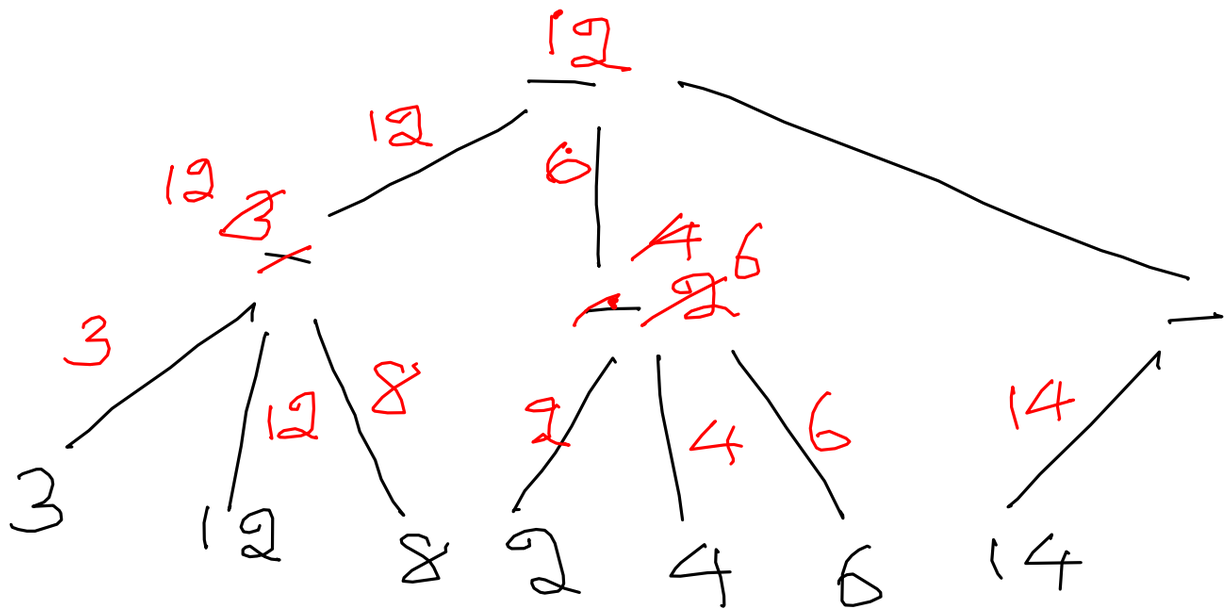


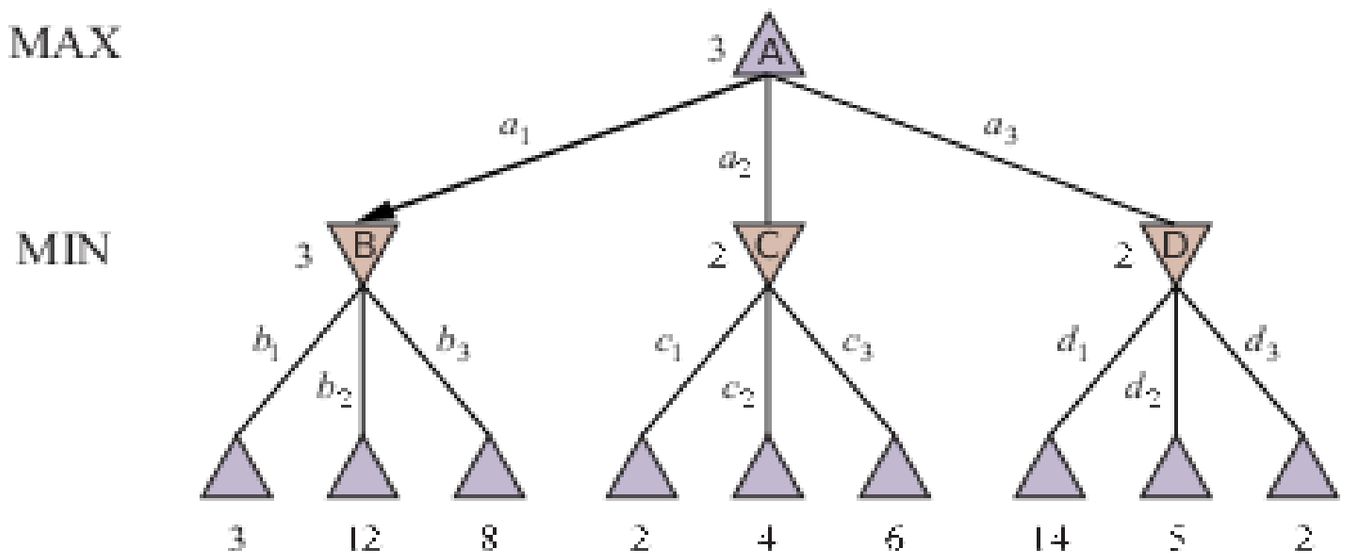
Maxvalue Example Trace



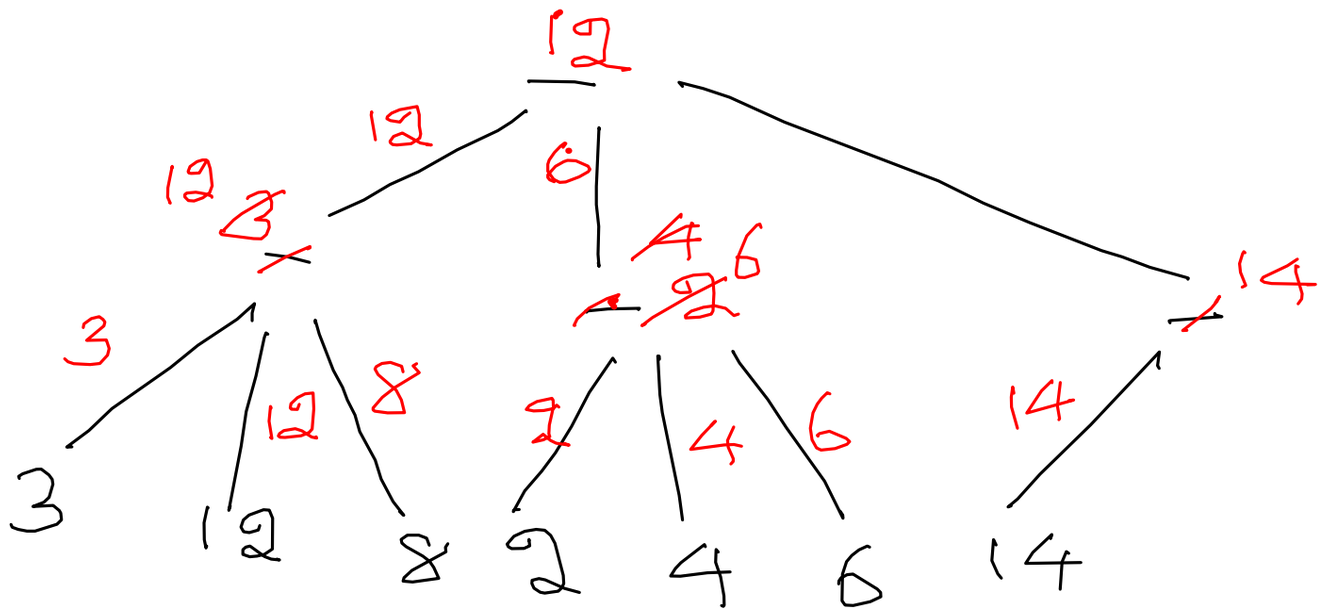


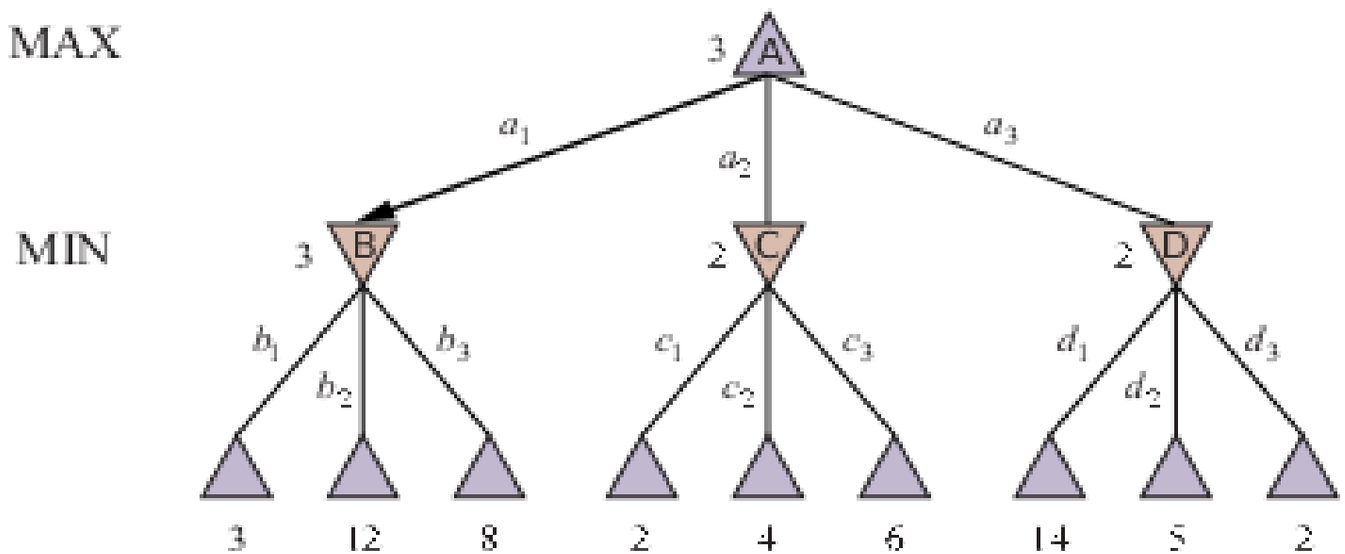
Maxvalue Example Trace



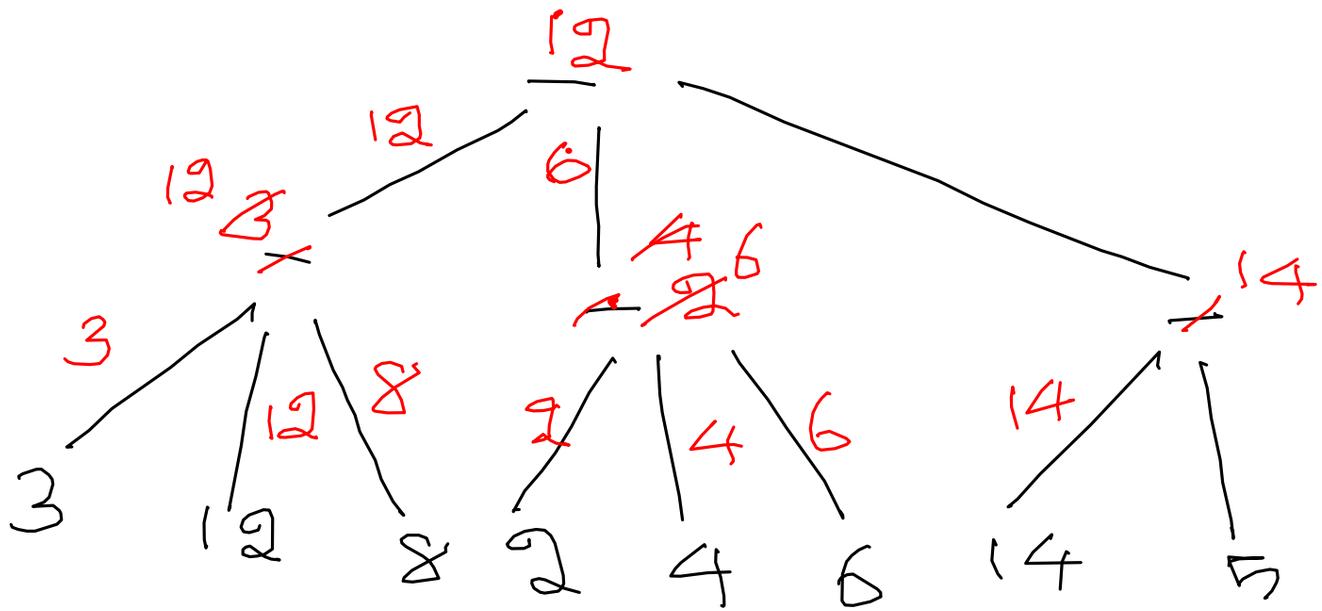


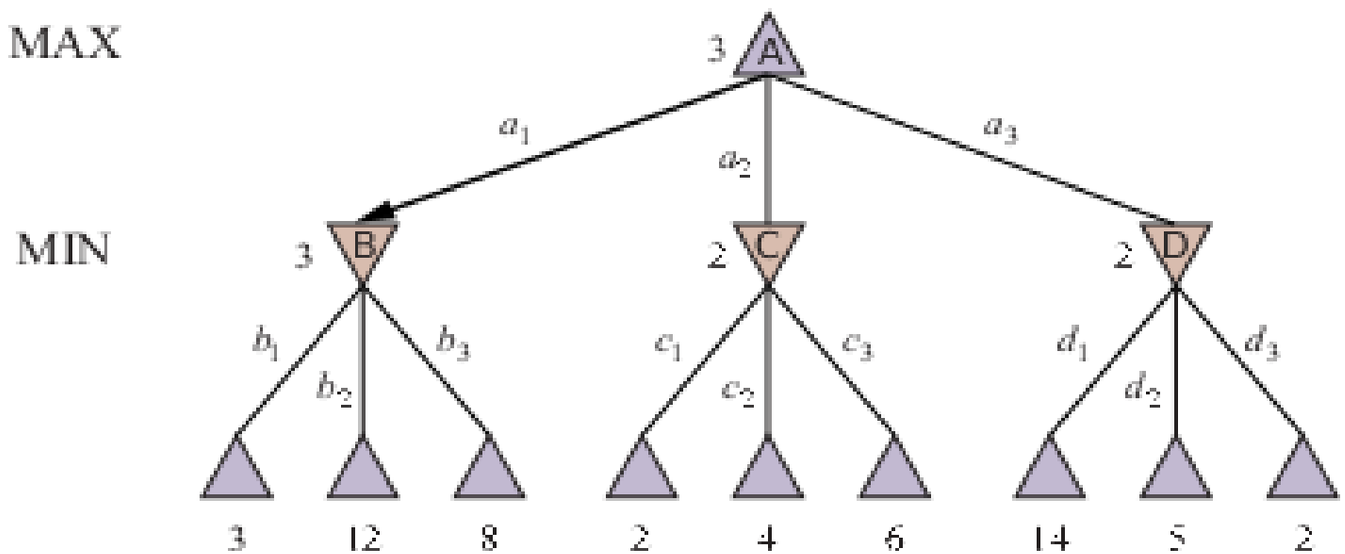
Maxvalue Example Trace



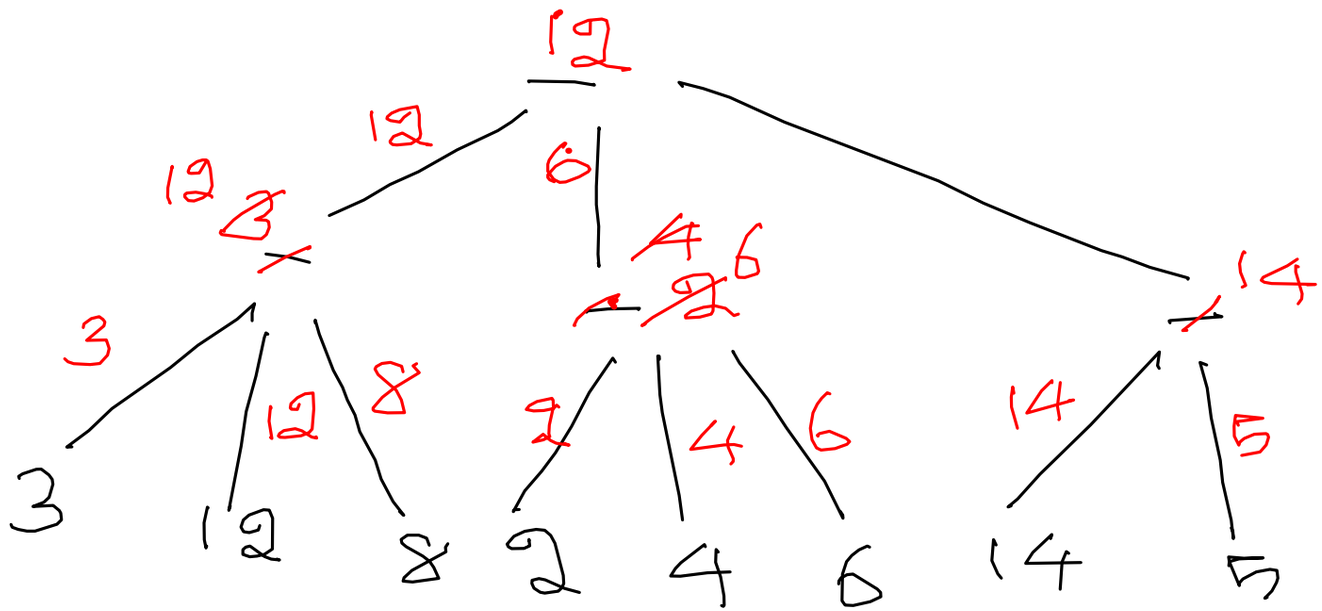


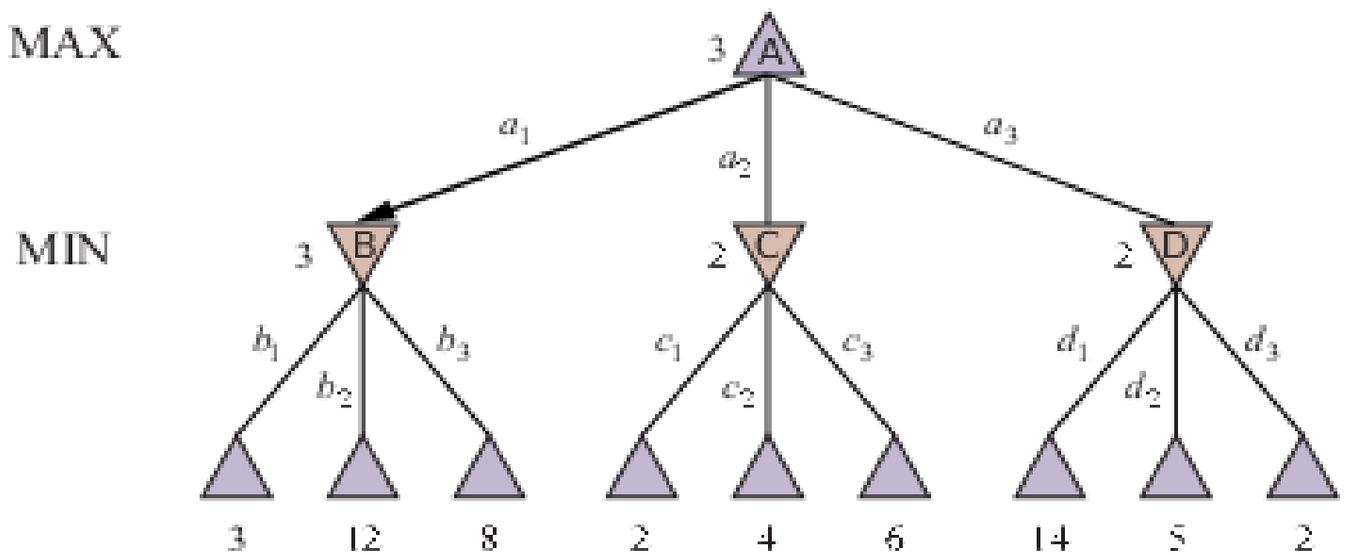
Maxvalue Example Trace



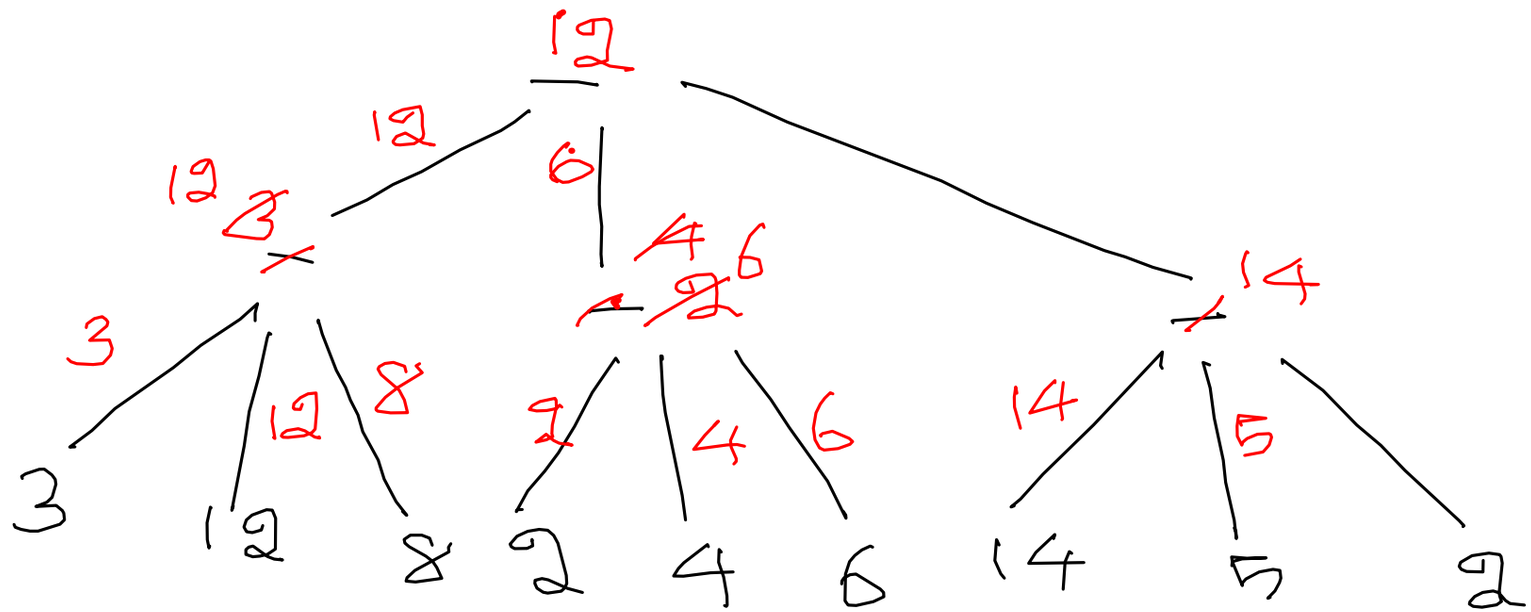


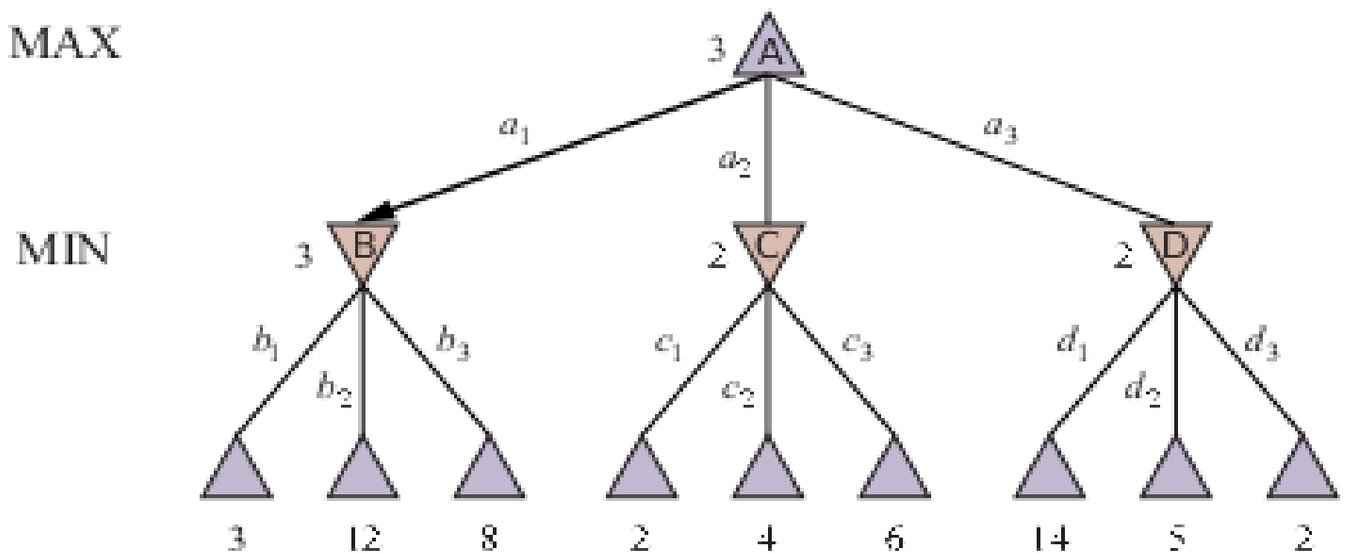
Maxvalue Example Trace



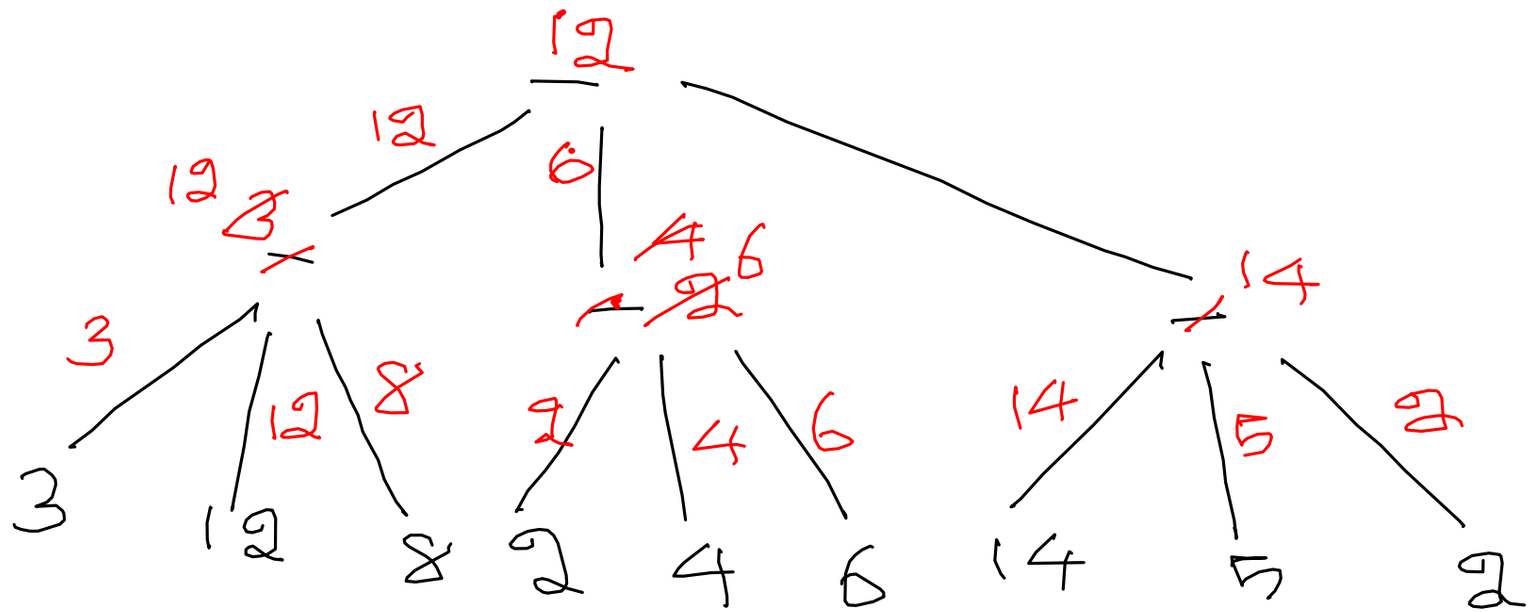


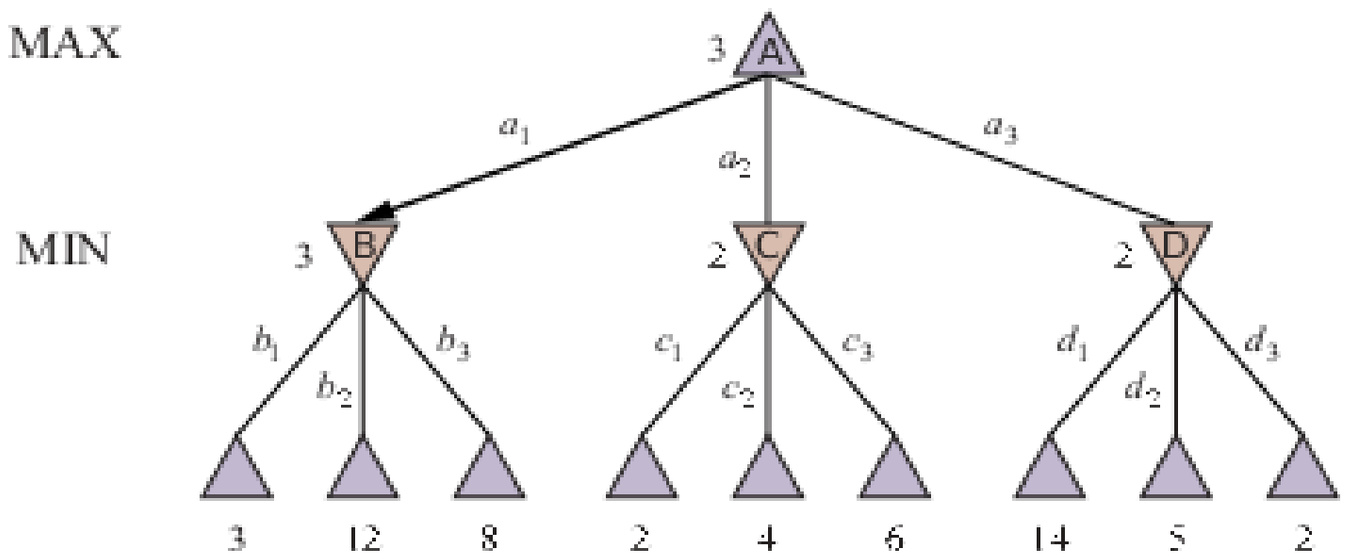
Maxvalue Example Trace



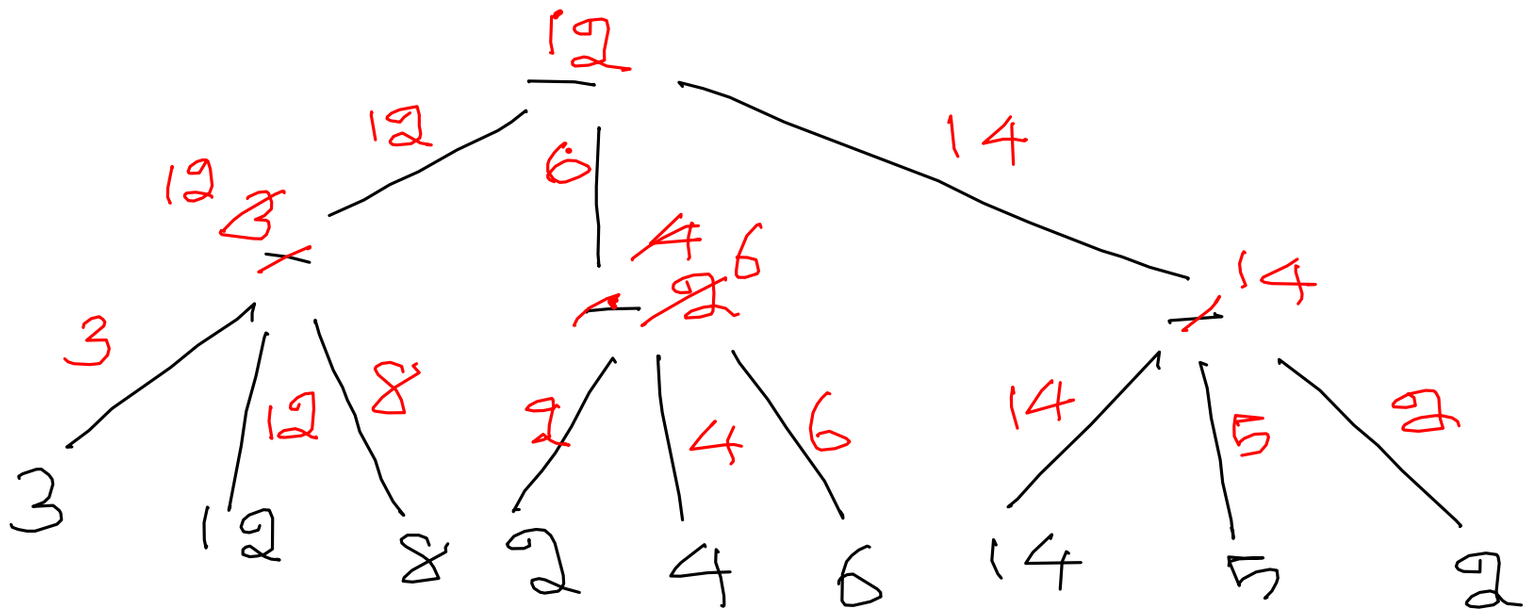


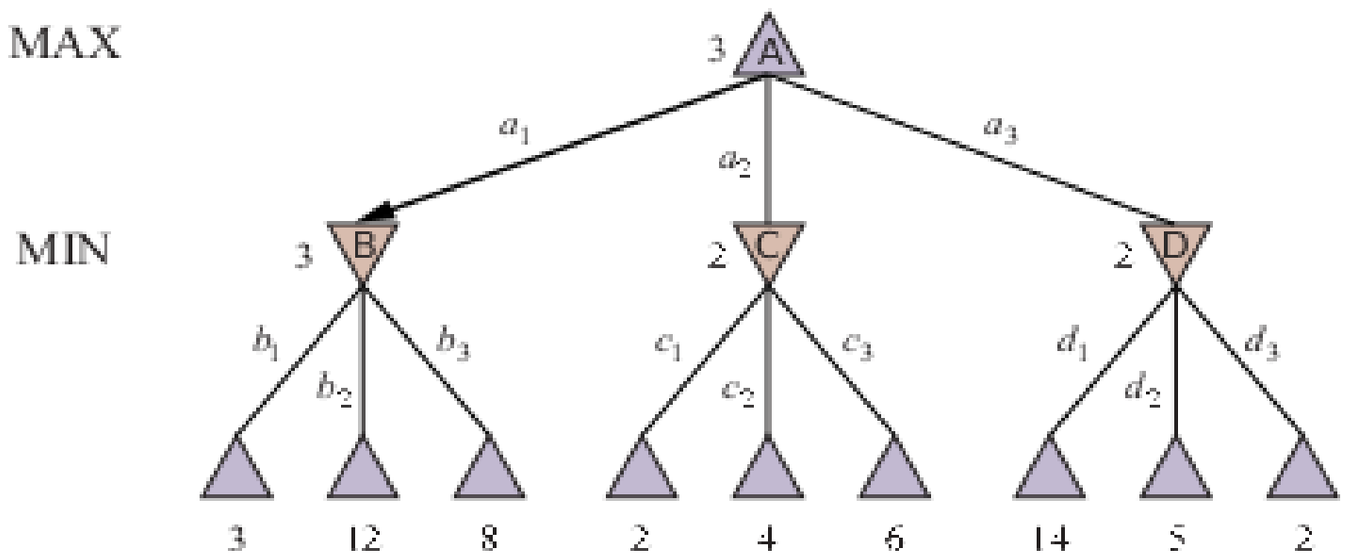
Maxvalue Example Trace



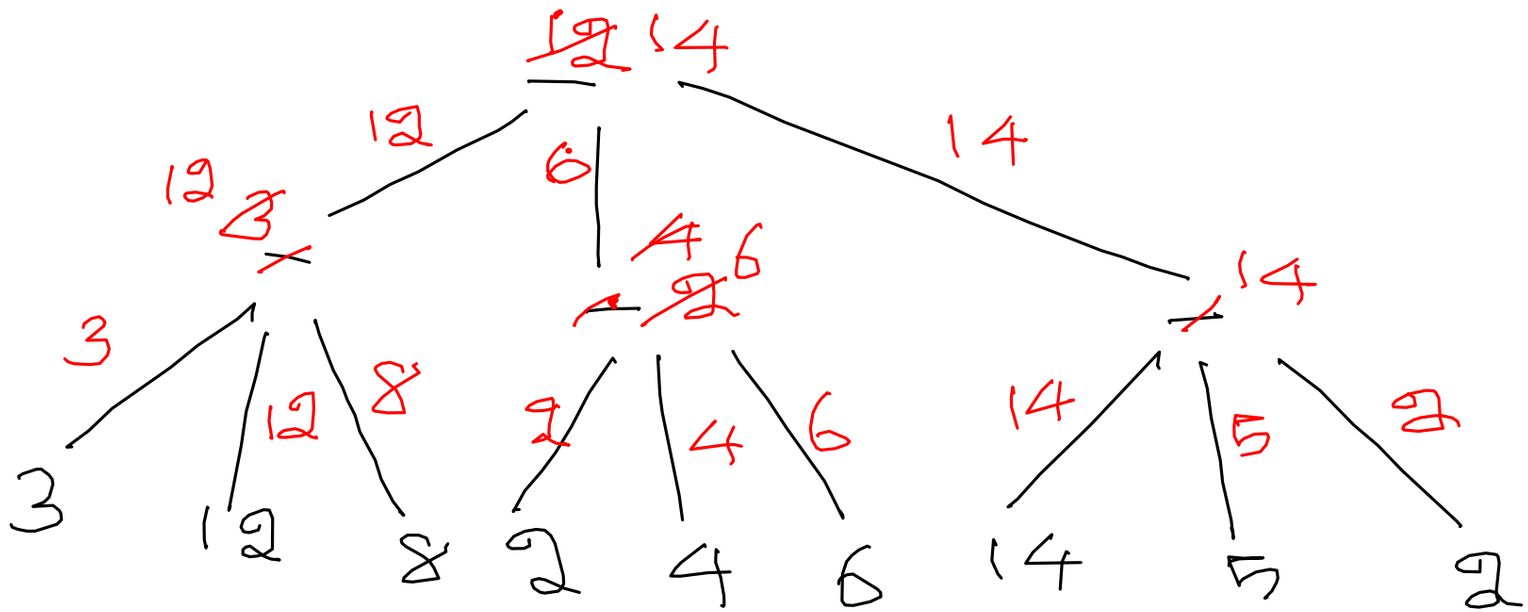


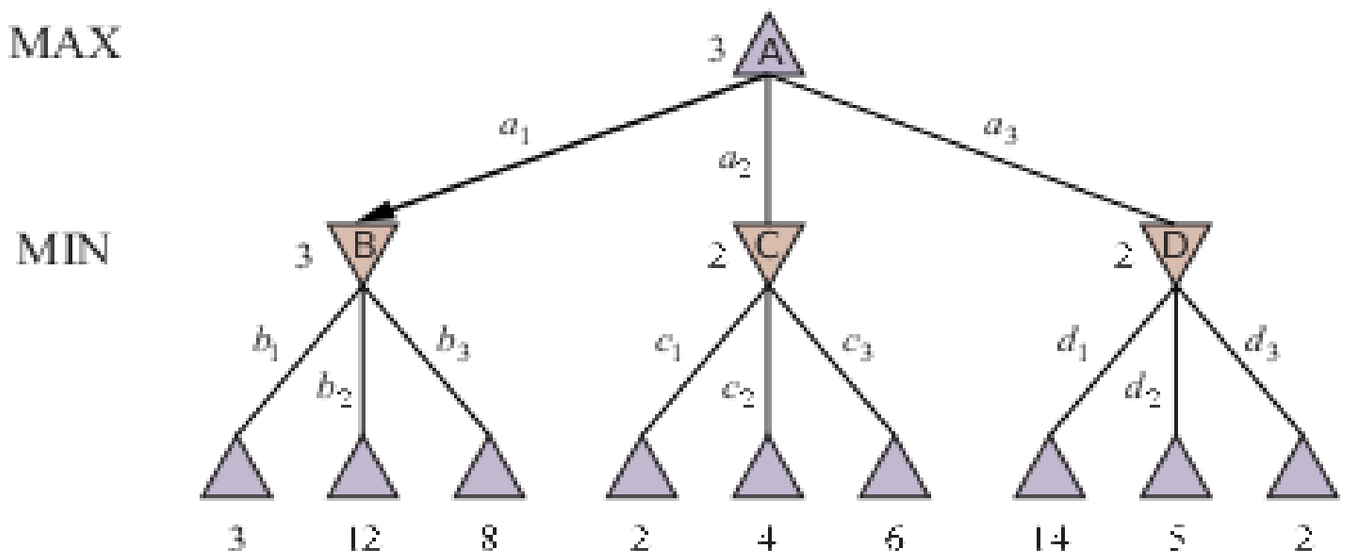
Maxvalue Example Trace



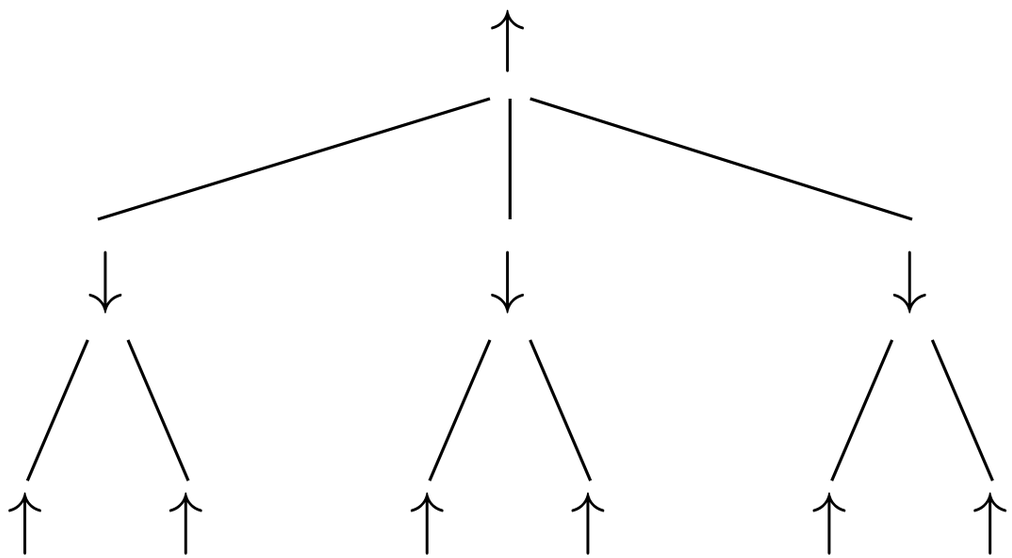


Maxvalue Example Trace





Minimax Value of States



MAX player ↑'s move in this state
$\text{Value}(n) = \max_{c \in \text{Successors}(n)} \text{Value(c)}$

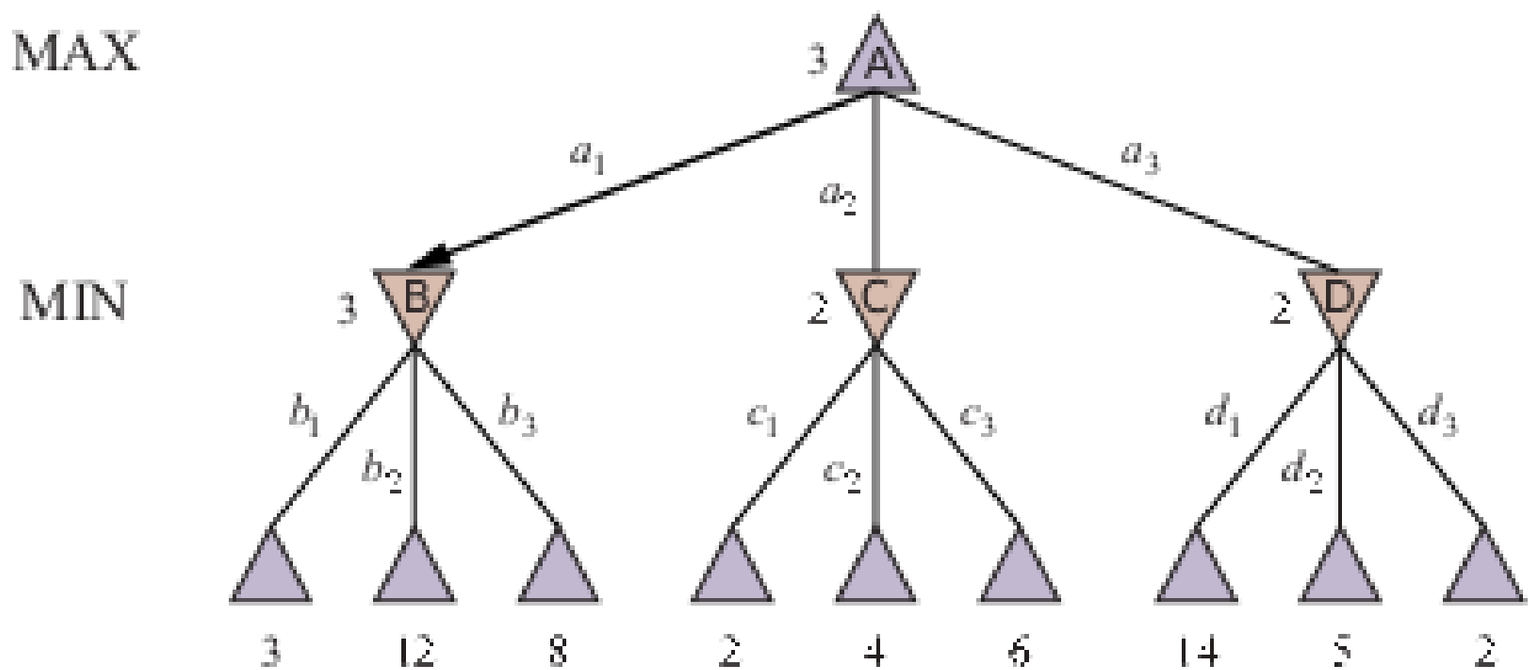
MIN player ↓'s move in this state
$\text{Value}(n) = \min_{c \in \text{Successors}(n)} \text{Value(c)}$

Terminal state
$\text{Value}(n) = \text{Utility}(n)$

Minimax Value and Minimax Action in a Node

- Utility of being in that state
- Assuming both the players play optimally from there to the end of the game

$$\text{Minimax}(n) = \begin{cases} \text{Utility}(n) & \text{if Leaf?}(n) \\ \max_{c \in \text{Successors}(n)} \text{Minimax(c)} & n \text{ is a MAX node} \\ \min_{c \in \text{Successors}(n)} \text{Minimax(c)} & n \text{ is a MIN node} \end{cases}$$



Two-ply Game Tree

- A two-ply game tree.
  - One move deep
  - One move = two half-moves (half-move = ply)
- "MAX nodes" – MAX's turn to move
- "MIN nodes" – MIN's turn to move
- Terminal nodes show the utility values for MAX
- Other nodes labeled with their minimax values
- MAX's best move at the root is a_1
  - leads to the state with the highest minimax value
- MIN's best reply move is b_1
  - leads to the state with the lowest minimax value.

Minimax Idea

- Minimax value of a state from the minimax values of its successors, recursively
- Base case: minimax values of terminal states (leaves)
- As the recursion unwinds, the minimax value are backed up from leaves to the root

Algorithm: Minimax(n)

Input: n a node in a game tree
Output: A pair, (value for the agent in node n, path that gives this value)

```
if n is a leaf node then  return evaluate(n)
else if n is a MAX node then
    max_val ← −∞
    foreach child c of n do
        val ← Minimax(c)
        if max_val < val  then
            max_val ← val
    return max_val
else
    min_val ← ∞
    foreach child c of n do
        val ← Minimax(c)
        if val < min_val then
            min_val ← val
    return min_val
```

Algorithm: Minimax(n)

Input: n a node in a game tree
Output: A pair, (value for the agent in node n, path that gives this value)

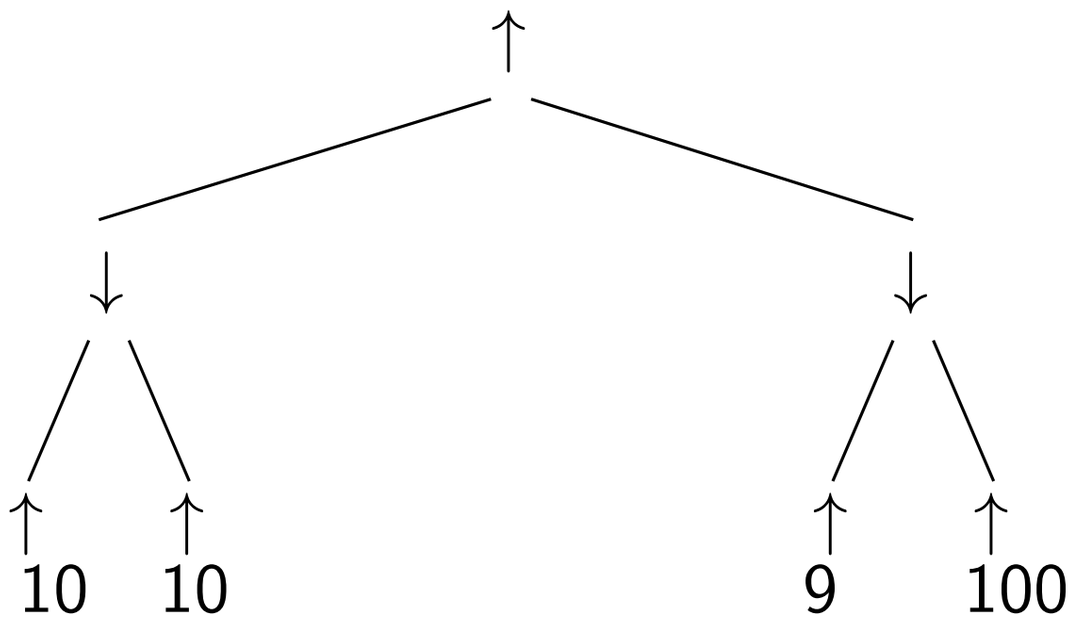
```
if n is a leaf node then  return evaluate(n), [ ]
else if n is a MAX node then
    max_val ← −∞
    foreach child c of n do
        val, path ← Minimax(c)
        if max_val < val  then
            max_val, max_path ← val, [c] + path
    return max_val, max_path
else
    min_val ← ∞
    foreach child c of n do
        val, path ← Minimax(c)
        if val < min_val  then
            min_val, min_path ← val, [c] + path
    return min_val, min_path
```

Minimax Algorithm Analysis

- DFS
- Height of the tree $= m$, Legal moves $= b$
- Time complexity $= O(b^m)$, impractical
- Space complexity $= O(bm)$ or $O(m)$

Minimax Performance

- Optimal play for MAX assumes MIN also plays optimally
- Maximize the worst-case outcome for MAX
- If MIN does not play optimally?
- There are better strategies against suboptimal opponents
  - Do worse against optimal opponents.



Resource Limits

- Problem: In realistic games, cannot search to leaves!
- Solution: Depth-limited search
  - Search only to a limited depth in the tree
  - Replace terminal utilities with an evaluation function for non-terminal positions
- Guarantee of optimal play is gone
- More plies makes a BIG difference
- Use iterative deepening for an anytime algorithm

Evaluation Functions

- Evaluation functions are always imperfect
- Evaluation functions score non-terminals in depth-limited search
- Ideal function: returns the actual minimax value of the position
- In practice: typically weighted linear sum of features:

$$\text{eval}(n) = w_1 f_1(n) + w_2 f_2(n) + \ldots + w_k f_k(n)$$

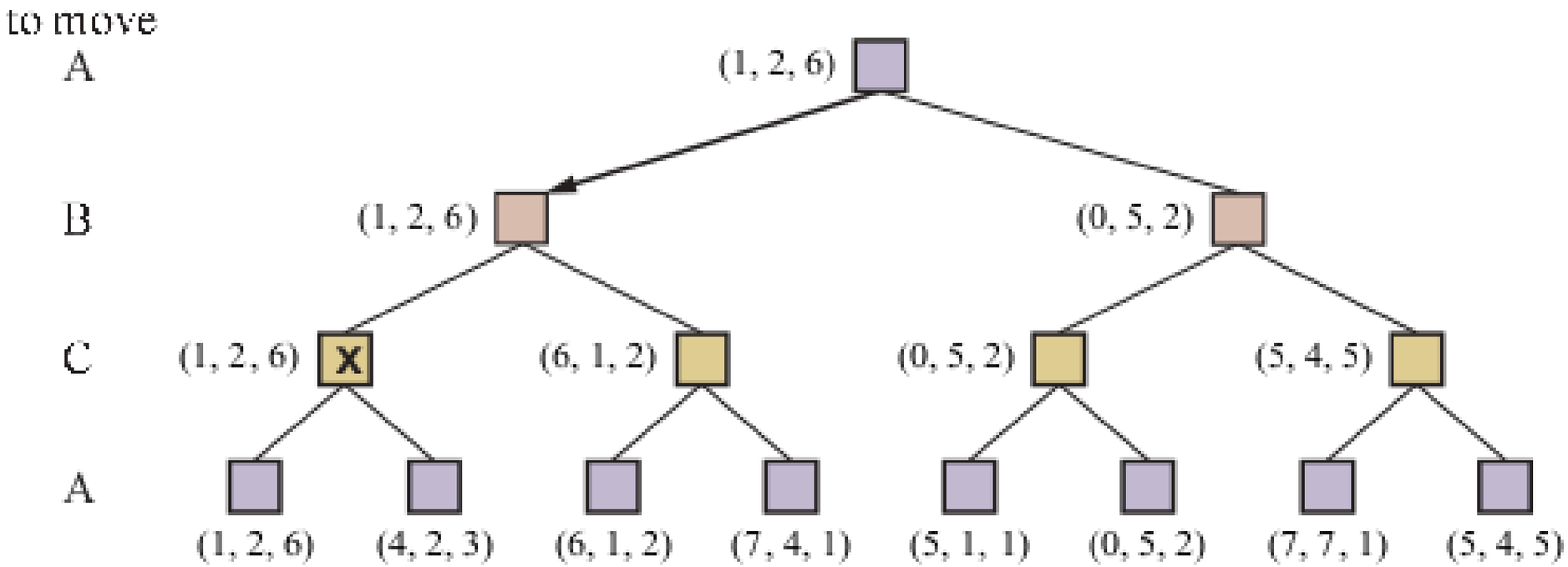
- e.g. $f_1(n) =$(num white queens $-$ num black queens), etc.

Optimal Decisions in Multiplayer Games

- Each node labeled with a vector of n values, from the viewpoint of n players

$$(v_A, v_B, v_C)$$

- Terminal nodes – `Utility` returns n - vector
- Backed-up value of node n = utility vector of the successor state with the highest value for the player choosing at n.



Minimax with Alpha-Beta Pruning

Alpha-Beta Pruning

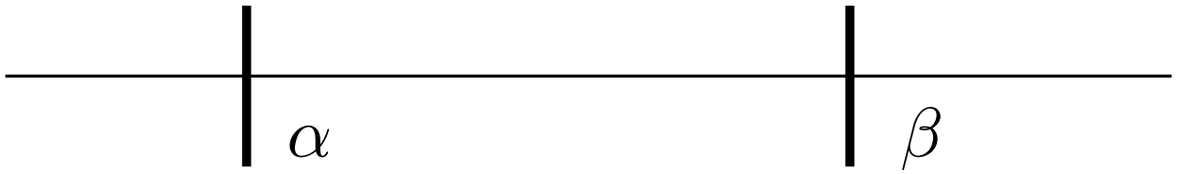
- Game tree: number of states exponential in the depth of the tree
- Is it possible to compute the correct minimax decision without looking at every node
- Pruning: eliminate subtrees from consideration
- Alpha-beta pruning: eliminate branches that cannot influence the final decision

α and β

- At any node, a subtree of the game tree has already been traversed.
- In the traversed subtree, the MAX player can achieve a maximum value of max_sofar (α) and there is a path to get it.
- The MIN player can achieve a minimum value of min_sofar (β) and there is a path to get it.
- As more of the game tree is traversed, max_sofar (α) can increase and min_sofar (β) can decrease.
- It is impossible that max_sofar (α) for the MAX player is greater than the min_sofar (β) for the MIN player.

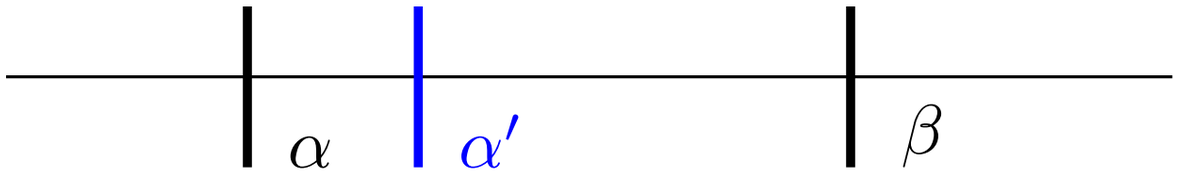
$$\text{max_sofar} \leq \text{min_sofar}$$

$$\alpha \leq \beta$$

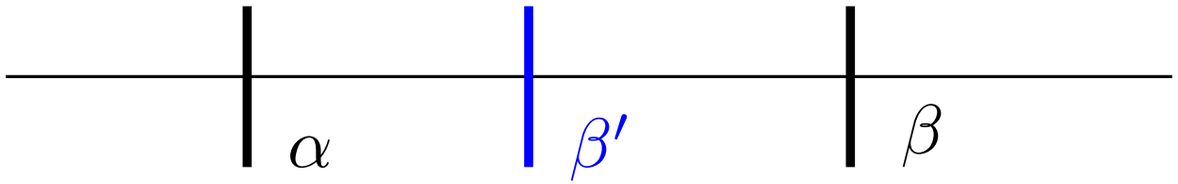


Improving α and β

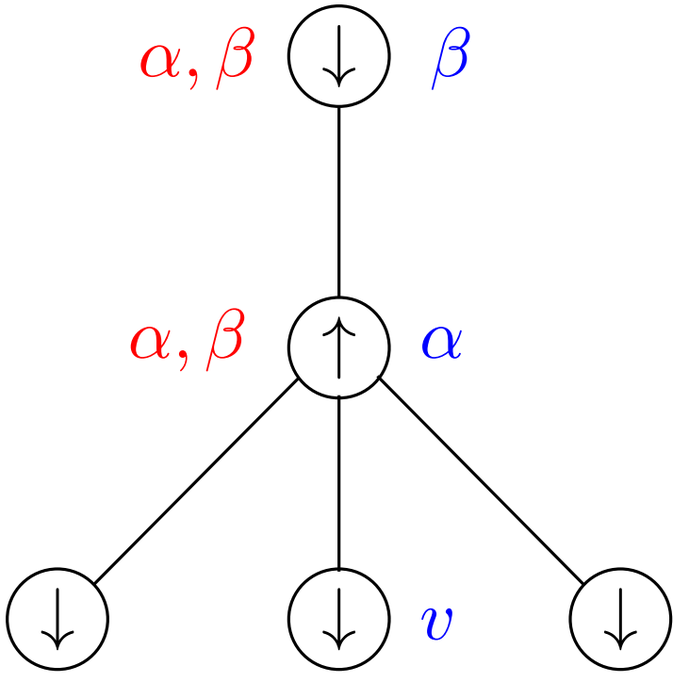
- Each node receives as arguments α and β of the subtree traversed so far.
- A MAX node may find a path with better maximum value through its subtrees and returns an updated α'.
- The returned α' should be in the range $[\alpha, \beta]$, inclusive, where α and β are the arguments received at the node.

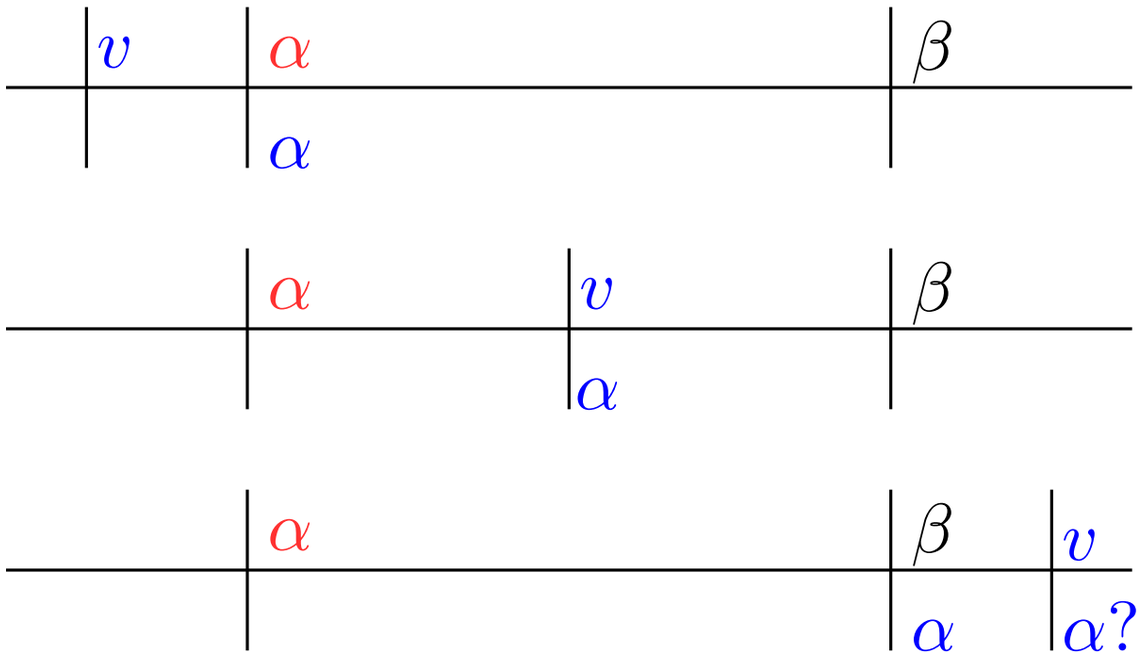


- A MIN node, after traversing its subtrees, returns an updated β.
- The returned β' should be in the range $[\alpha, \beta]$, inclusive, where α and β are the arguments received at the node.



Updating α in a MAX Node



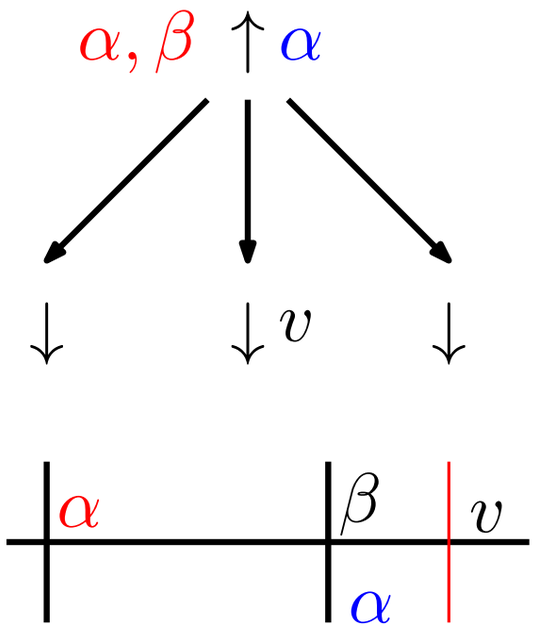


if $v < \alpha$ **then** // $v < \alpha$
  $\alpha \leftarrow \alpha$
else if $v < \beta$ **then** // $\alpha \leq v < \beta$
  $\alpha \leftarrow v$
else // $\beta \leq v$
  $\alpha \leftarrow \beta$

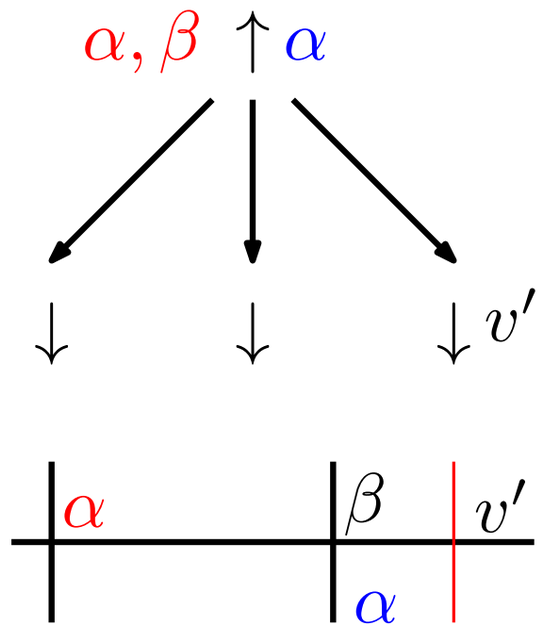
$\equiv$

if $\alpha < v$ **then** // $\alpha < v$
  $\alpha \leftarrow v$
if $\beta \leq v$ **then** // $\beta \leq v$
  $\alpha \leftarrow \beta$

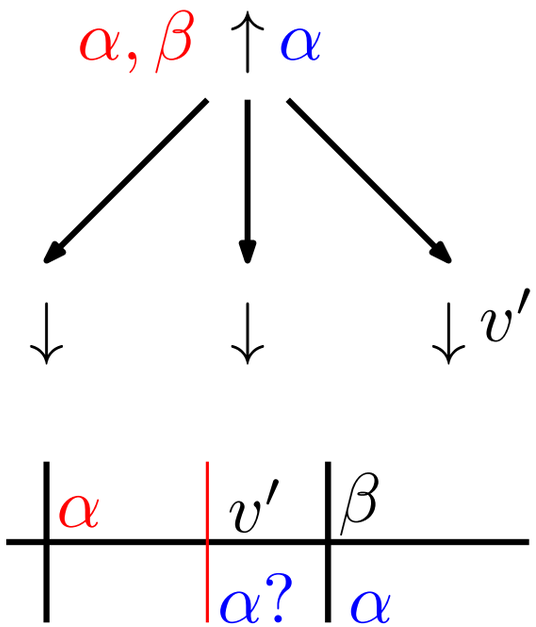
Pruning



$\beta \leq v$



$\beta \leq v'$



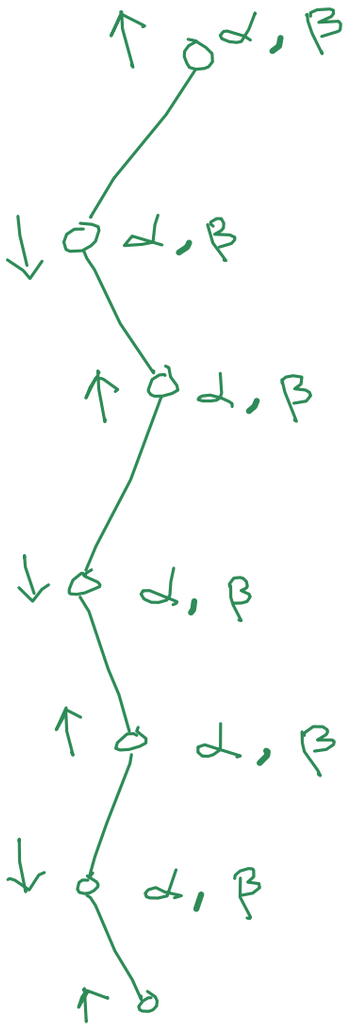
$\alpha \leq v' < \beta$

Pruning Logic

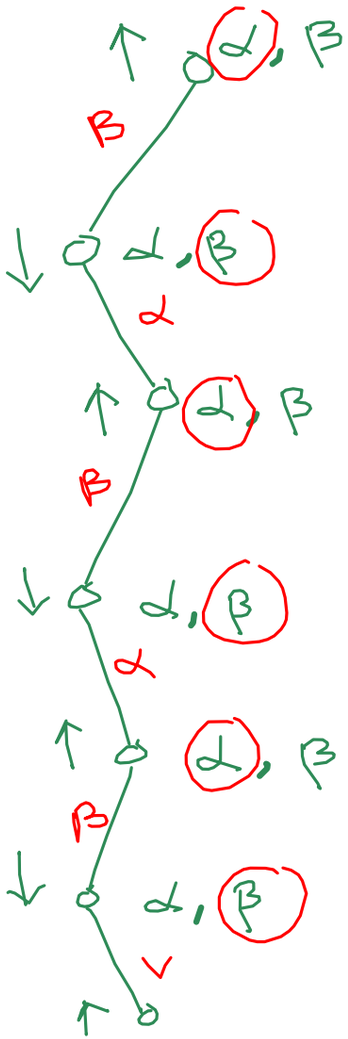
```
// update α
if α < val then          // α < val
    α ← val
if β ≤ val then          // β ≤ val
    α ← β
    return α
```

```
// update β
if val < β then          // val < β
    β ← val
if val ≤ α then          // val ≤ α
    β ← α
    return β
```

DFS: Descending and Ascending



DFS: Descending and Ascending



Minimax with α-β Pruning

Algorithm: Alpha_Beta(n, α, β)

Input: n a node in a game tree; α, β real numbers
Output: A pair of a value for each agent for node n, path that gives this value

```
if n is a leaf node then  return evaluate(n), None
else if n is a MAX node then
    foreach child c of n do
        α ← α ↑ Alpha_Beta(c, α, β)
        if β ≤ α then
            α ← β
            return α
    return α
else
    foreach child c of n do
        β ← β ↓ Alpha_Beta(c, α, β)
        if β ≤ α then
            β ← α
            return β
    return β
```

Minimax with $\alpha\beta$ Pruning

Algorithm: Alpha_Beta(n, α, β)

Input: n a node in a game tree; α, β real numbers
Output: A pair of a value for each agent for node n, path that gives this value

```
if n is a leaf node then  return evaluate(n), None
else if n is a MAX node then
    foreach child c of n do
        val, path ← Alpha_Beta(c, α, β)
        if α < val  then
            α ← val
            best_path ← [c] + path
        if β ≤ val then  return val, best_path
    return α, best_path
else
    foreach child c of n do
        val, path ← Alpha_Beta(c, α, β)
        if val < β then
            β ← val
            best_path ← [c] + path
        if val ≤ α then  return val, best_path
```

Alpha-Beta Pruning Trace

α — ∞, ∞

β

α

β

α

Alpha-Beta Pruning Trace

α $\quad -\infty, \infty$

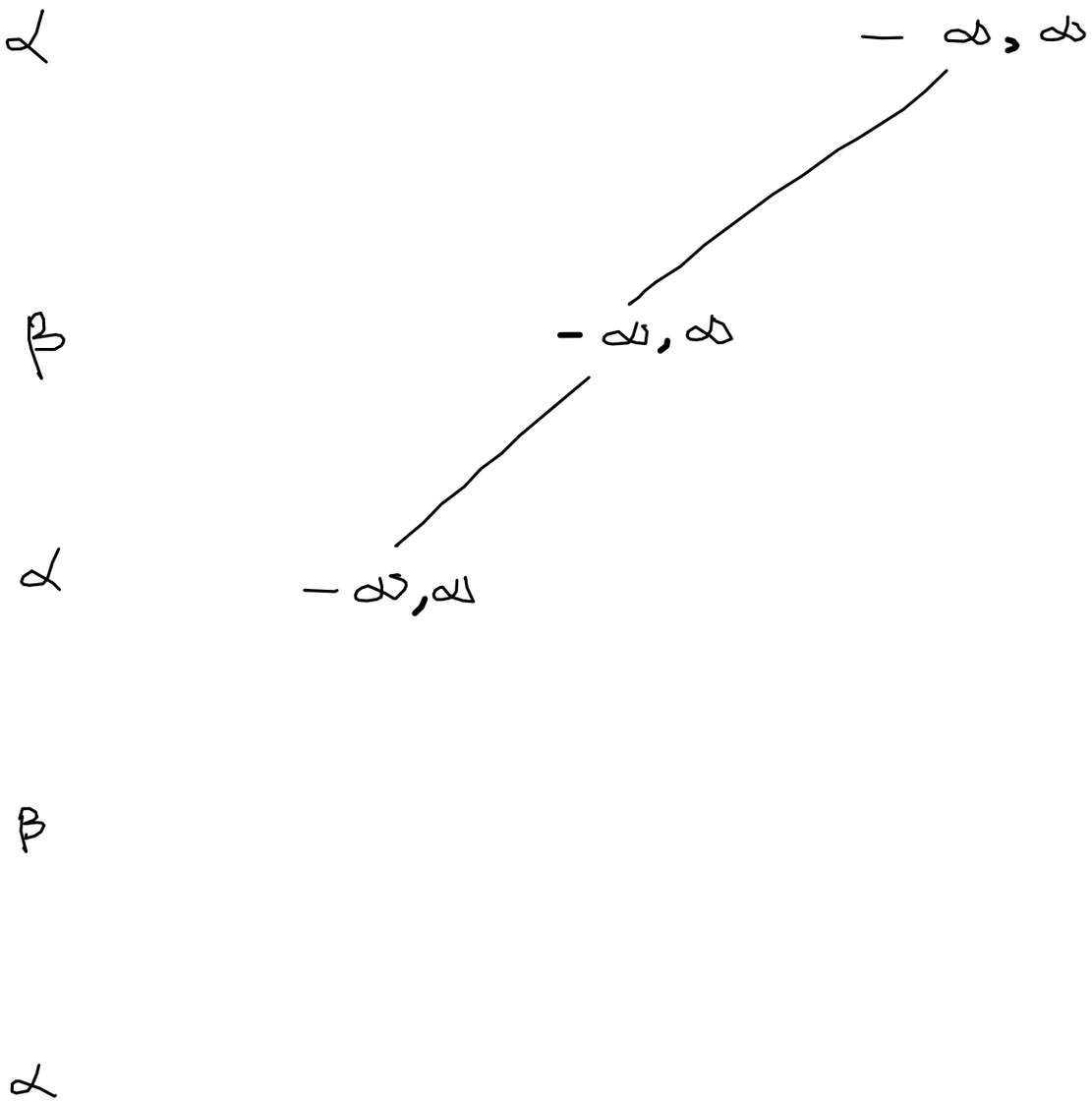
β $\quad -\infty, \infty$

α

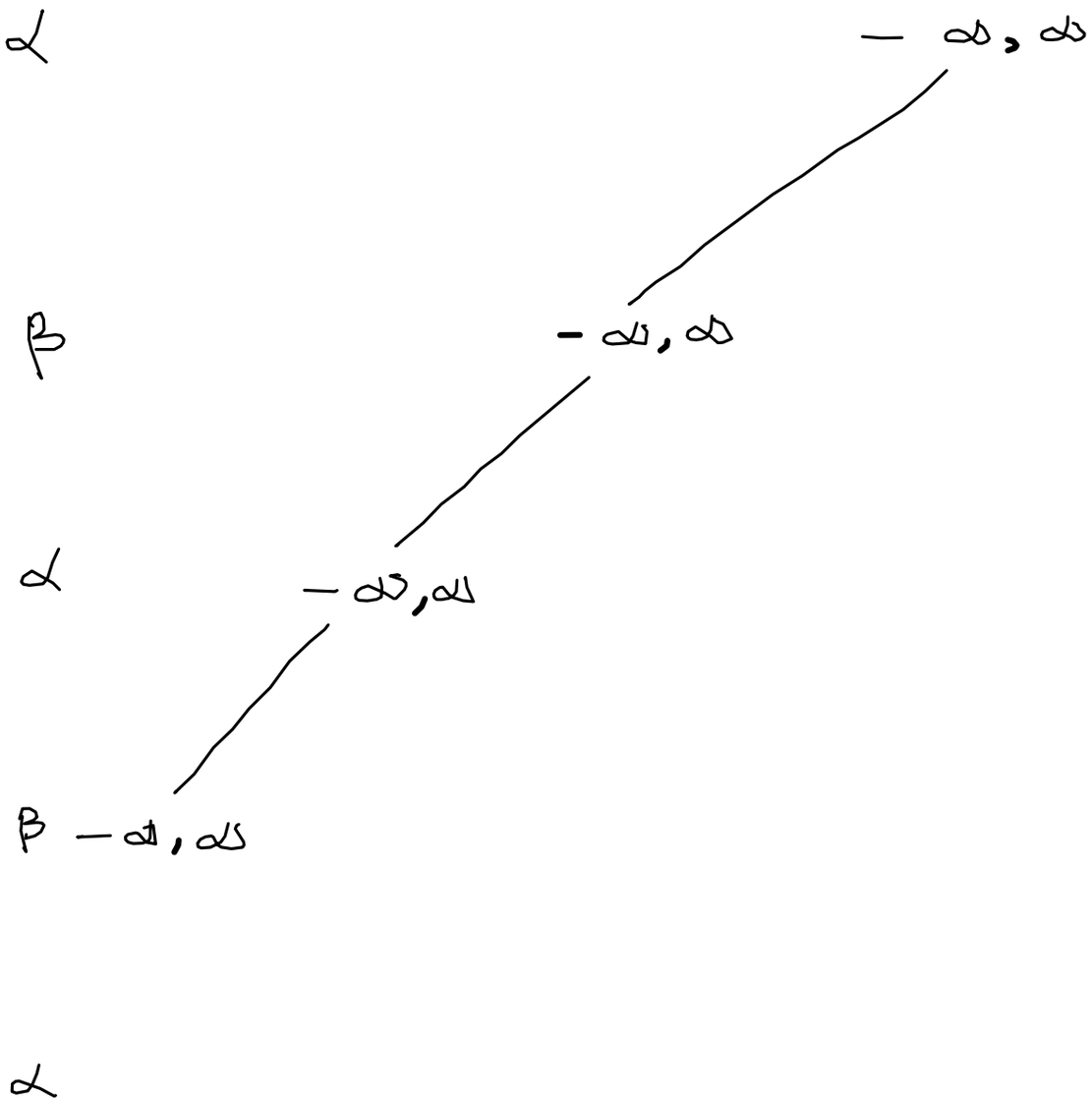
β

α

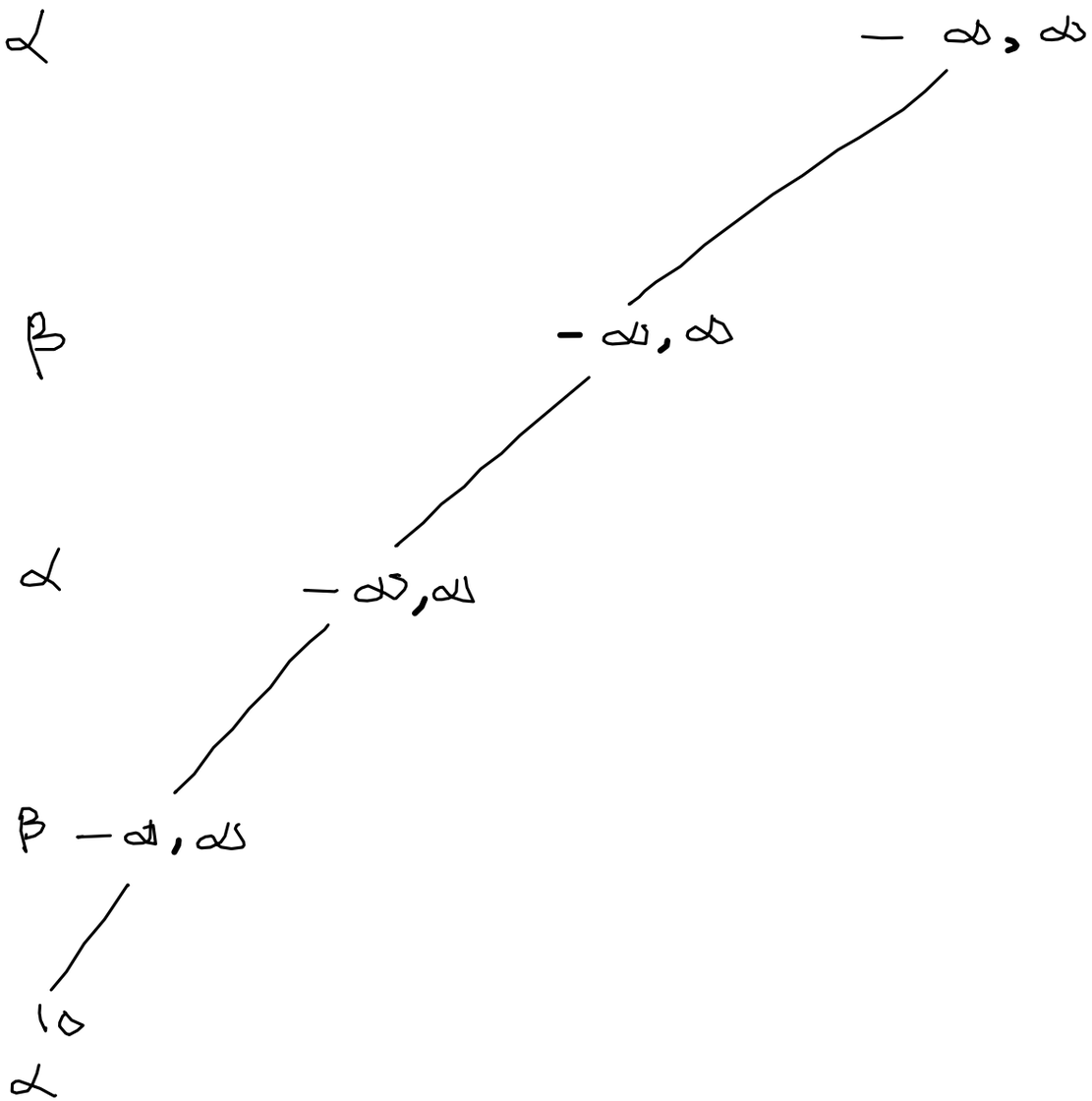
Alpha-Beta Pruning Trace



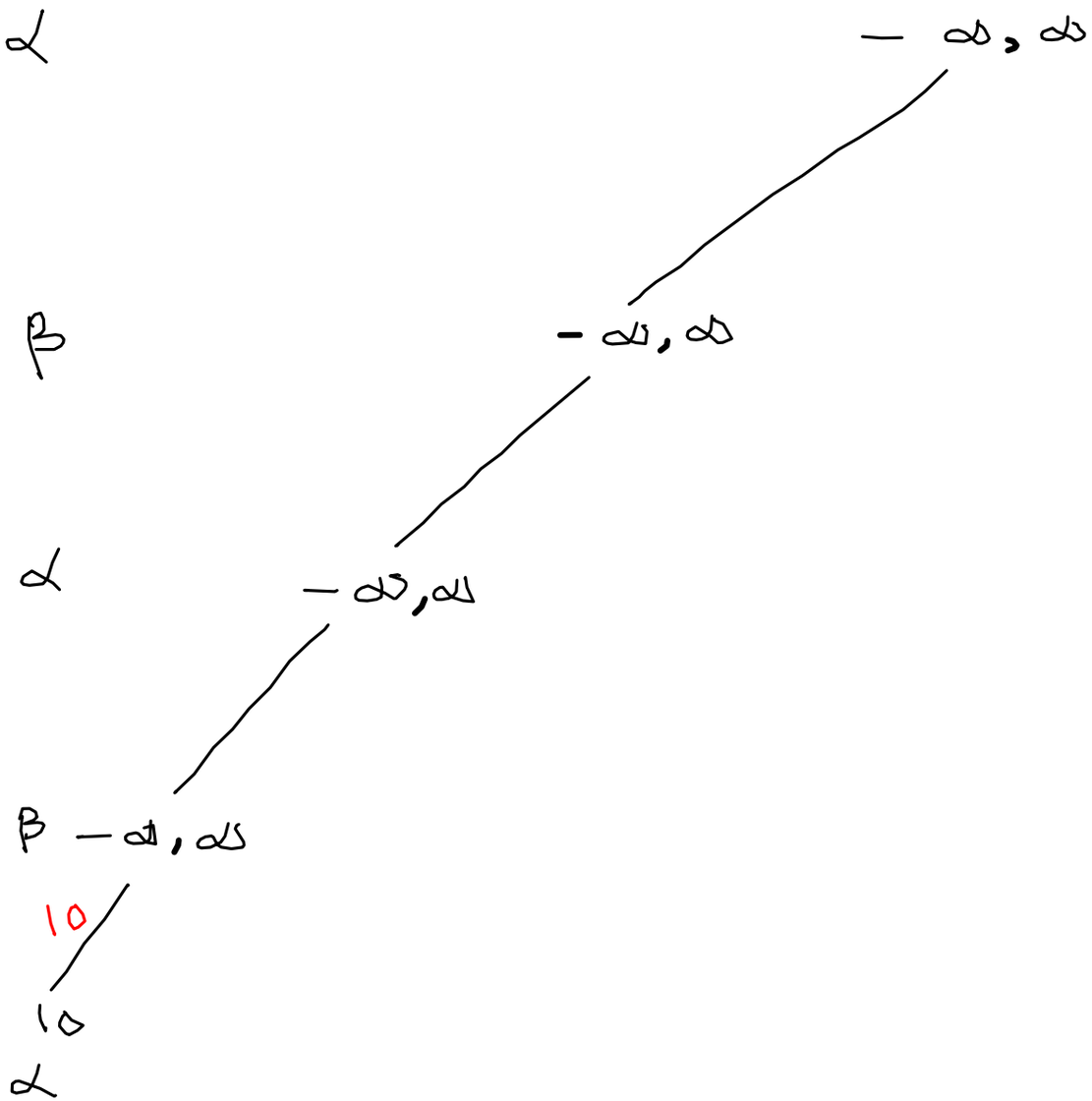
Alpha-Beta Pruning Trace



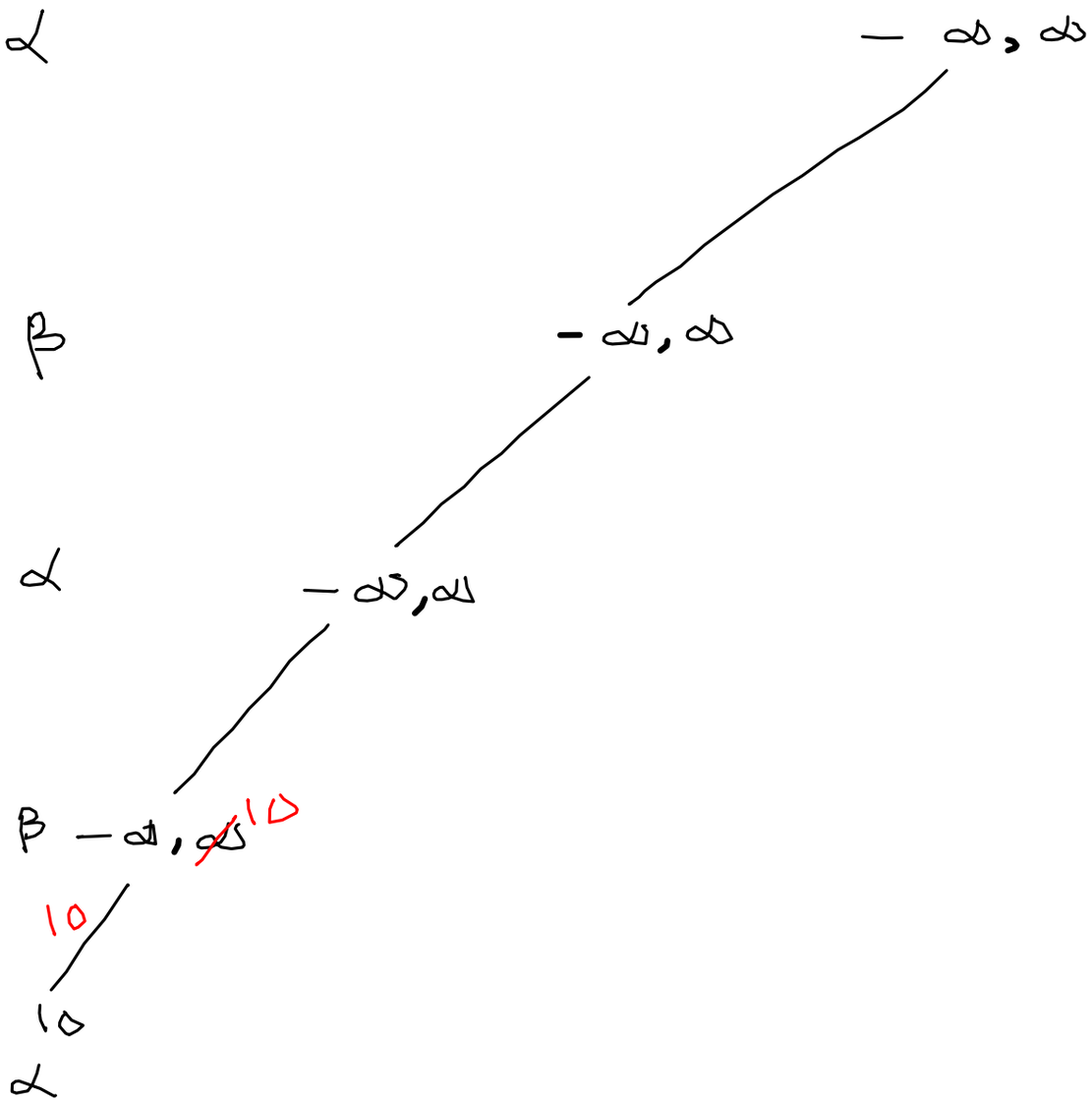
Alpha-Beta Pruning Trace



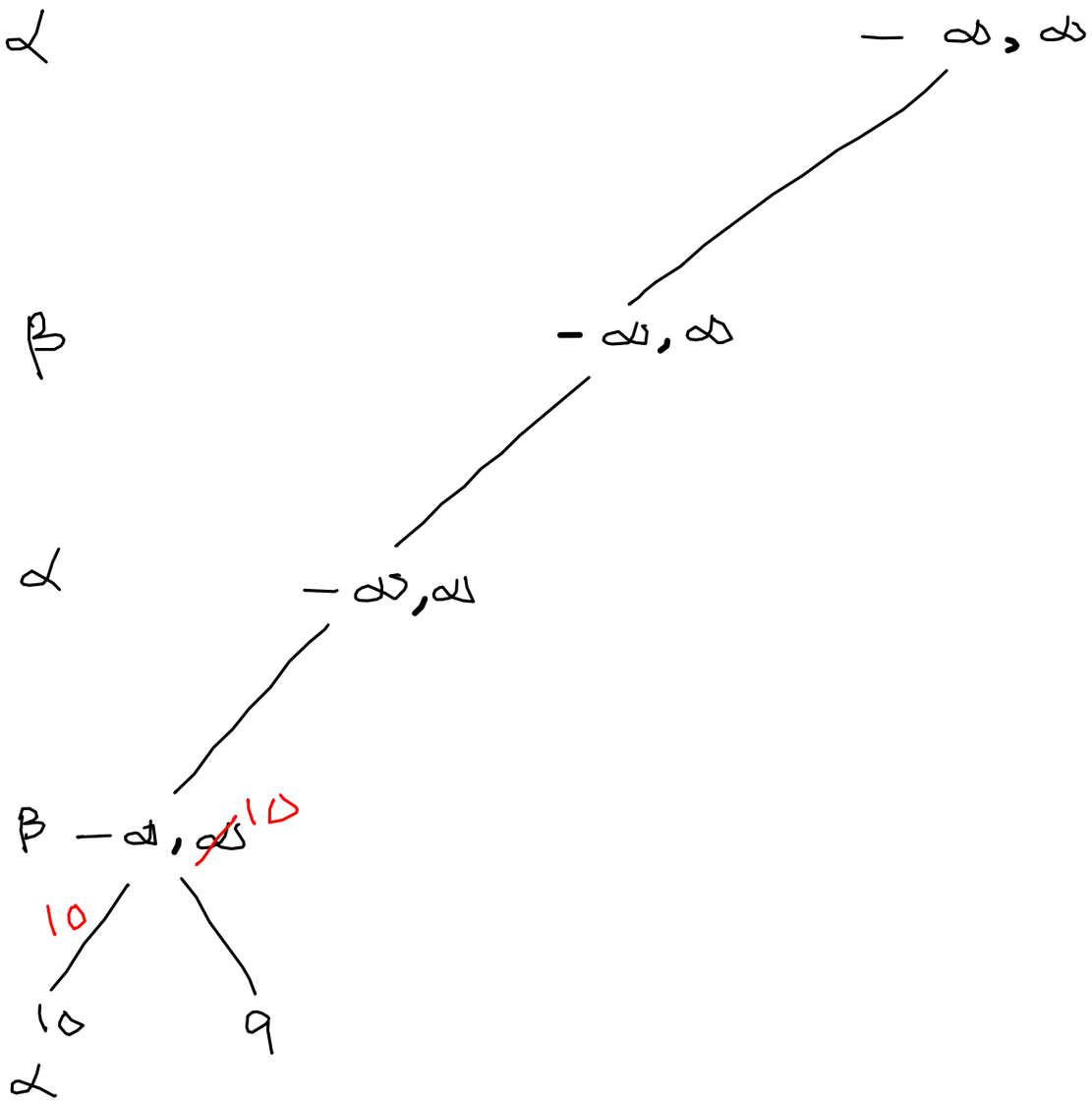
Alpha-Beta Pruning Trace



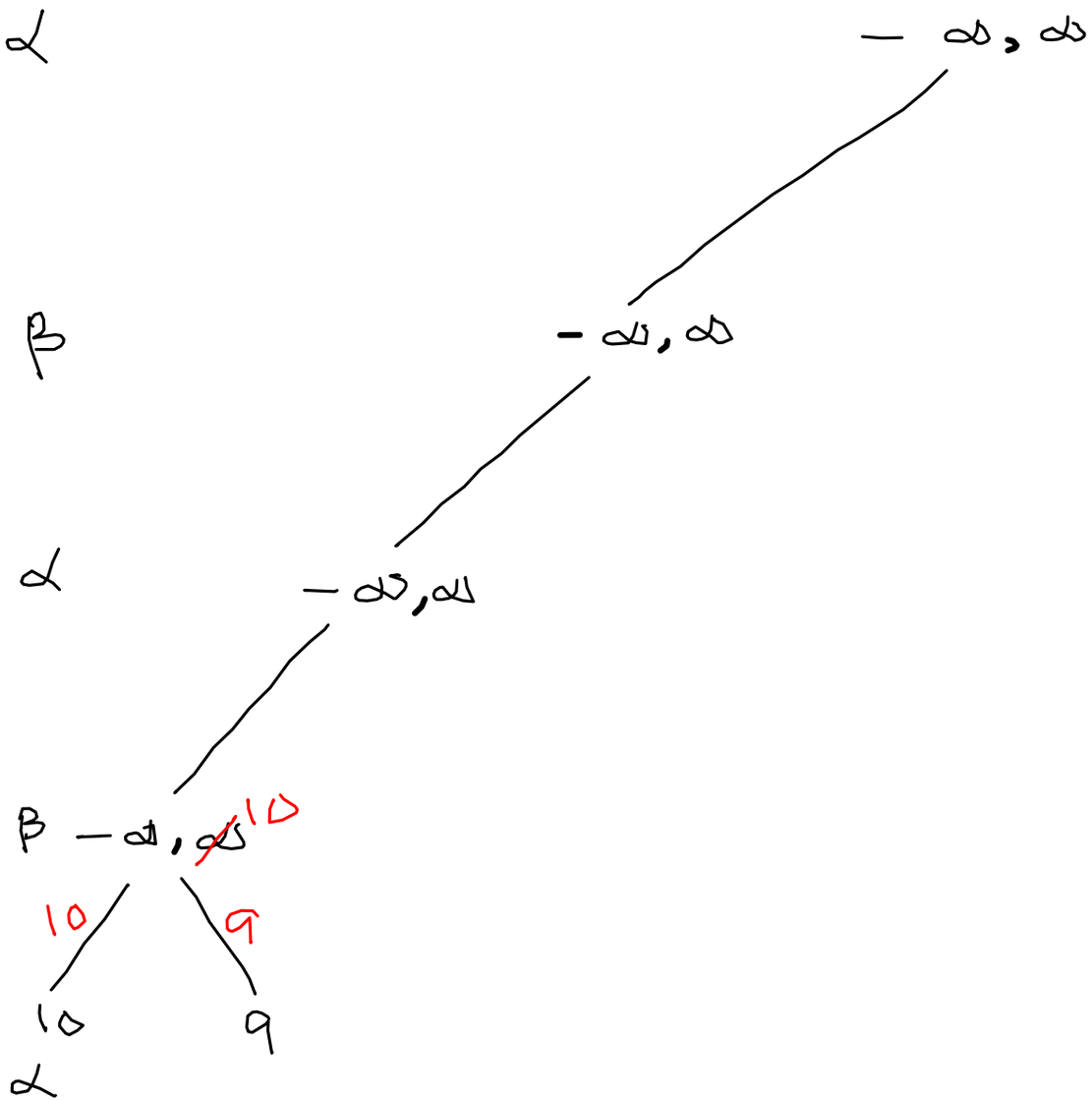
Alpha-Beta Pruning Trace



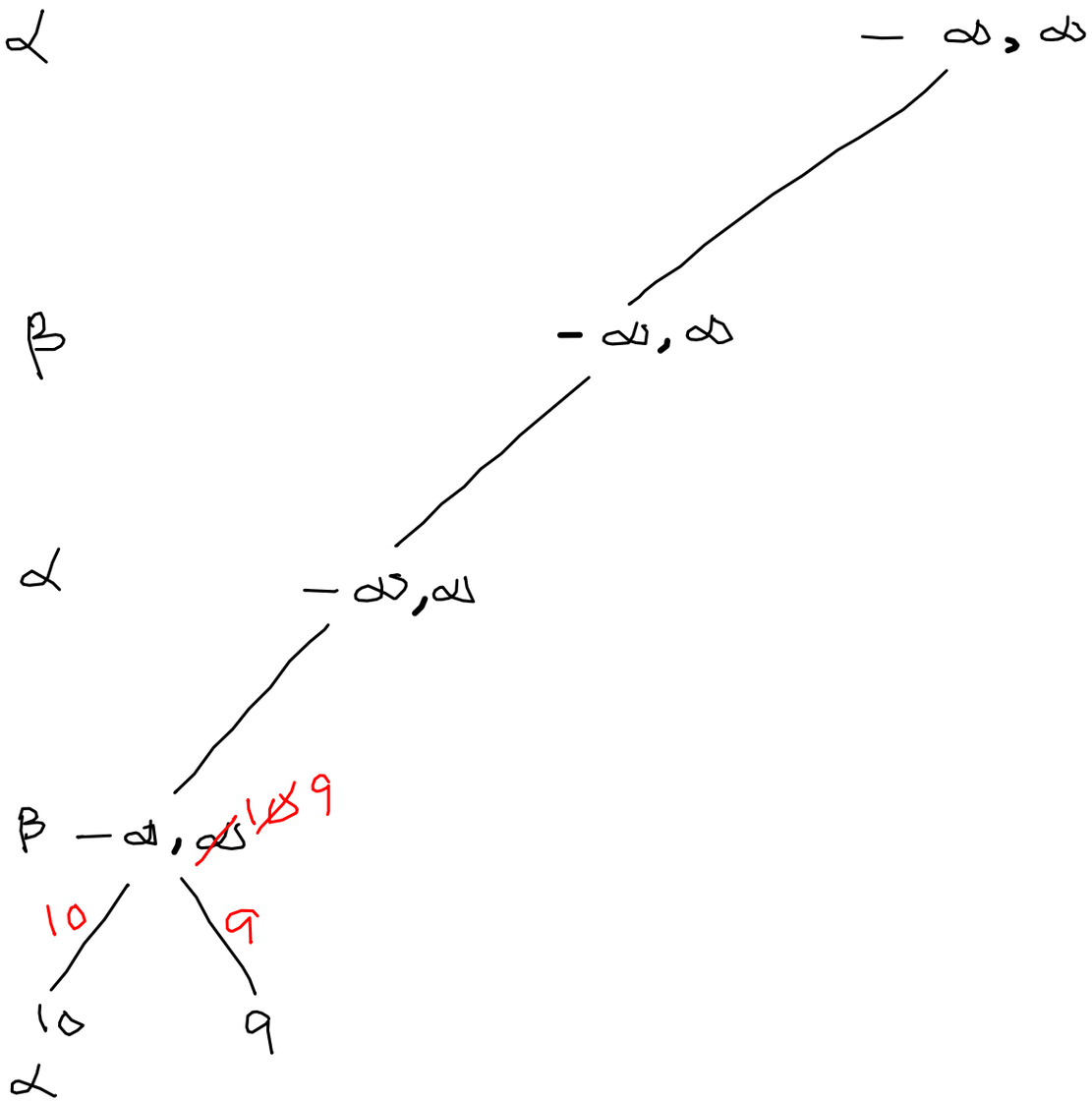
Alpha-Beta Pruning Trace



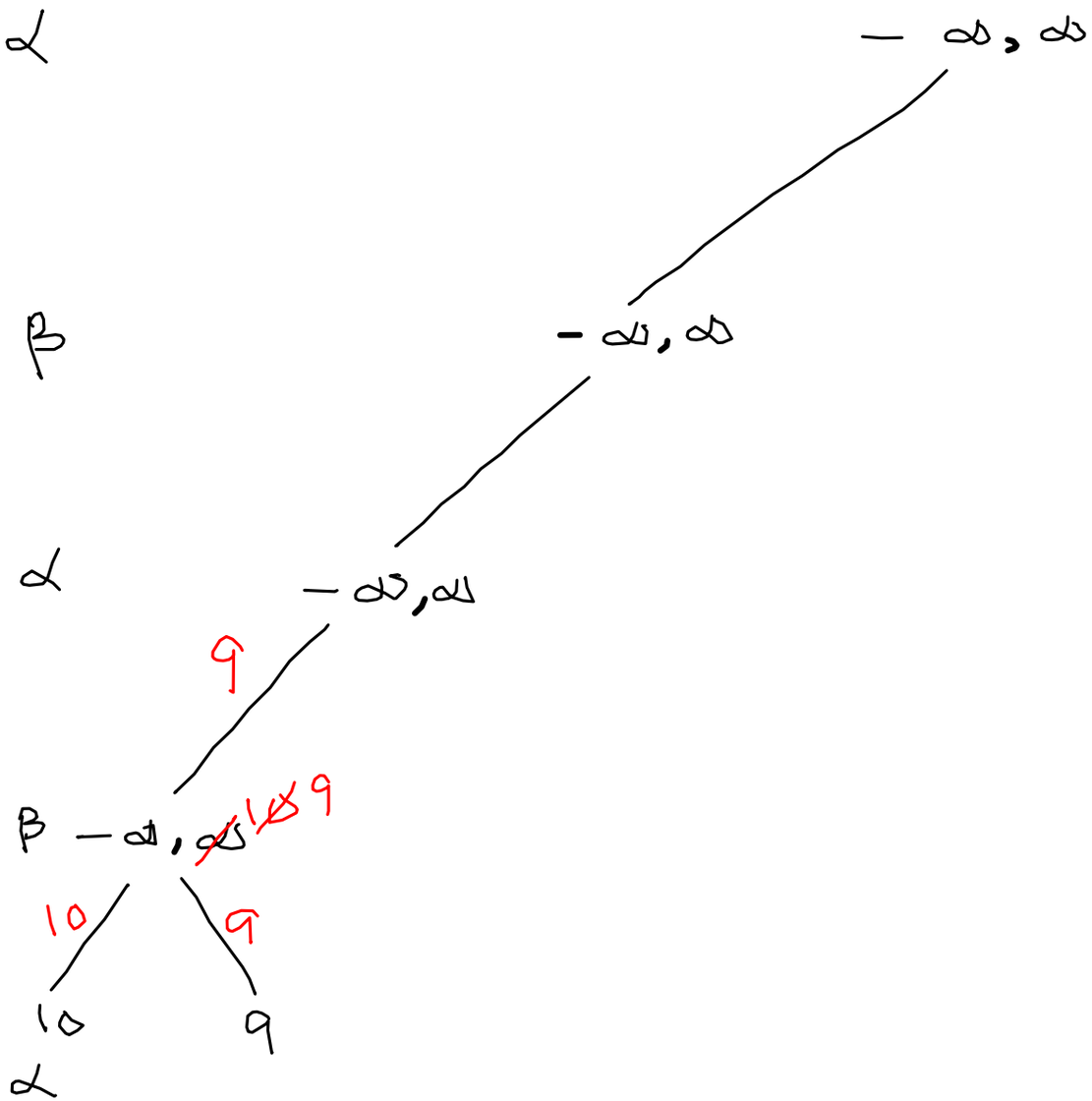
Alpha-Beta Pruning Trace



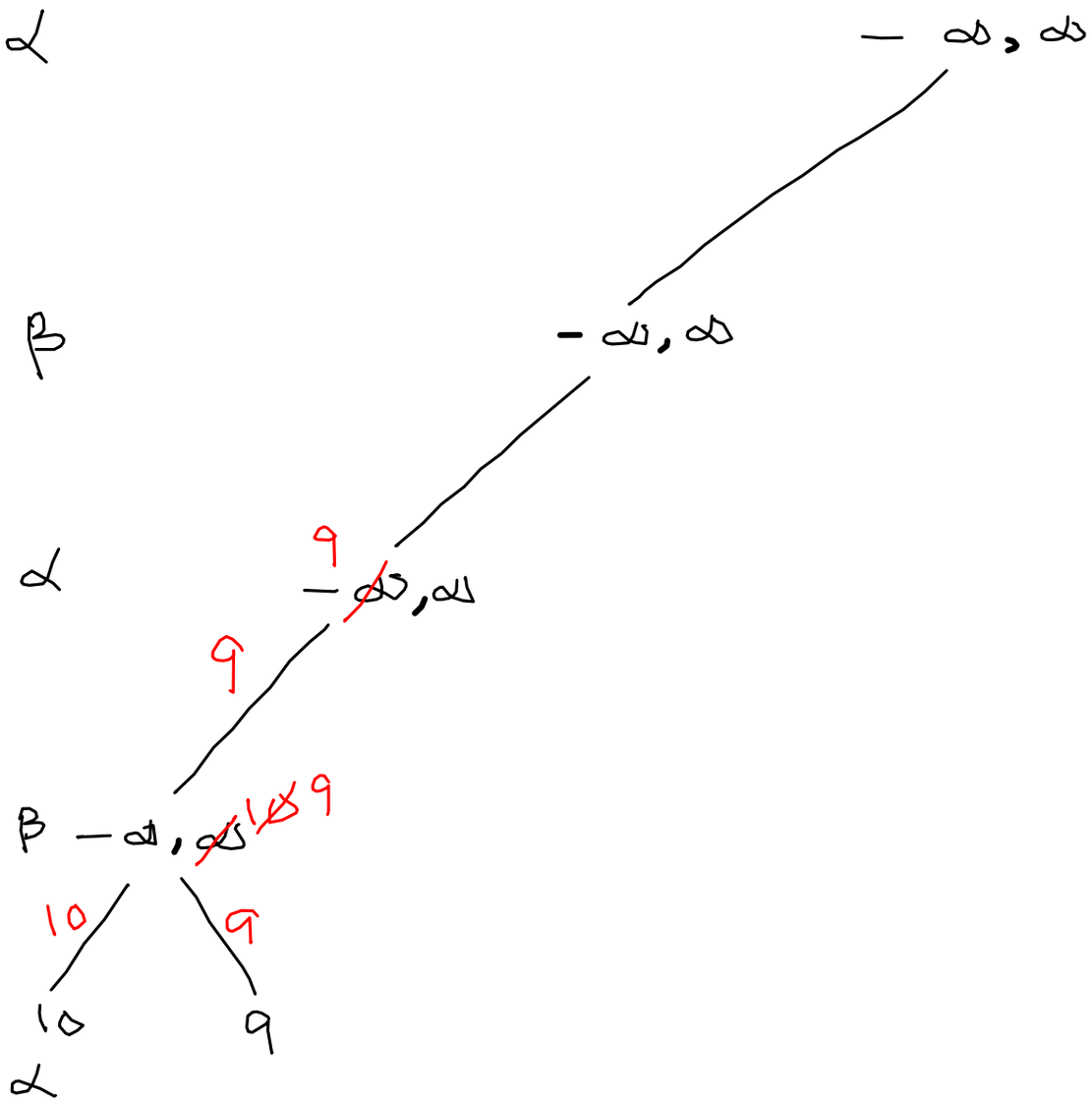
Alpha-Beta Pruning Trace



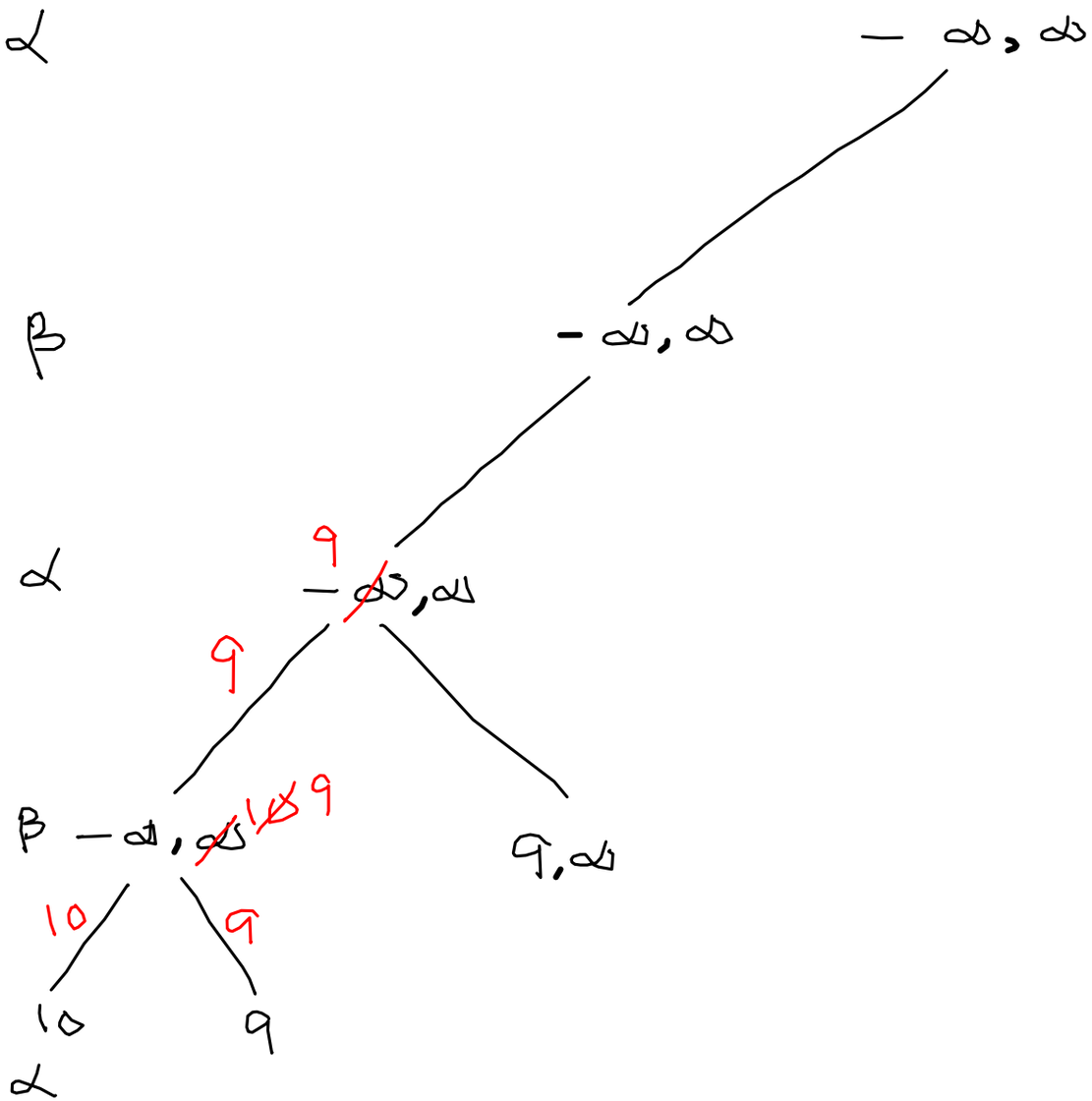
Alpha-Beta Pruning Trace



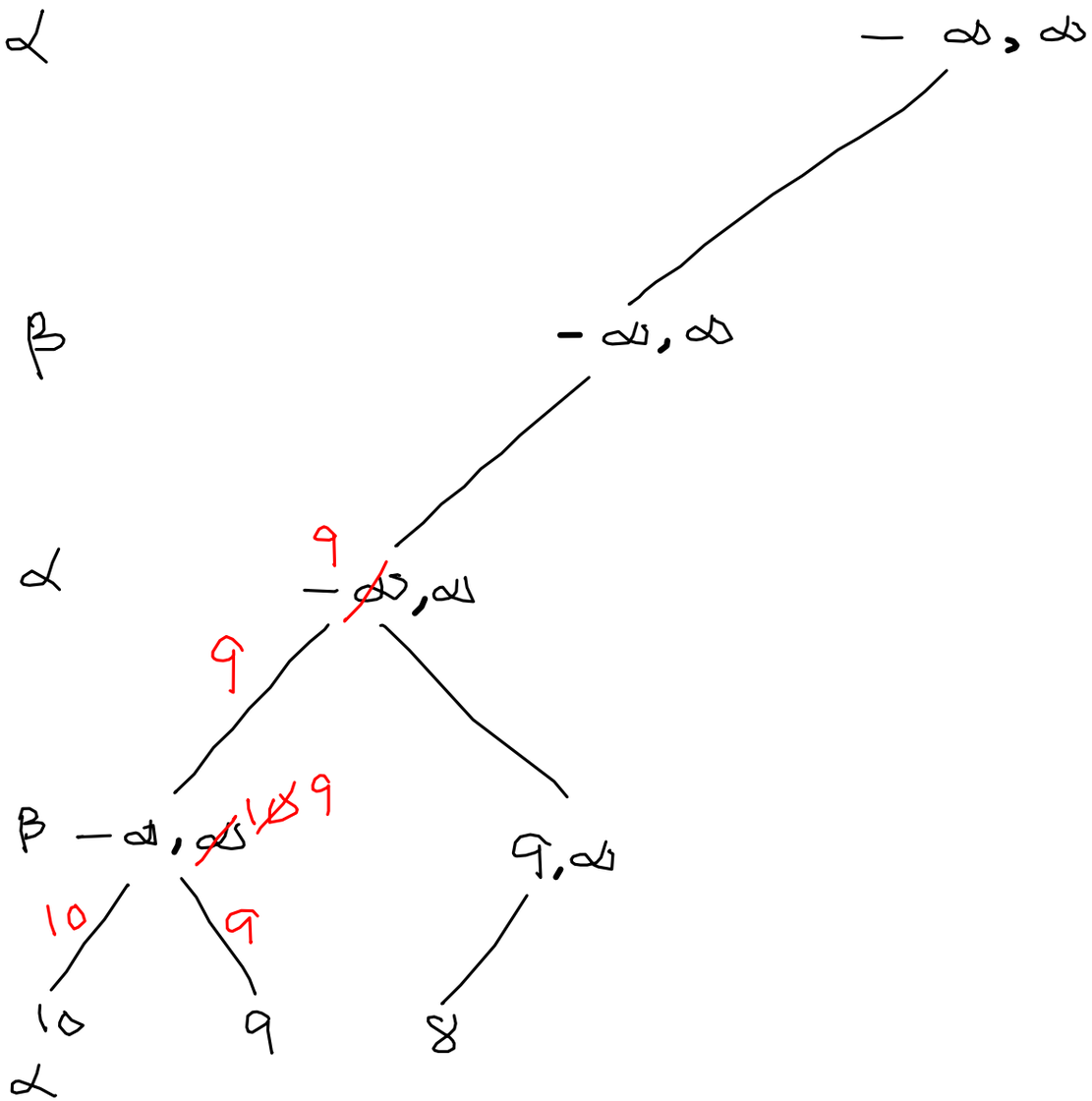
Alpha-Beta Pruning Trace



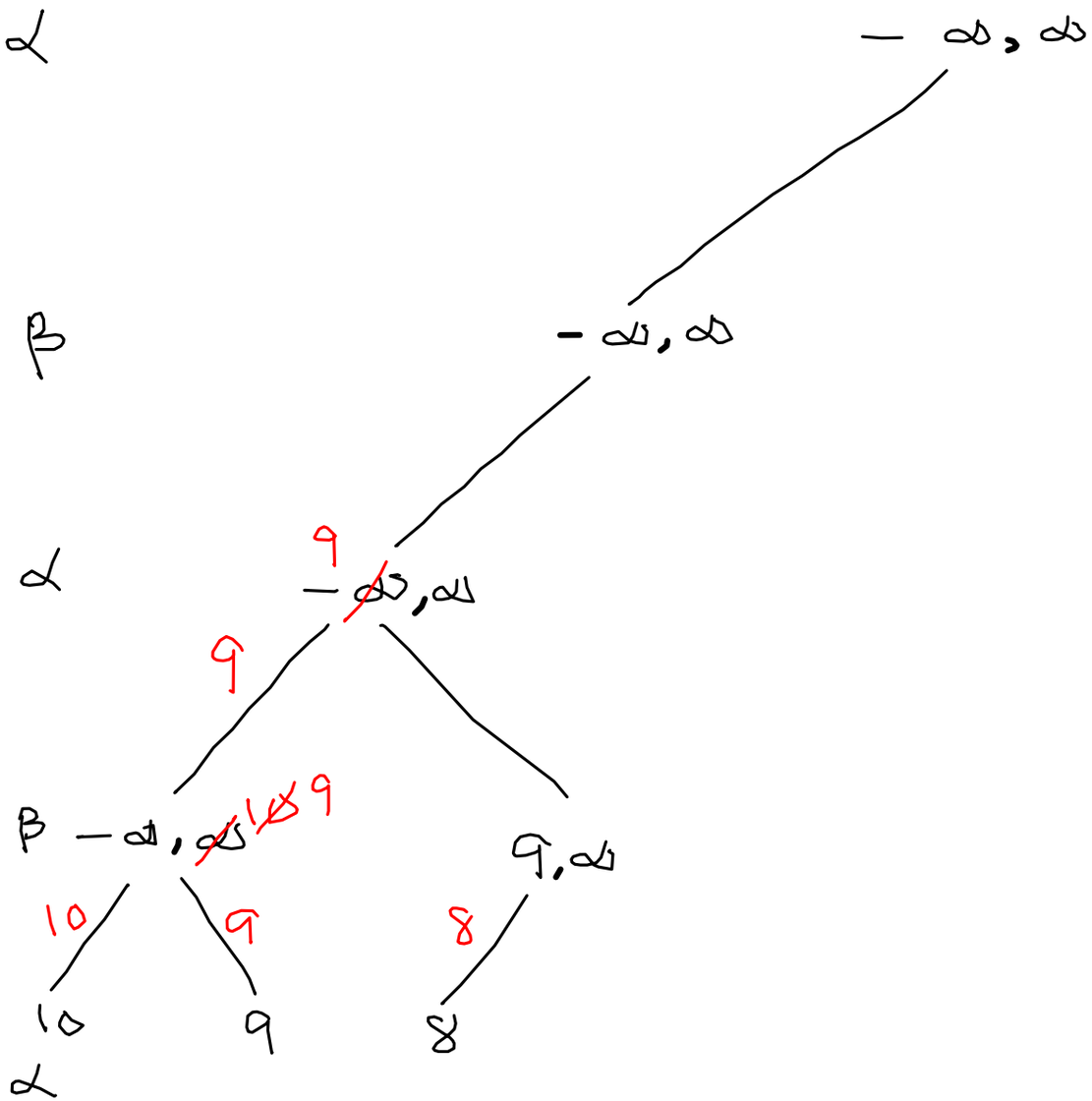
Alpha-Beta Pruning Trace



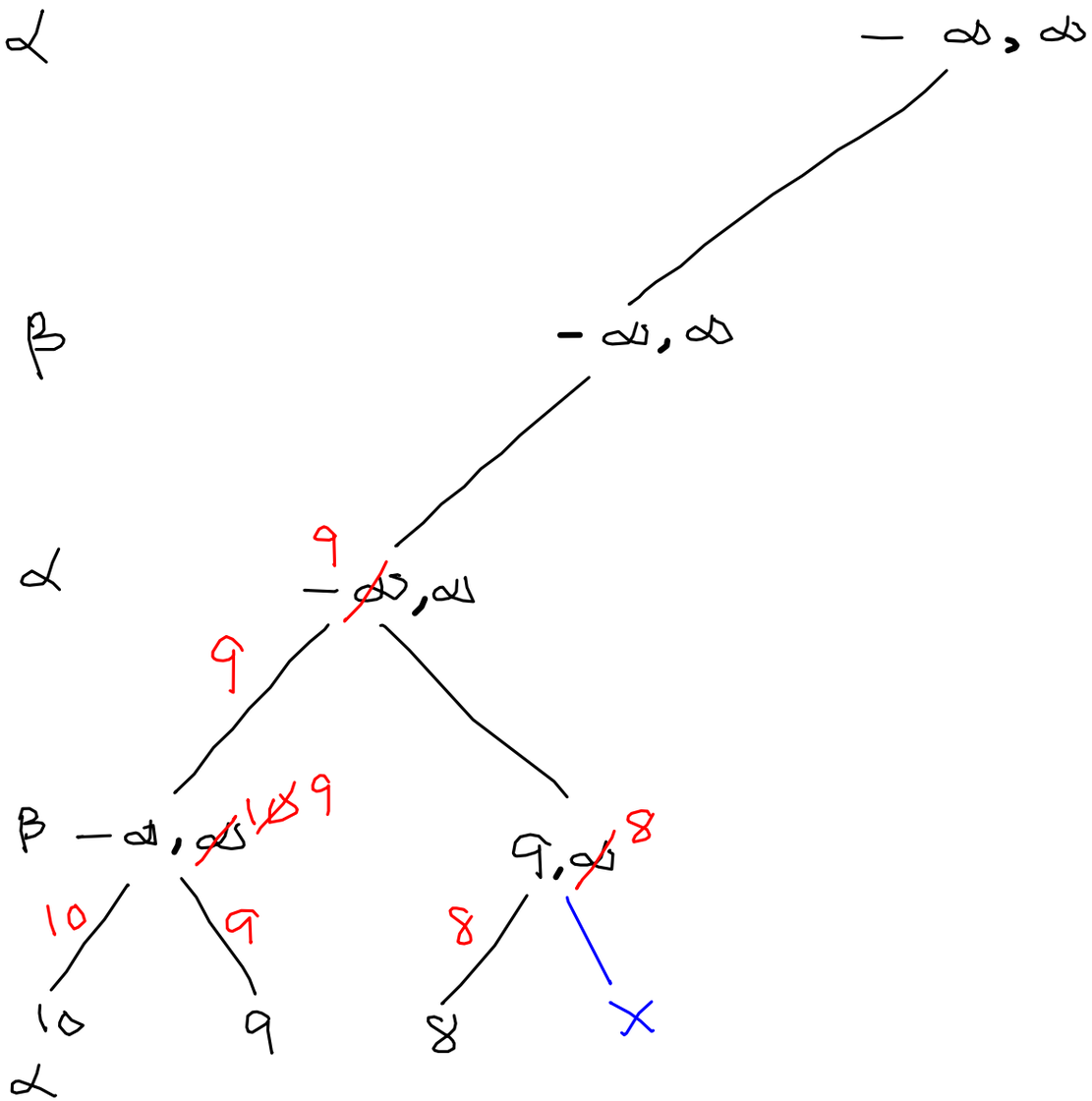
Alpha-Beta Pruning Trace



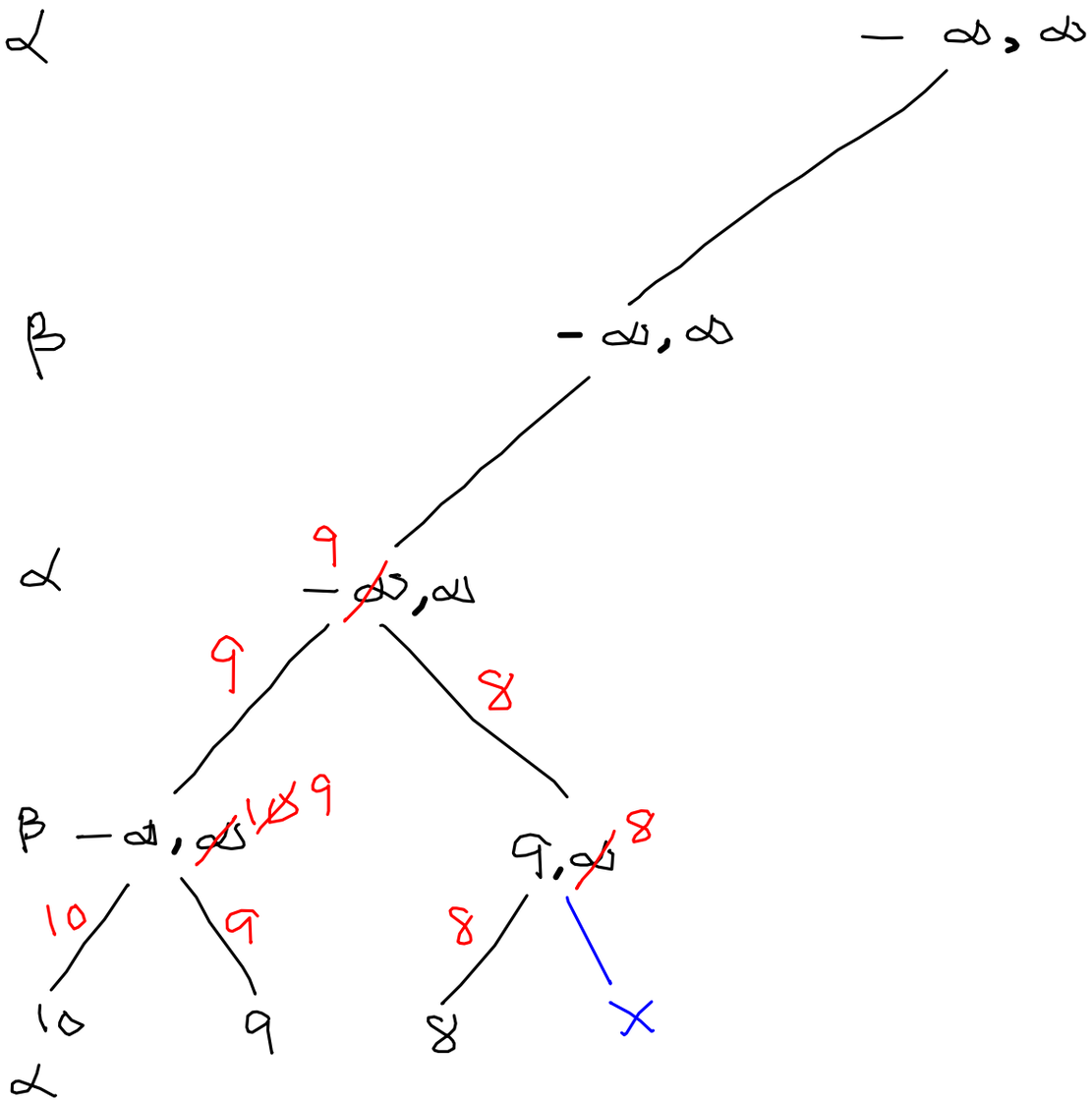
Alpha-Beta Pruning Trace



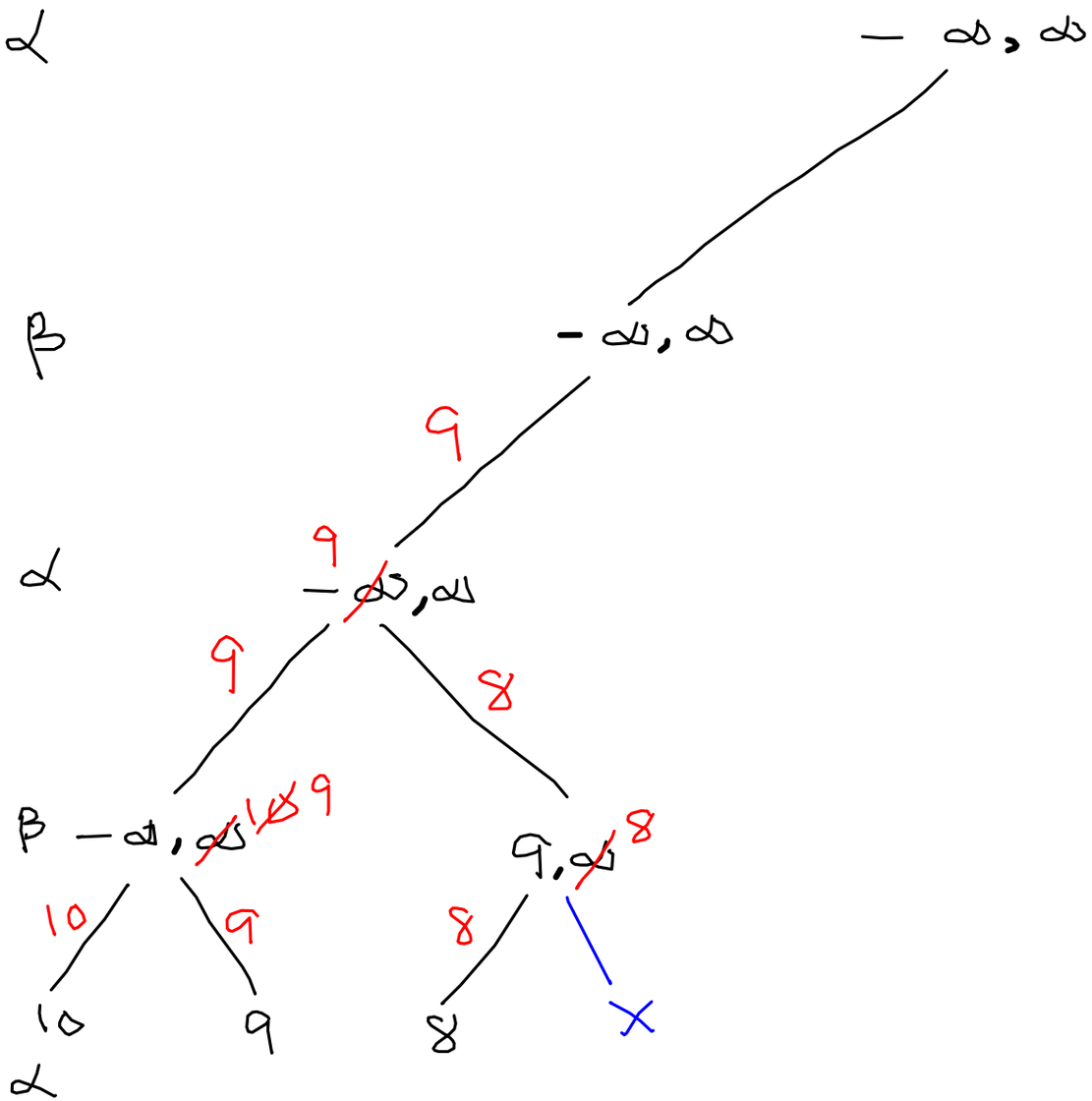
Alpha-Beta Pruning Trace



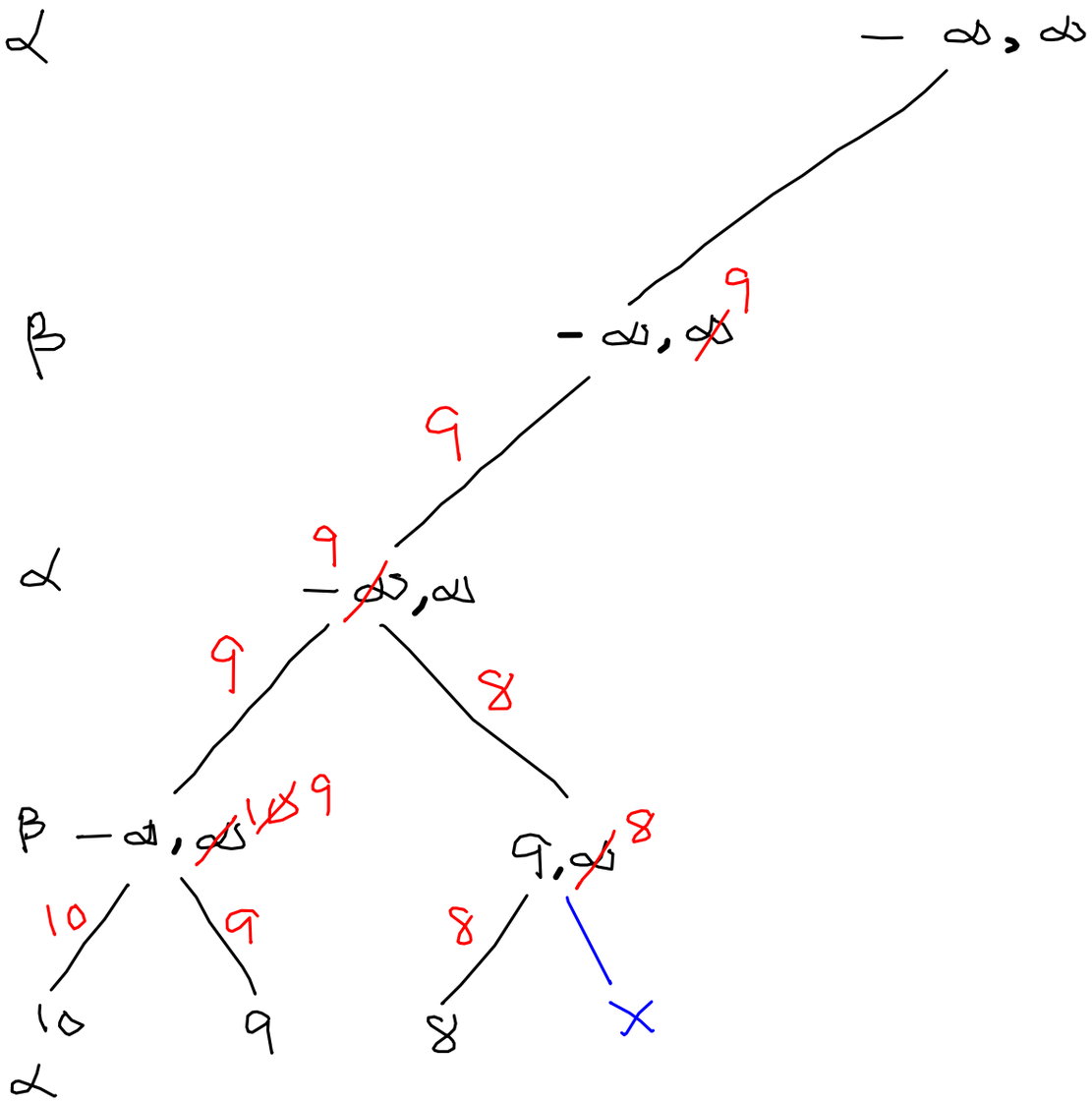
Alpha-Beta Pruning Trace



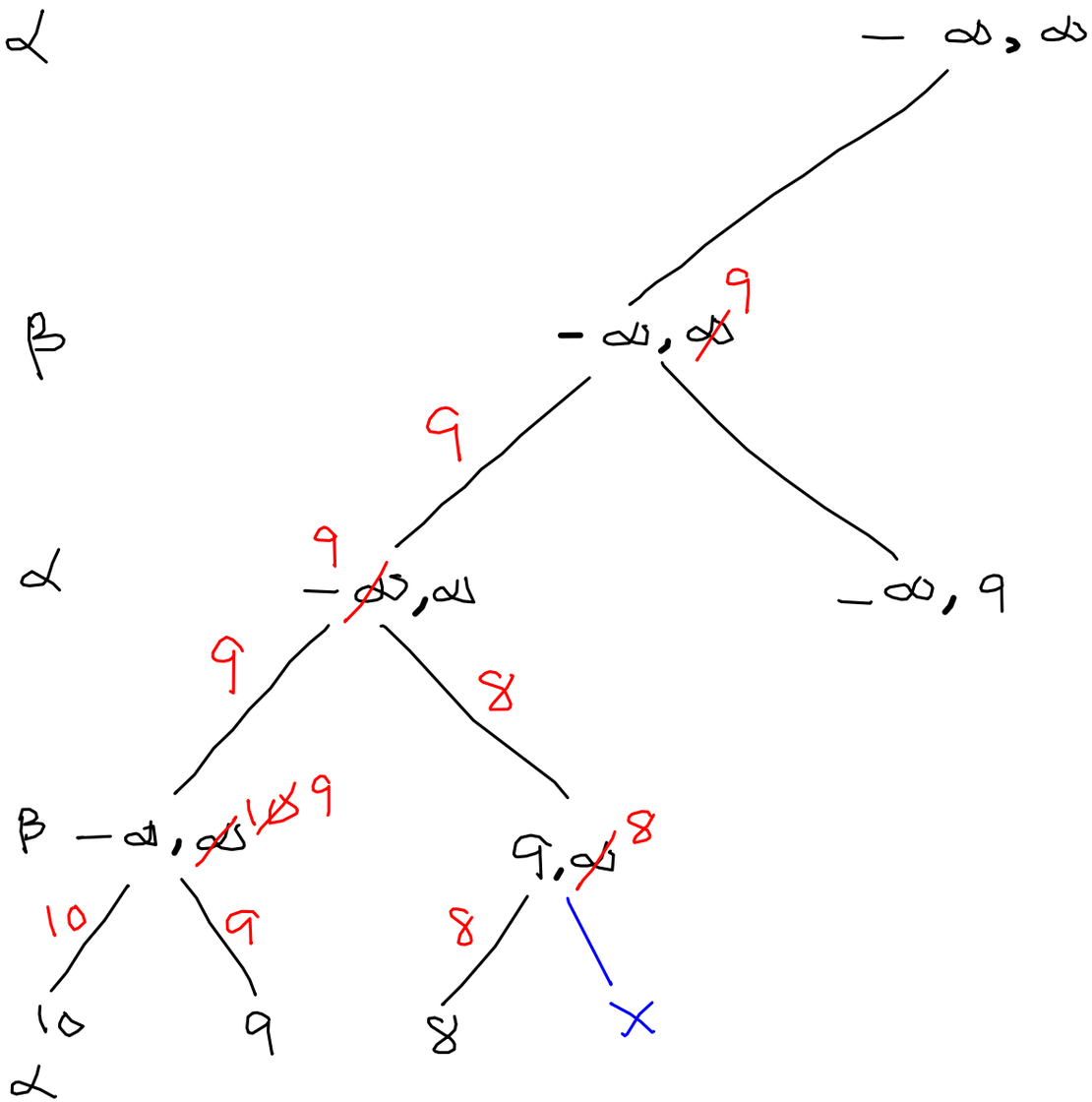
Alpha-Beta Pruning Trace



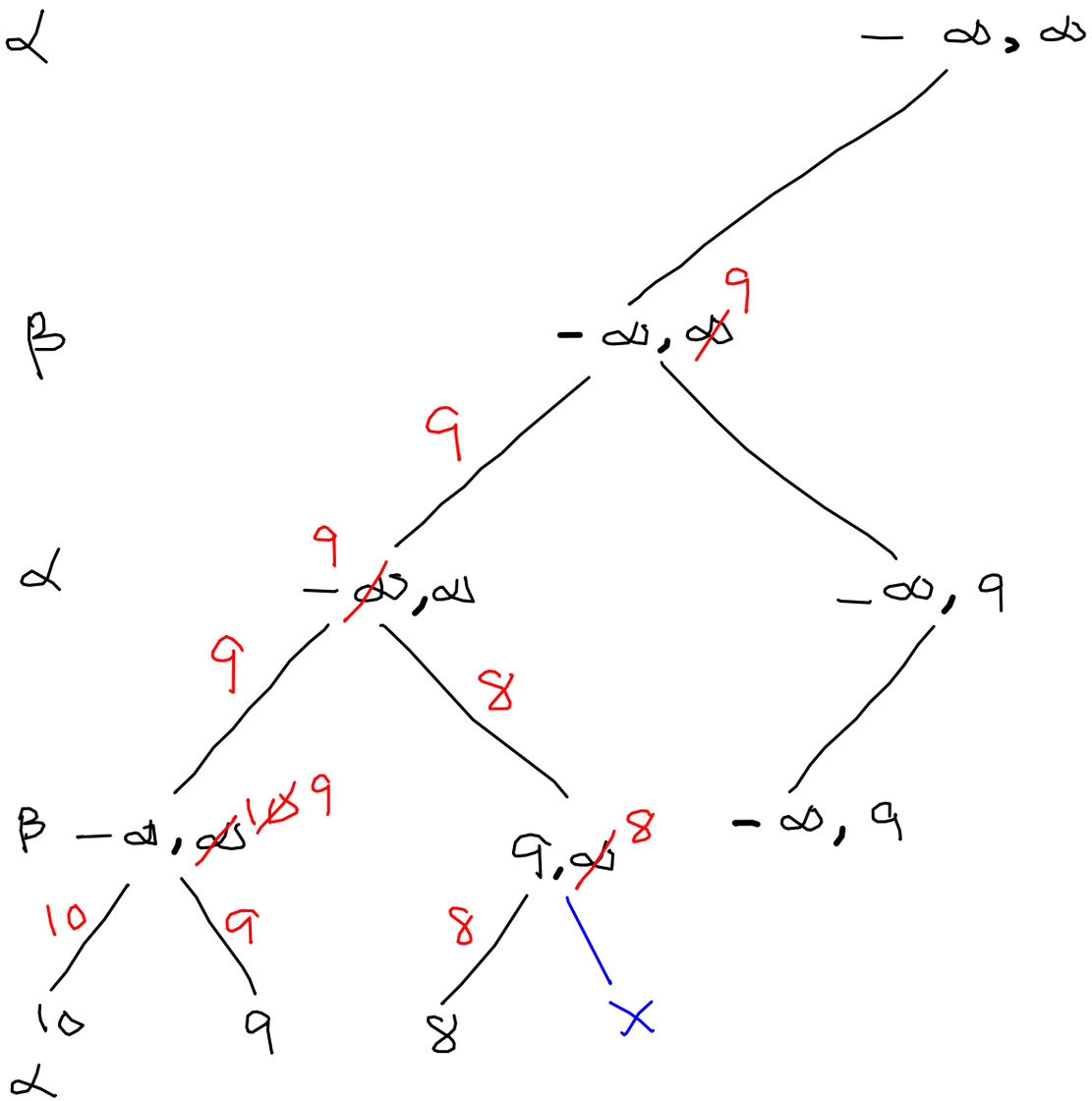
Alpha-Beta Pruning Trace



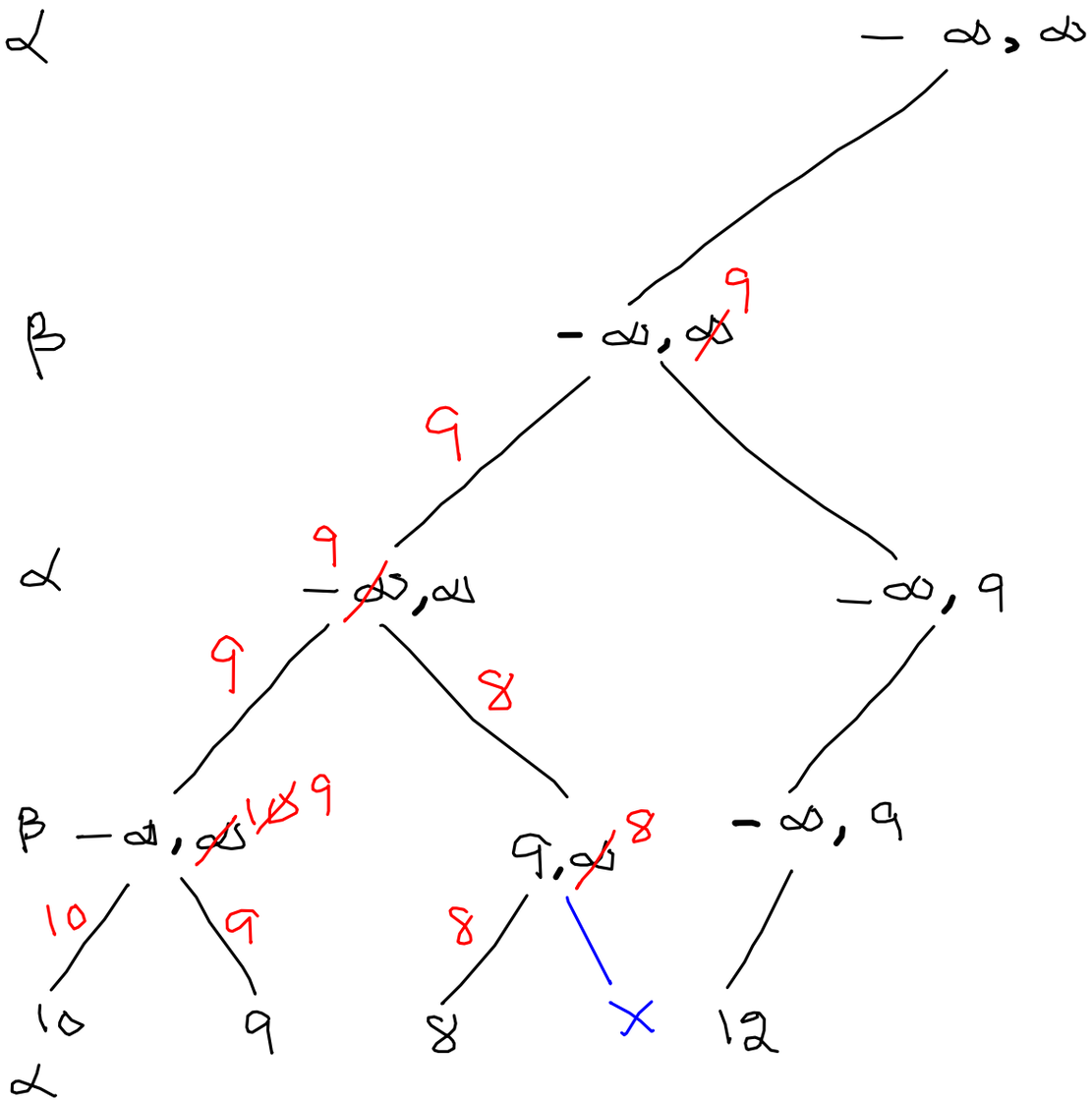
Alpha-Beta Pruning Trace



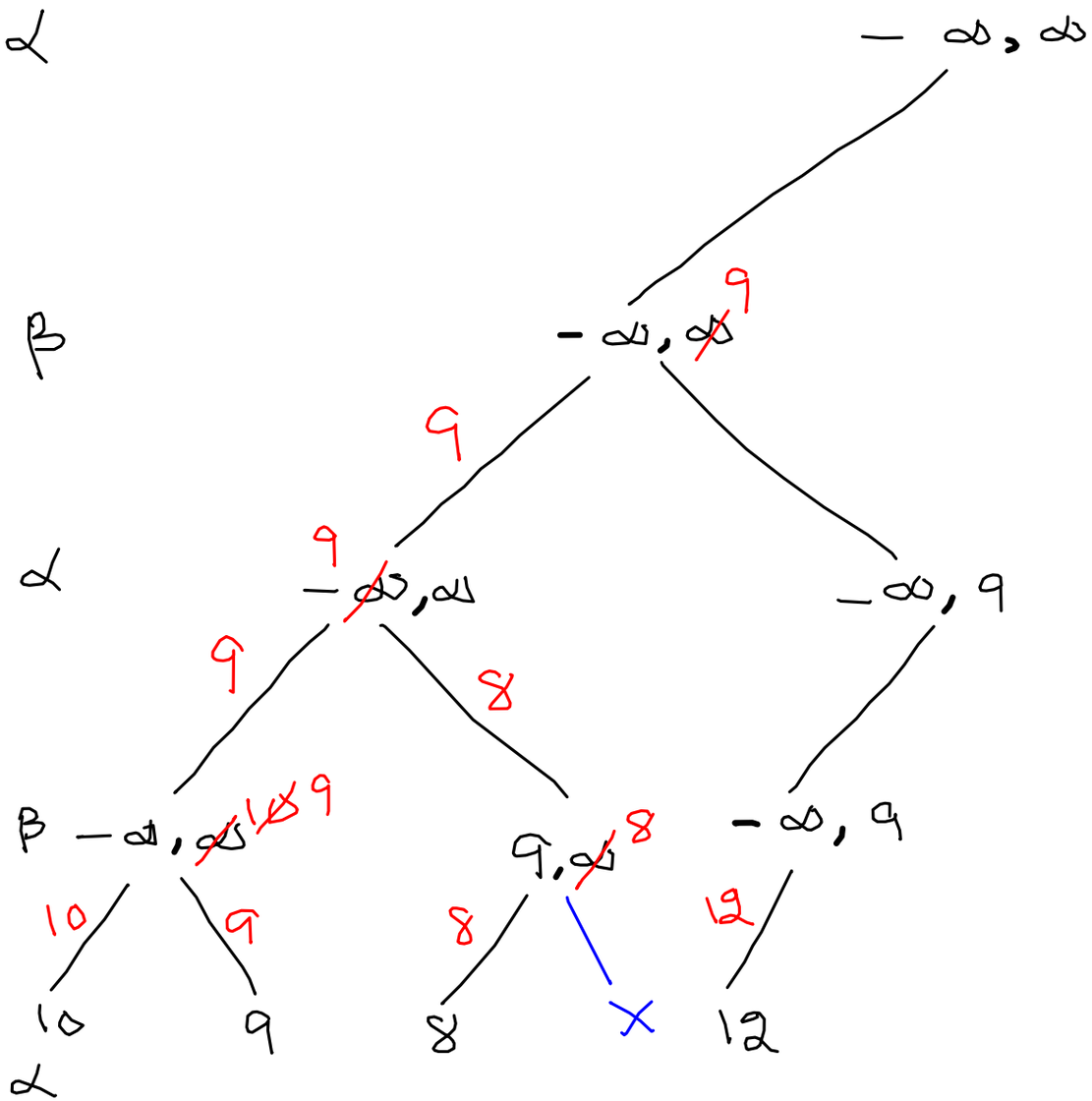
Alpha-Beta Pruning Trace



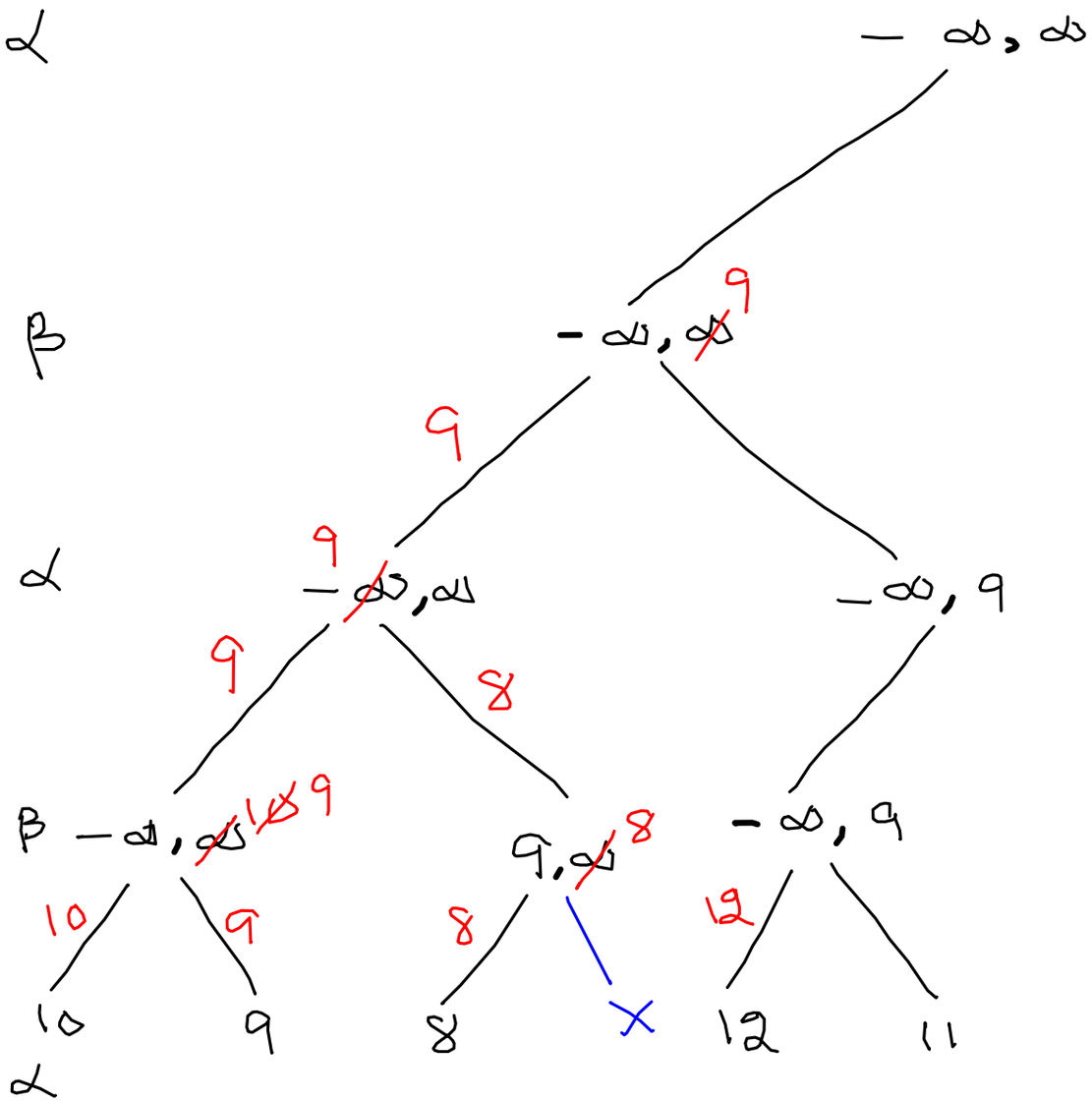
Alpha-Beta Pruning Trace



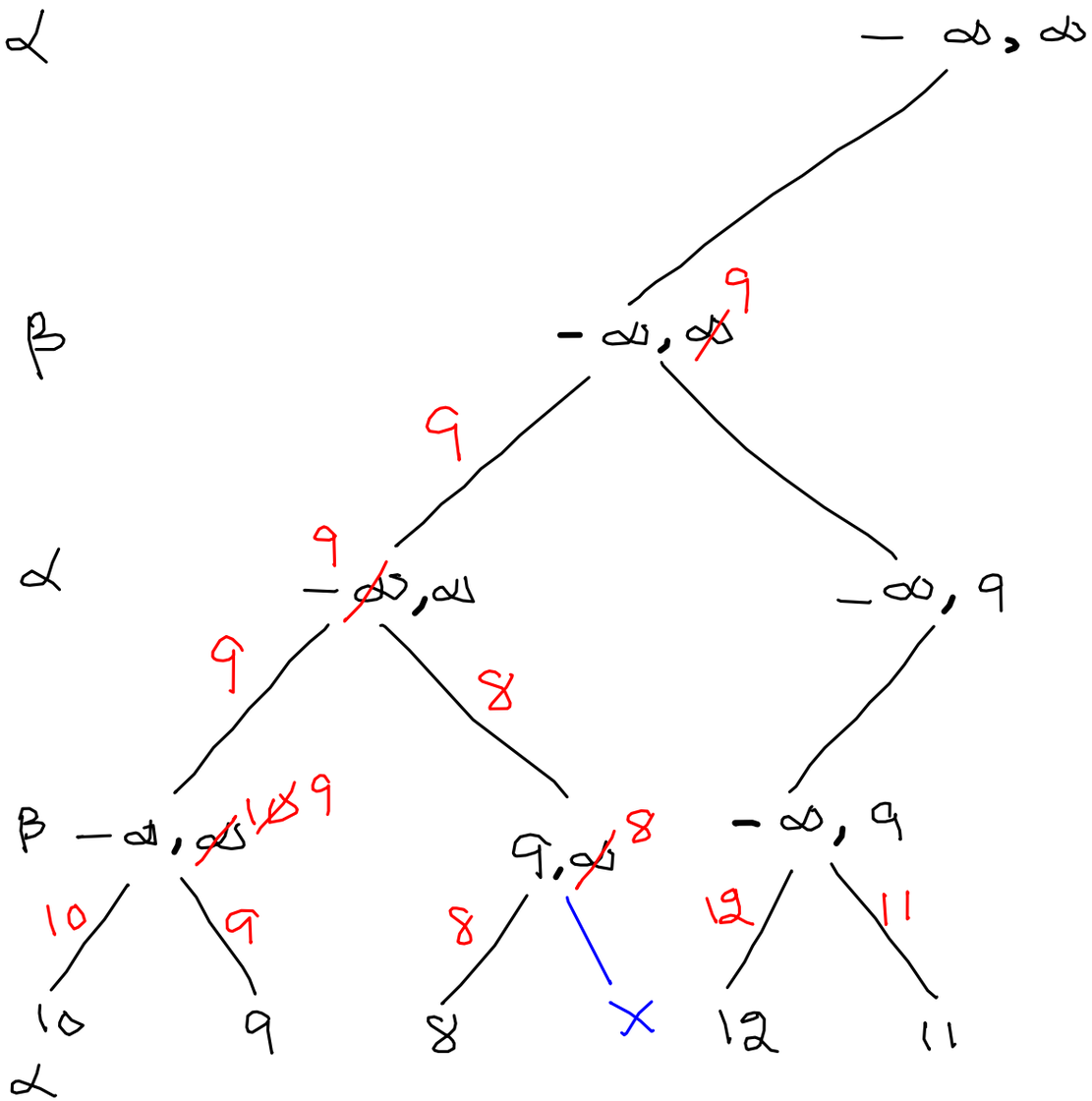
Alpha-Beta Pruning Trace



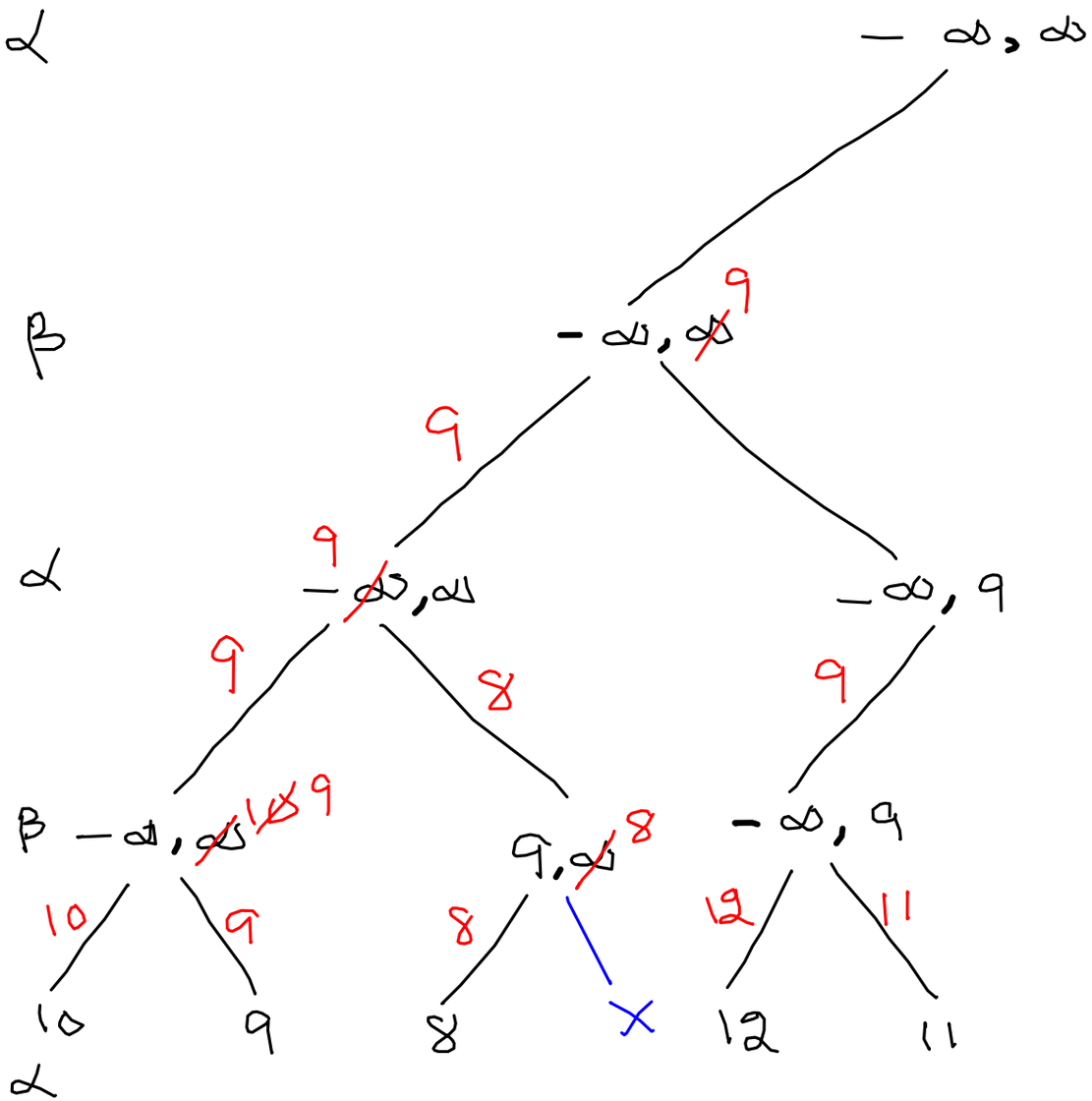
Alpha-Beta Pruning Trace



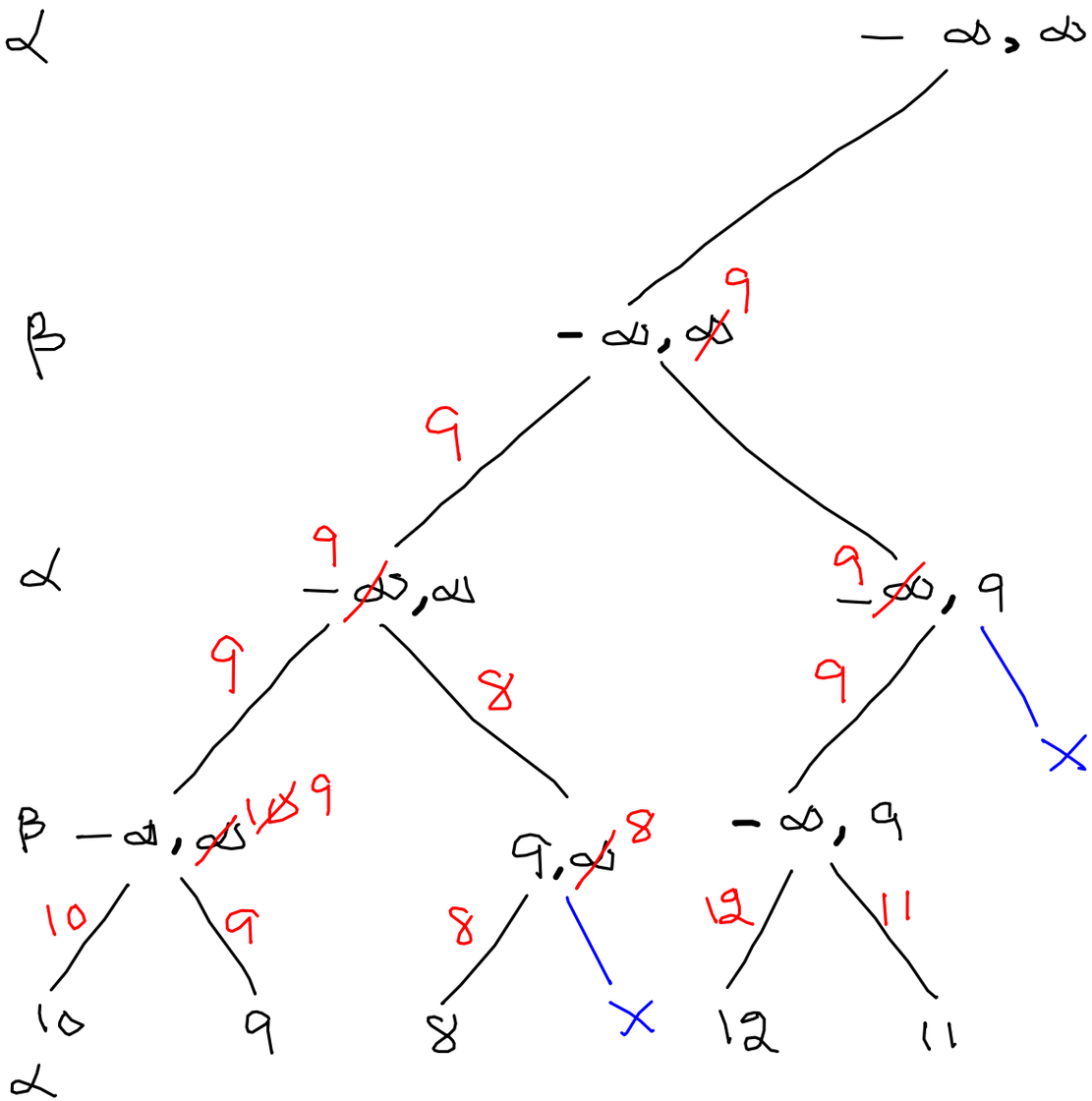
Alpha-Beta Pruning Trace



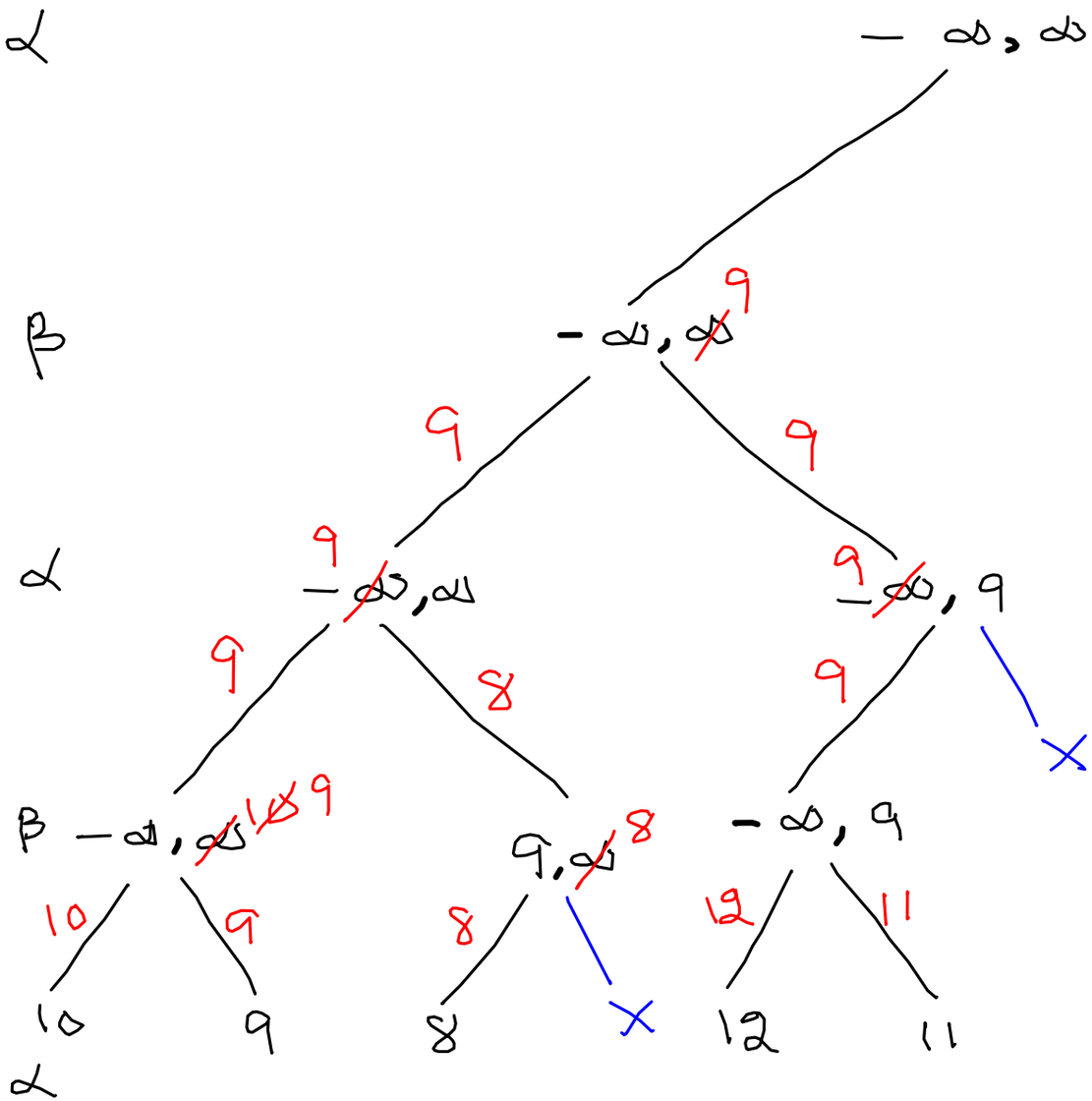
Alpha-Beta Pruning Trace



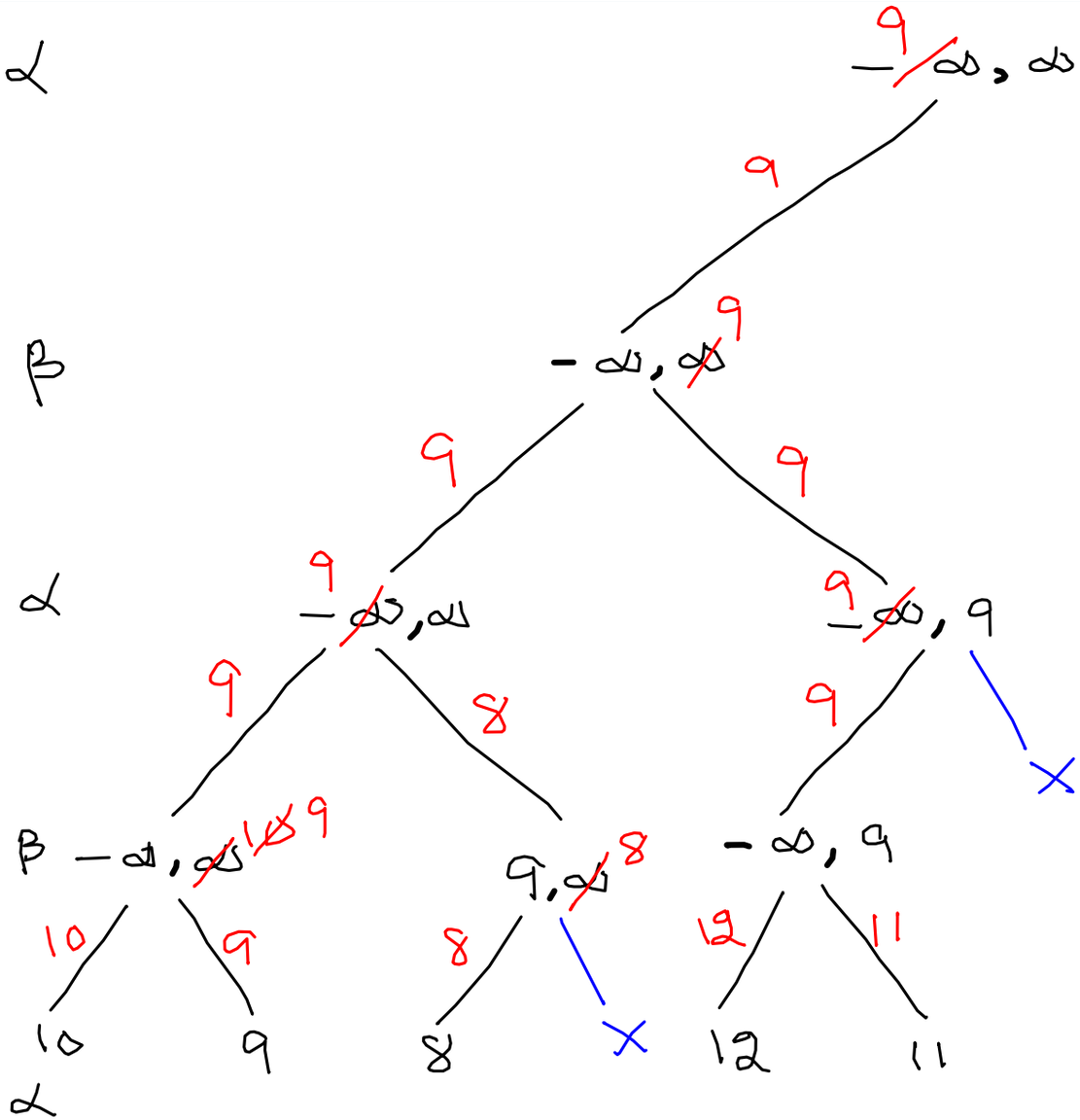
Alpha-Beta Pruning Trace



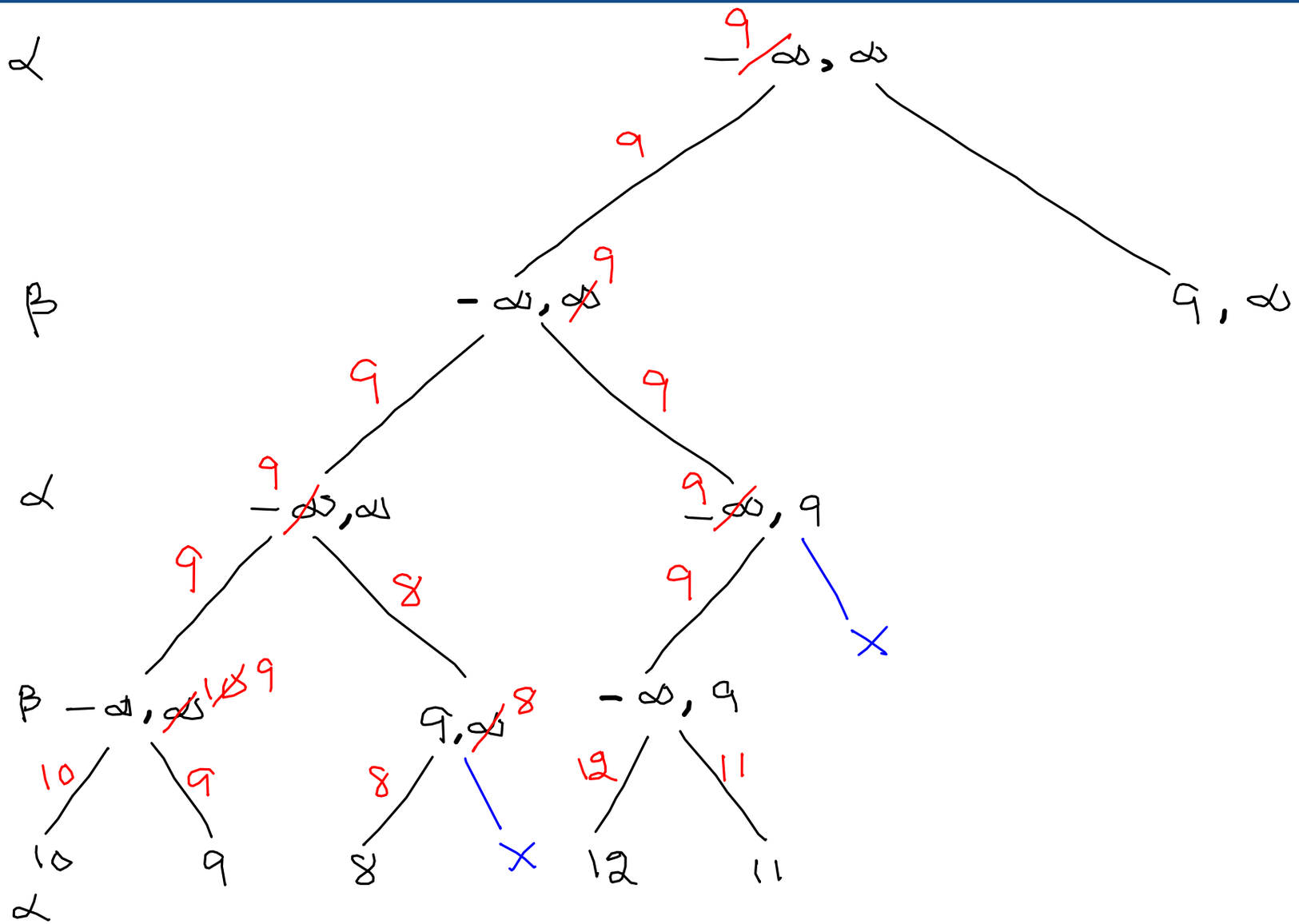
Alpha-Beta Pruning Trace



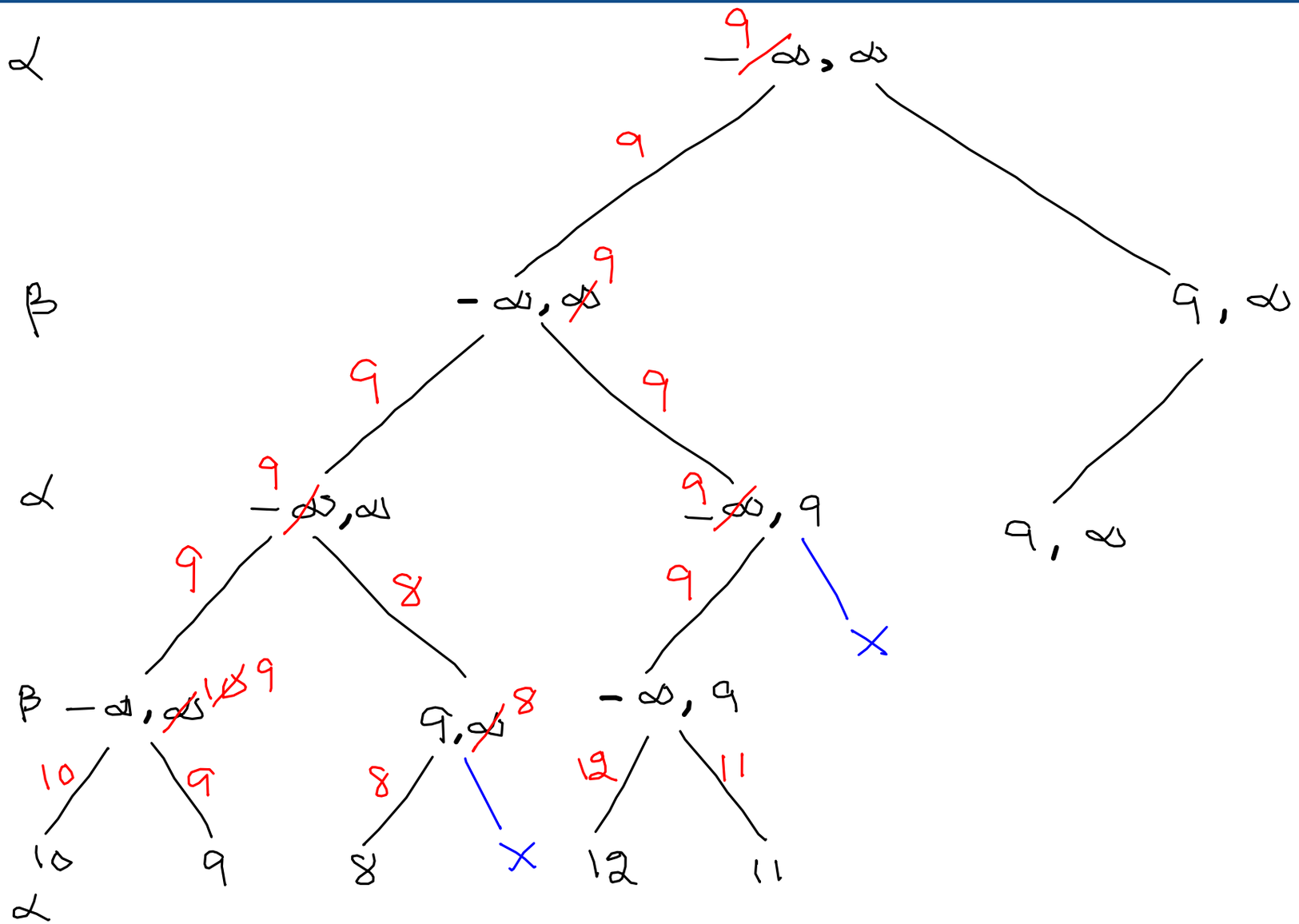
Alpha-Beta Pruning Trace



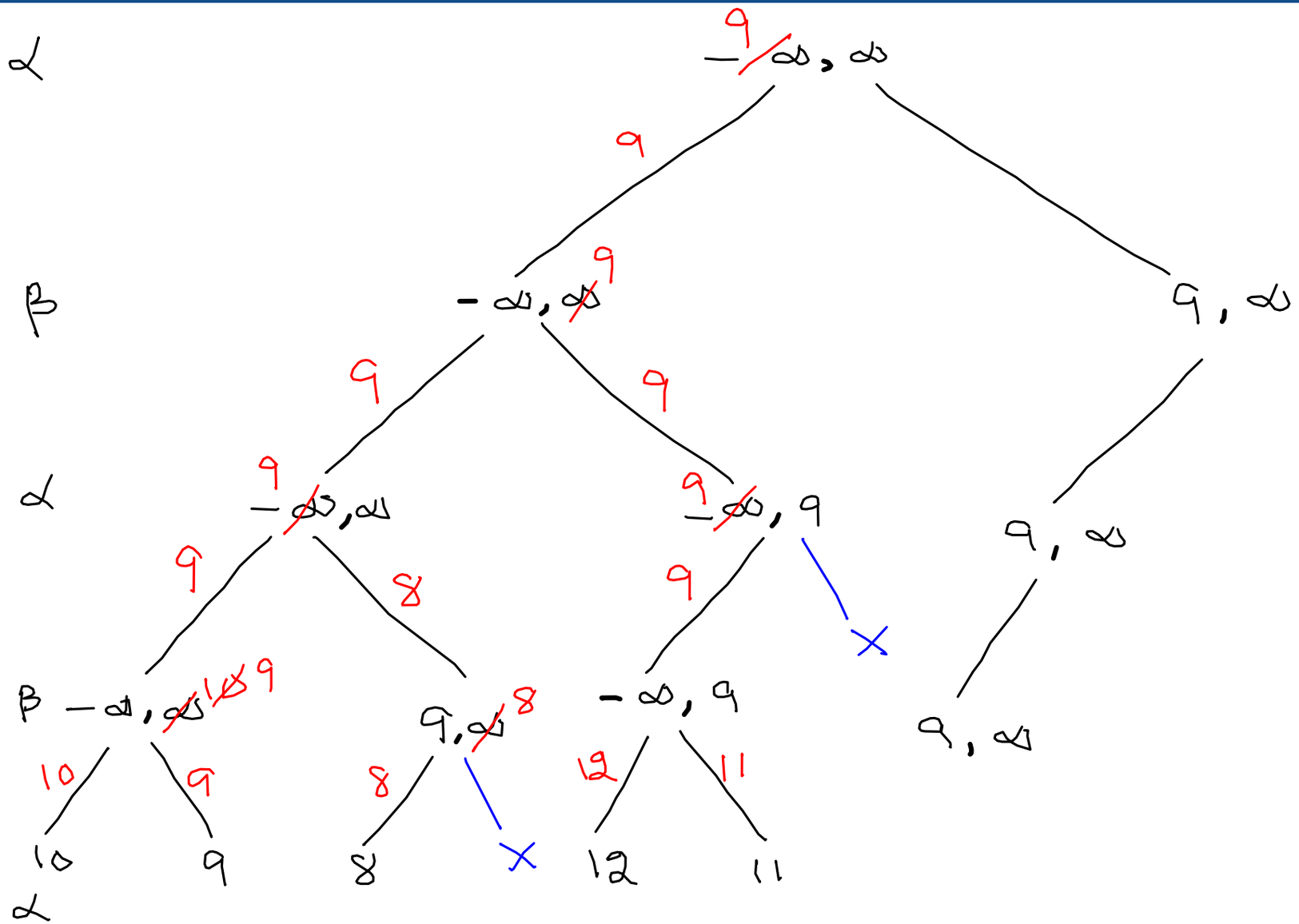
Alpha-Beta Pruning Trace



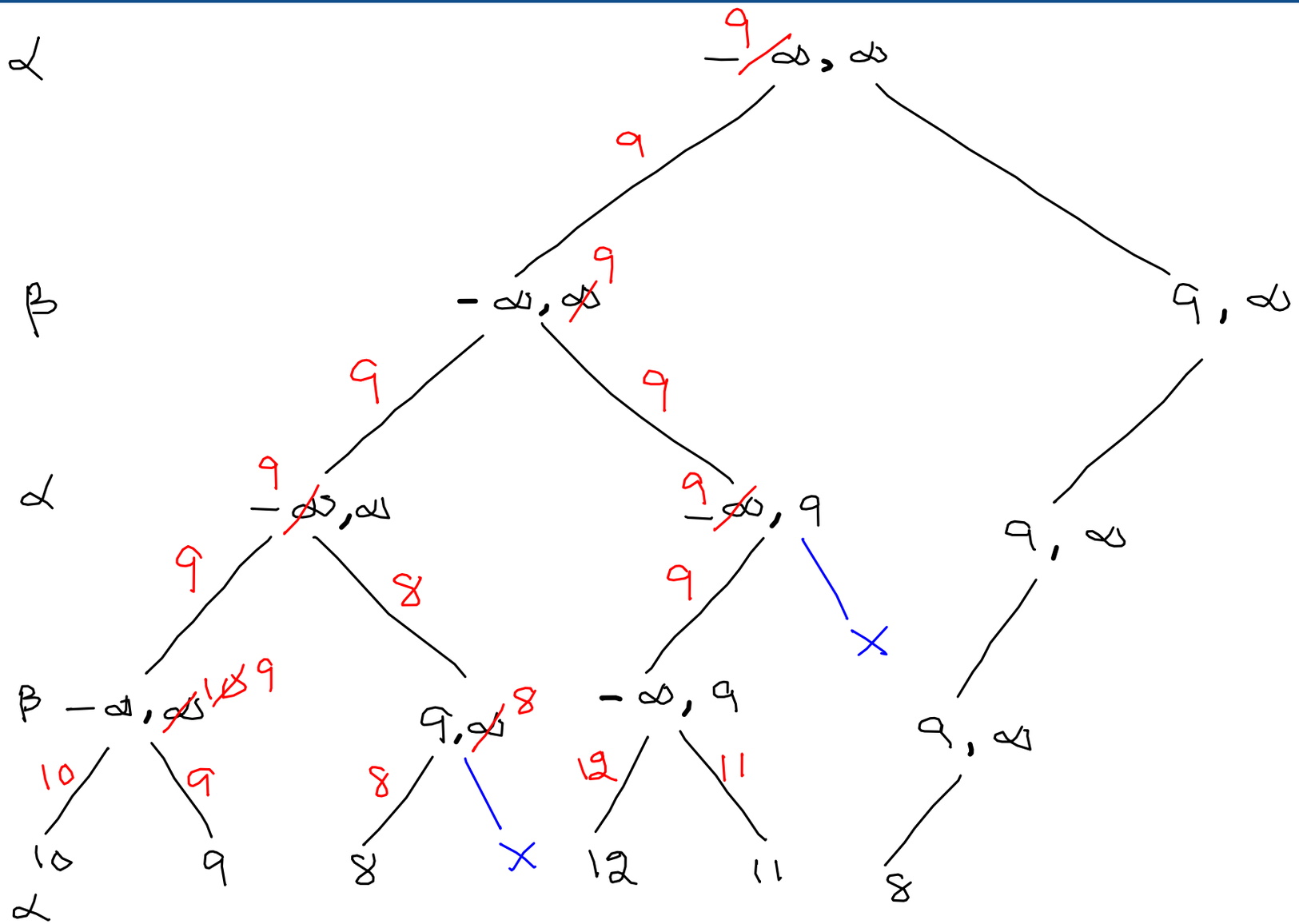
Alpha-Beta Pruning Trace



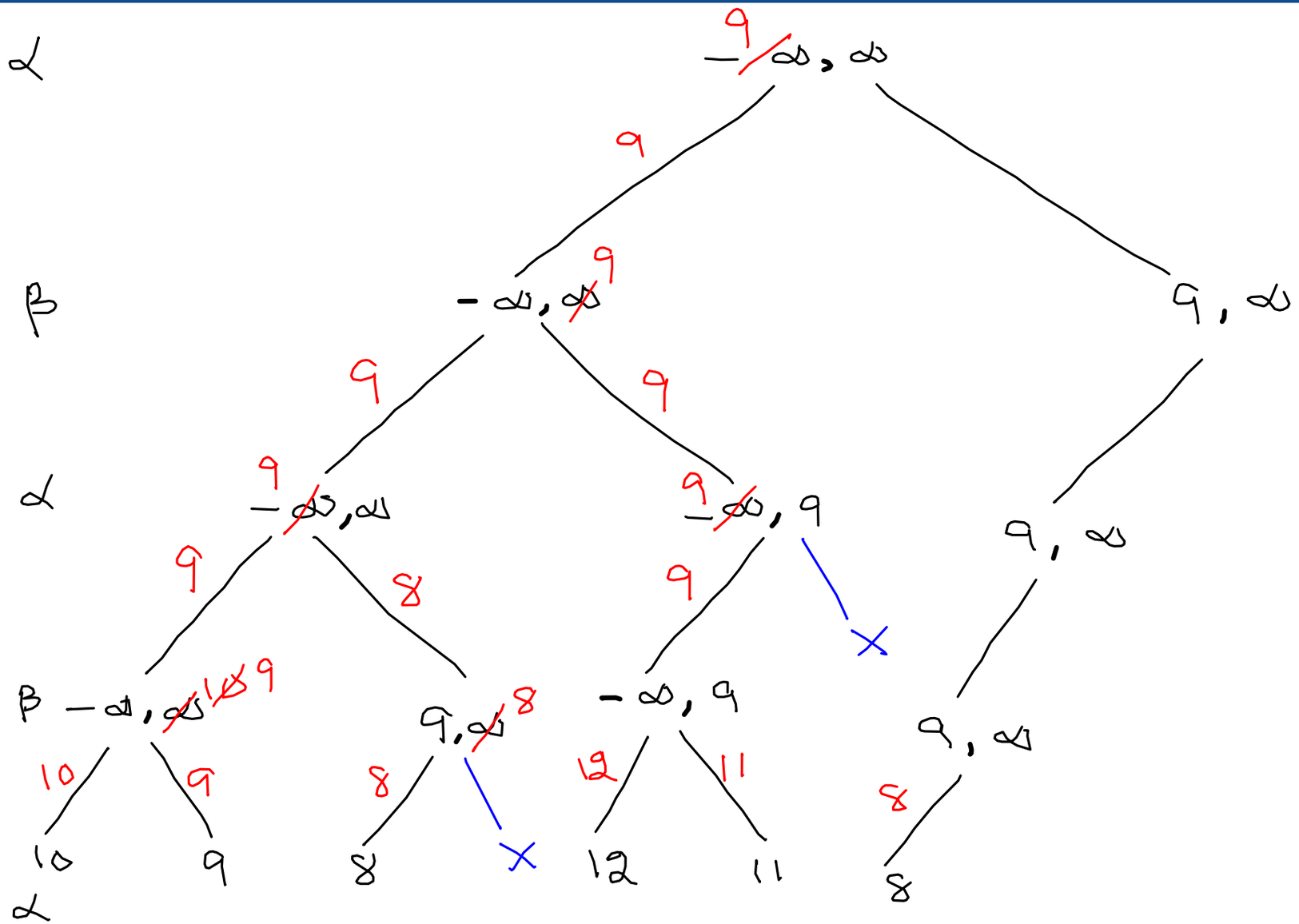
Alpha-Beta Pruning Trace



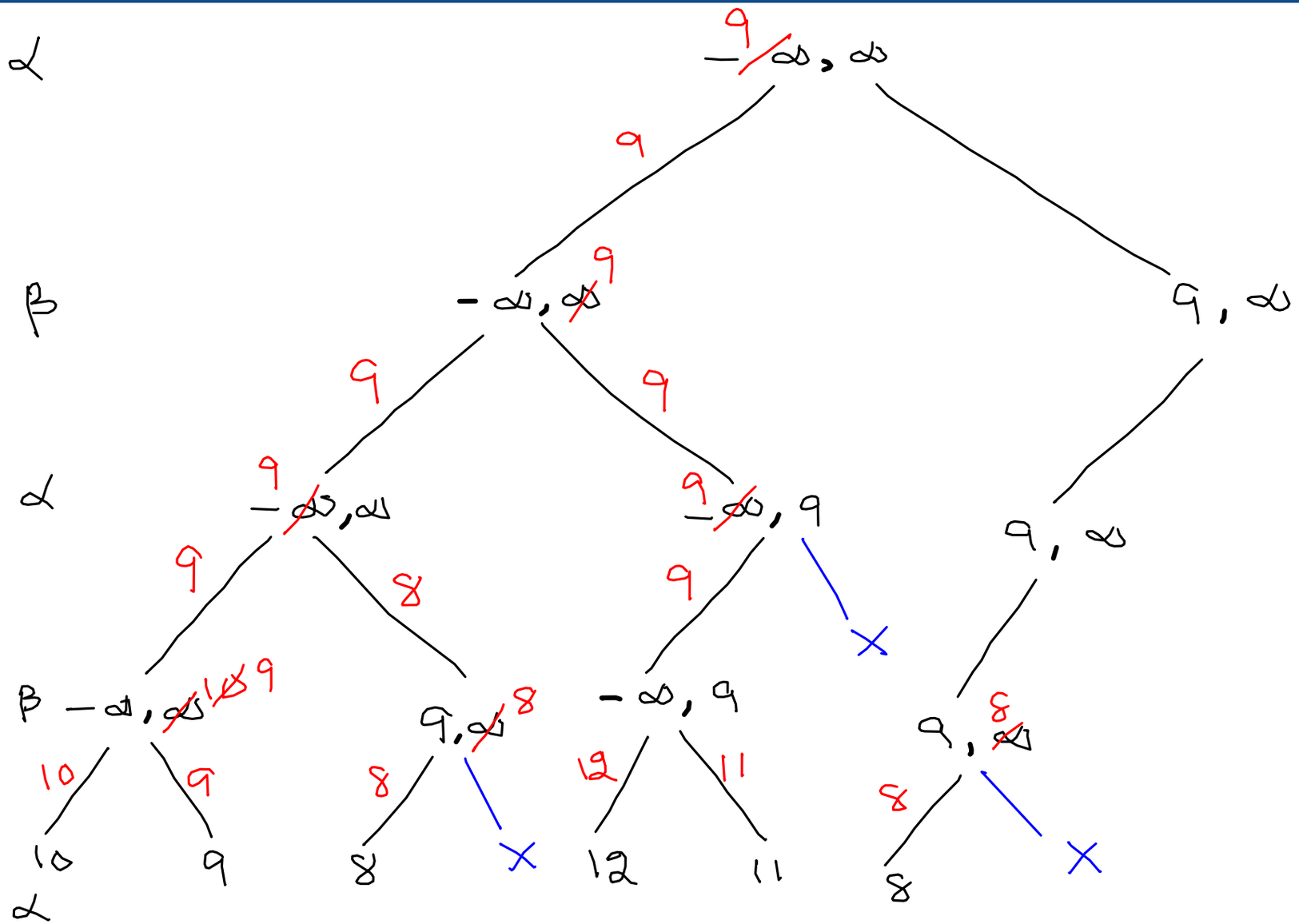
Alpha-Beta Pruning Trace



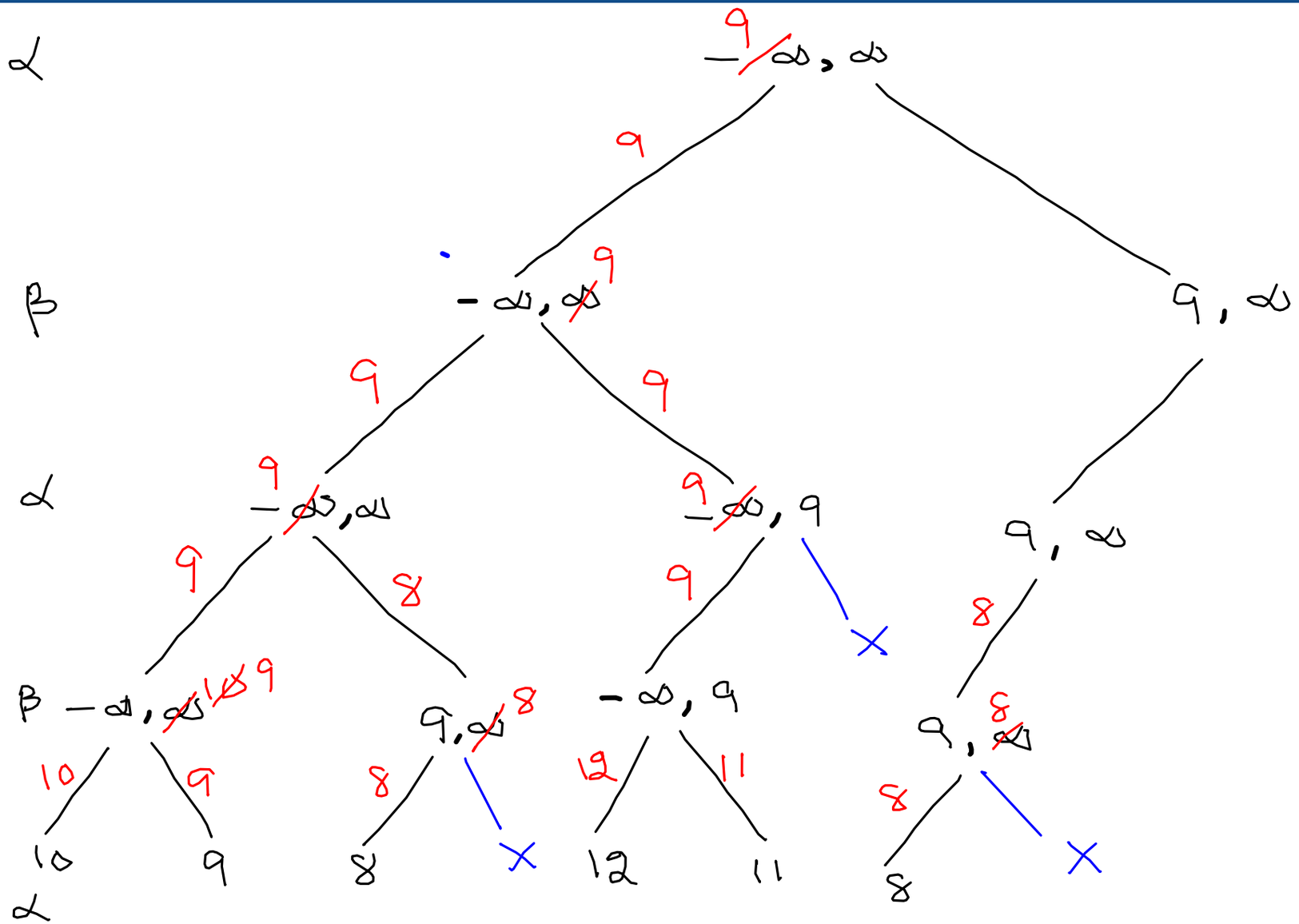
Alpha-Beta Pruning Trace



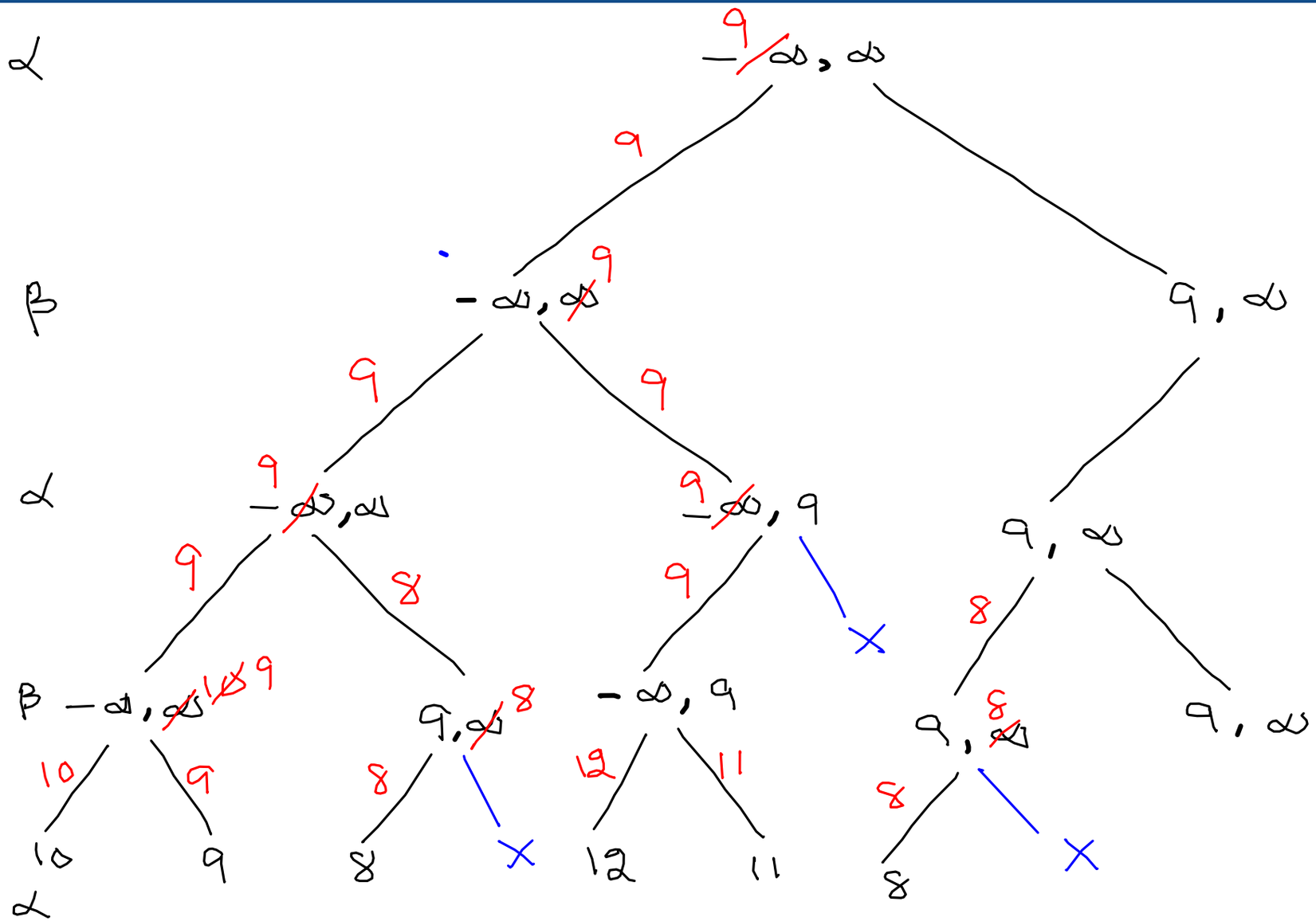
Alpha-Beta Pruning Trace



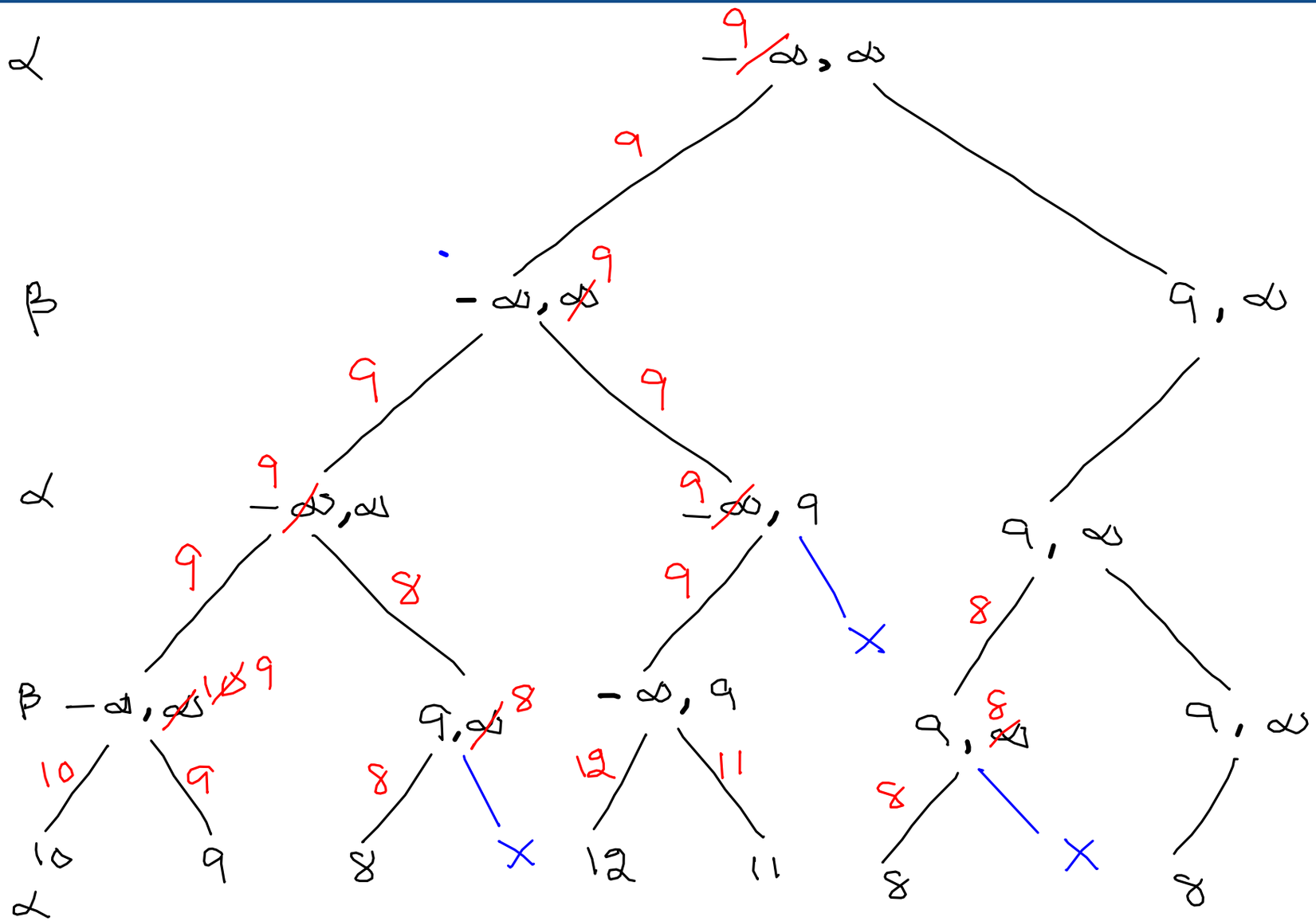
Alpha-Beta Pruning Trace



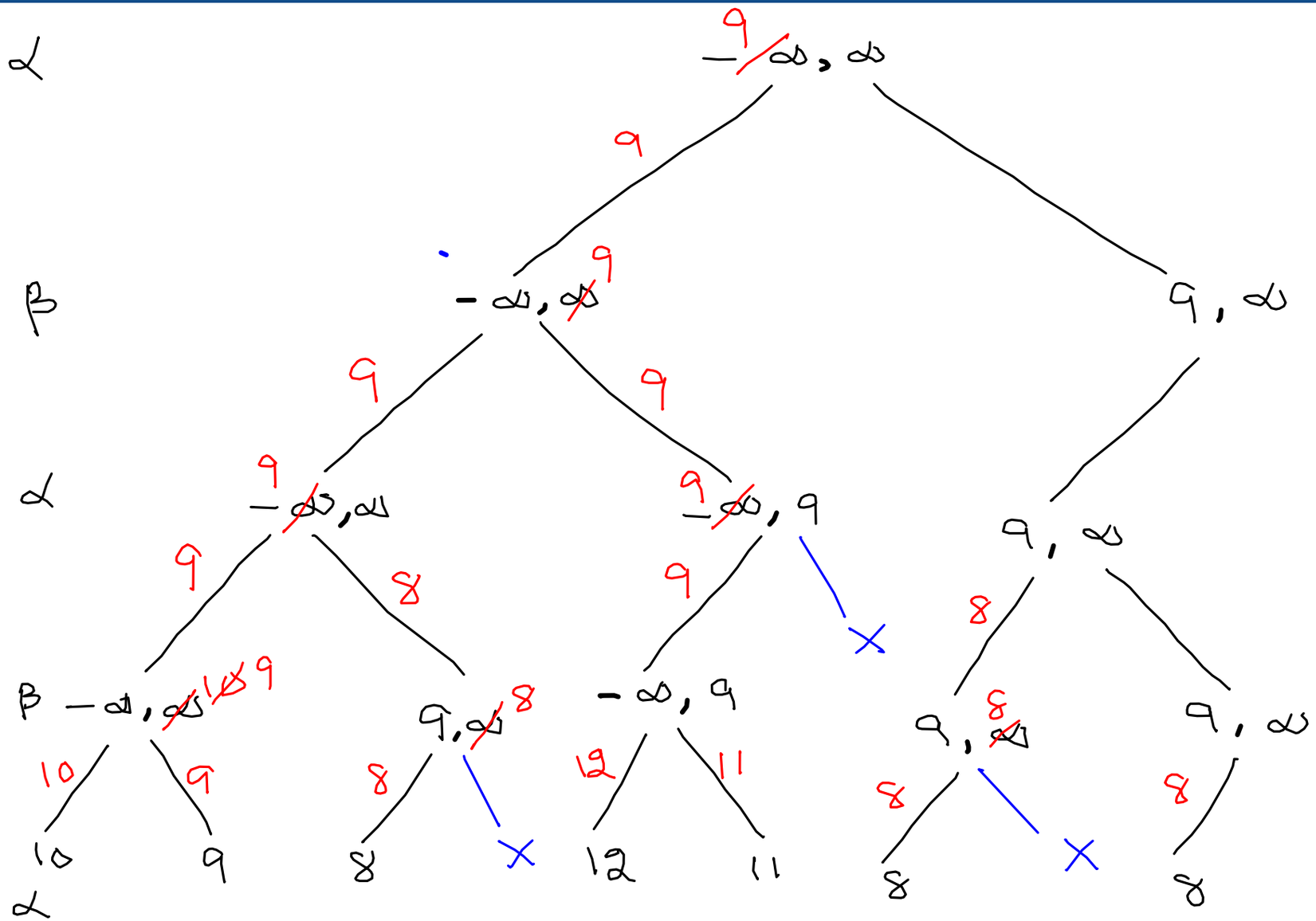
Alpha-Beta Pruning Trace



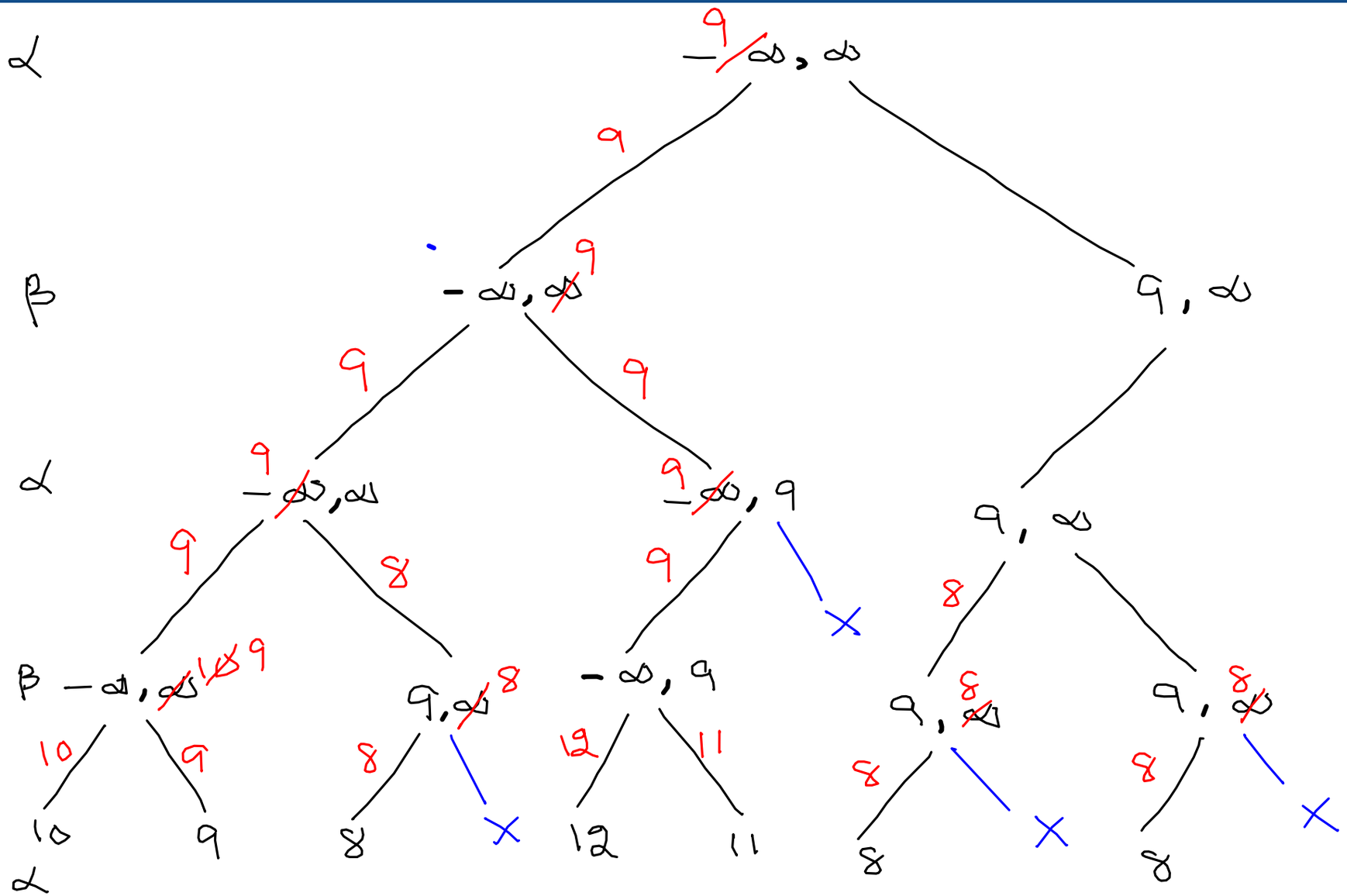
Alpha-Beta Pruning Trace



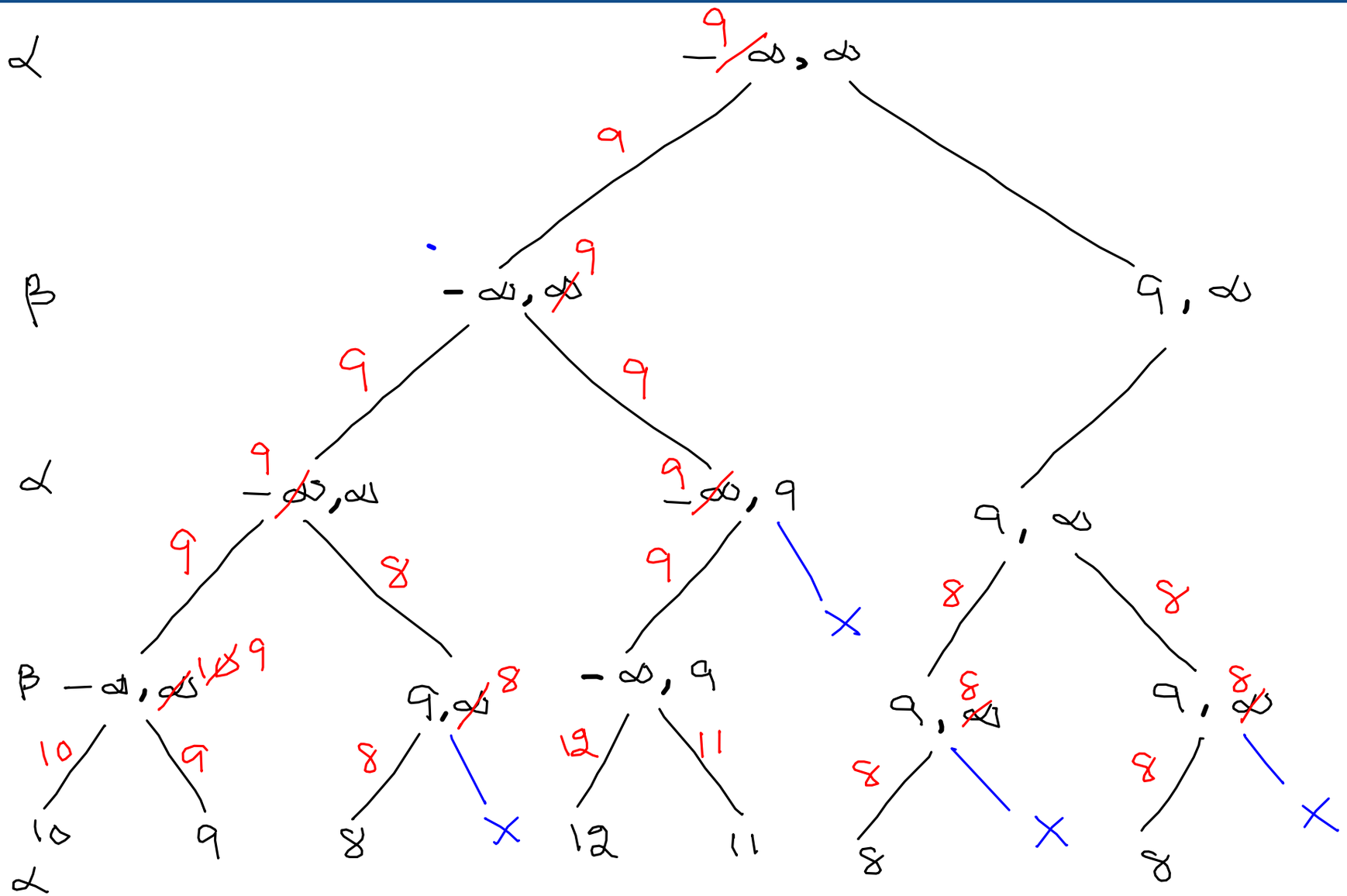
Alpha-Beta Pruning Trace



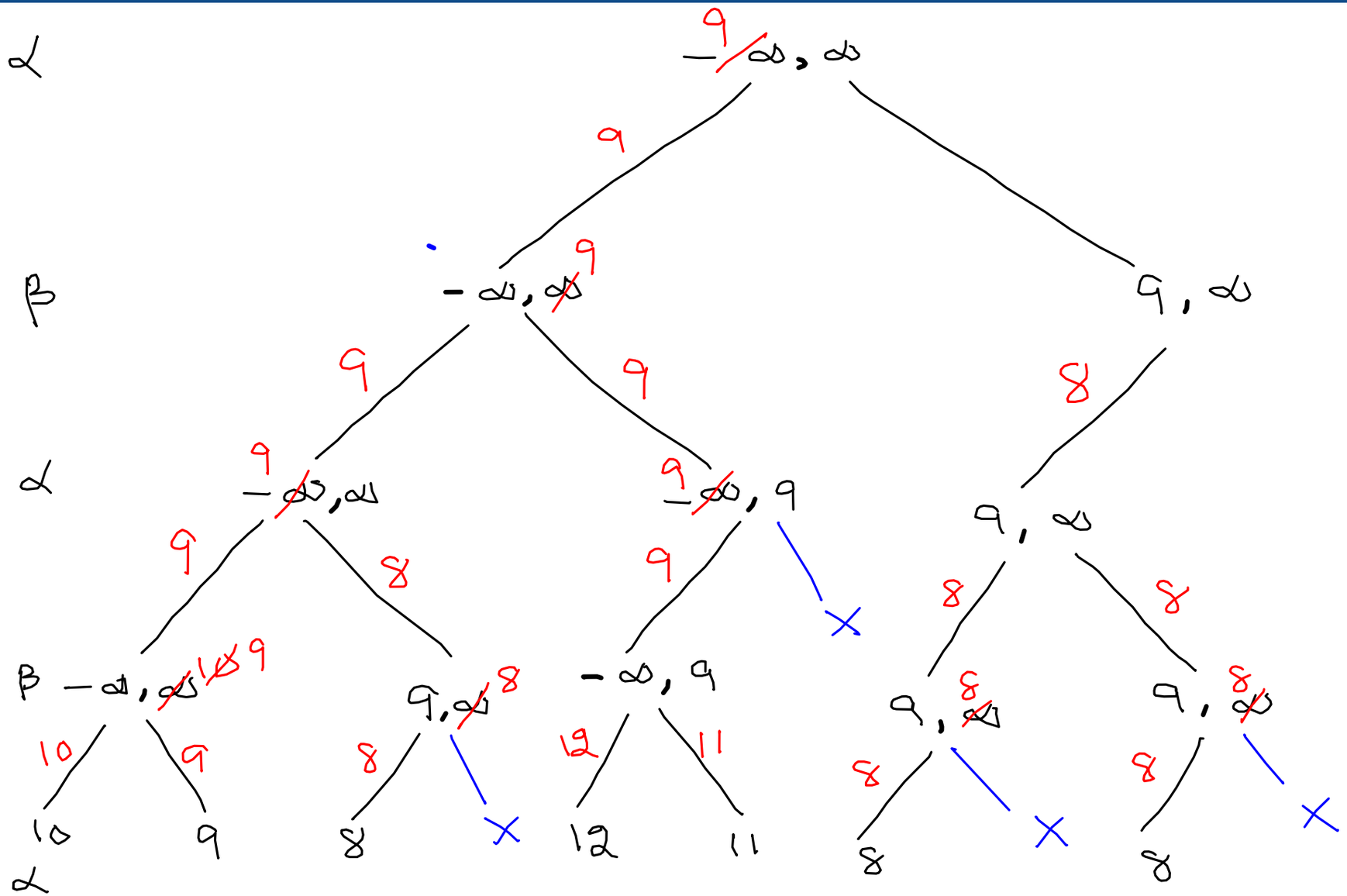
Alpha-Beta Pruning Trace



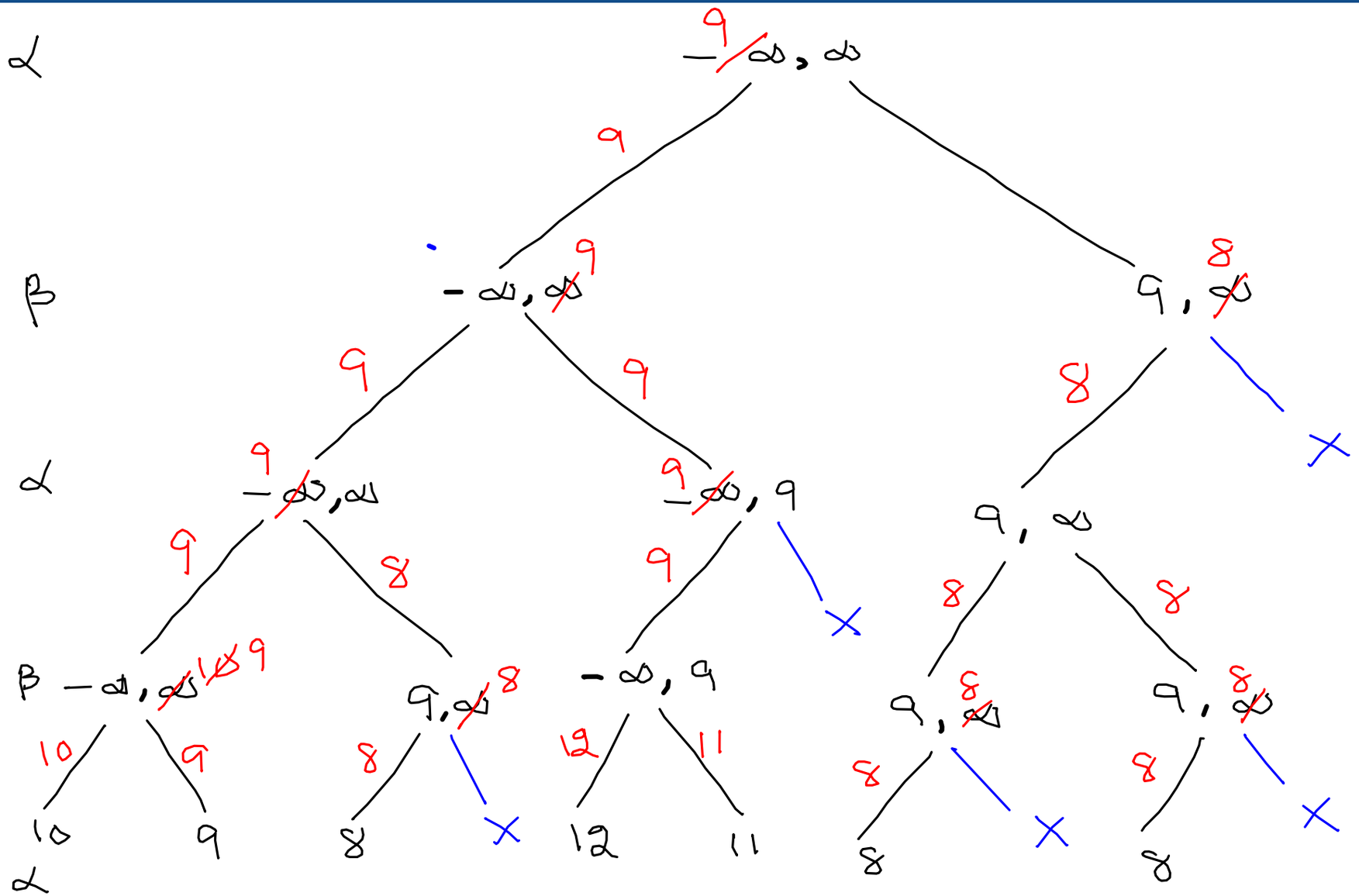
Alpha-Beta Pruning Trace



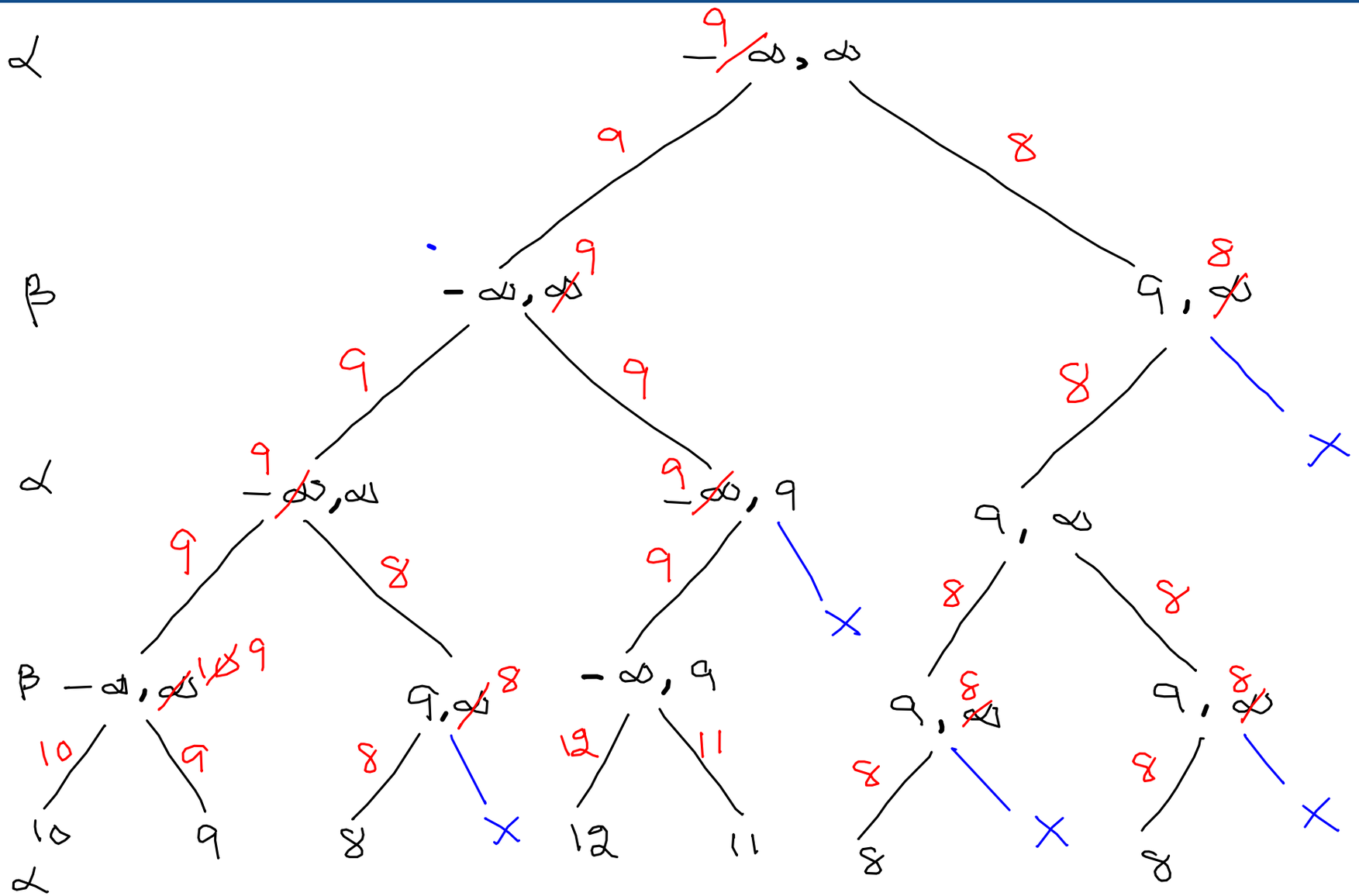
Alpha-Beta Pruning Trace



Alpha-Beta Pruning Trace



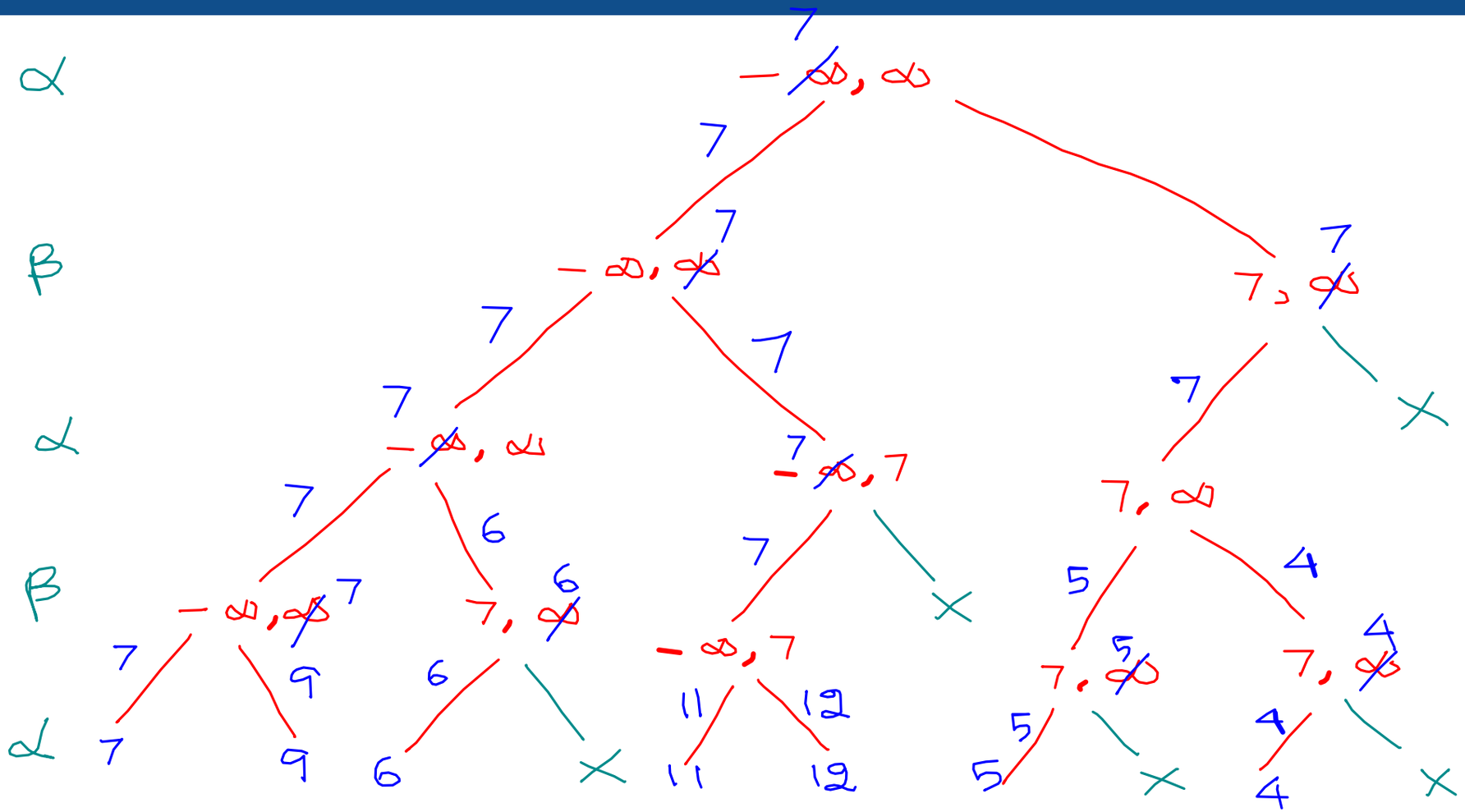
Alpha-Beta Pruning Trace



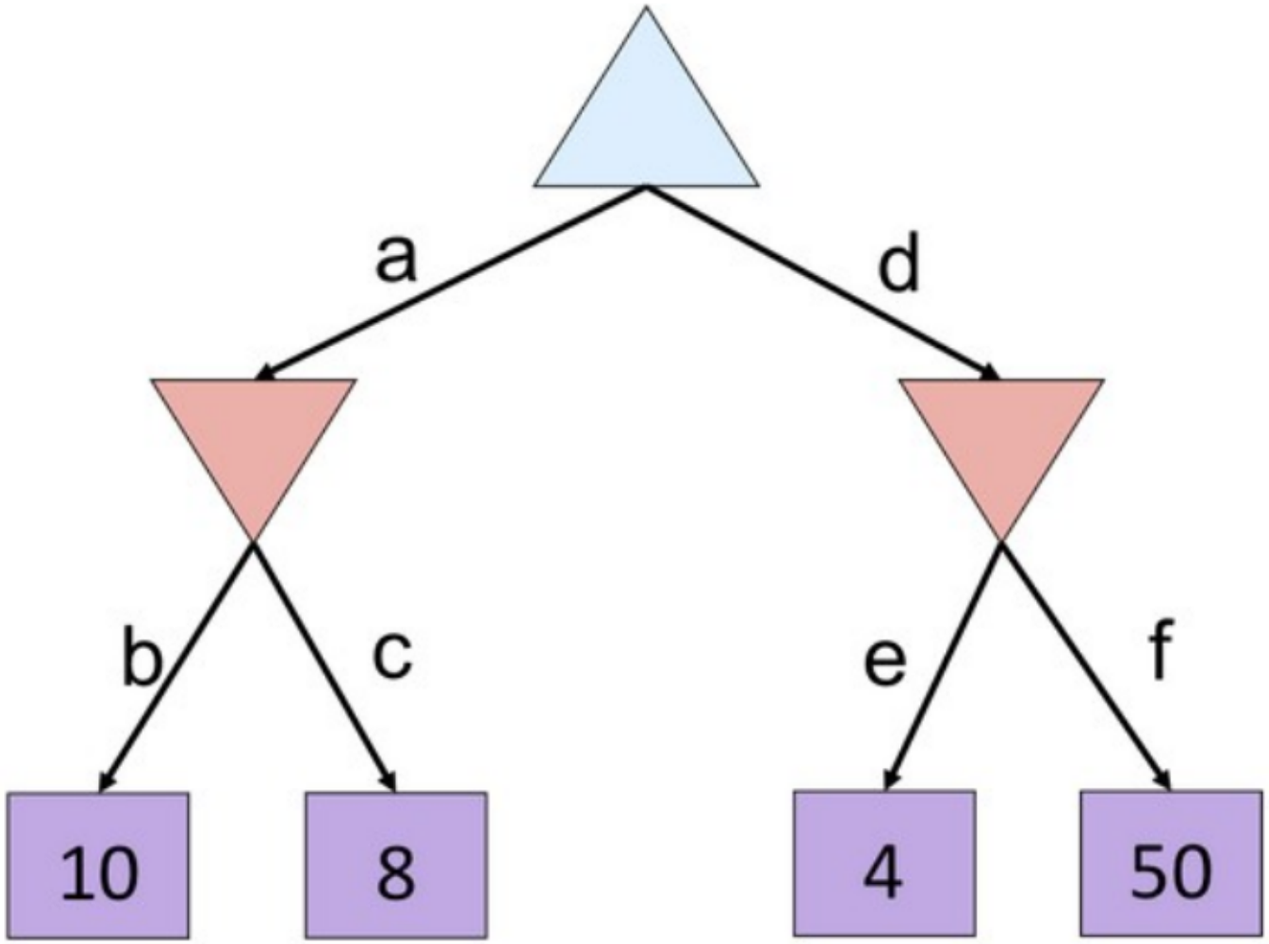
Move Ordering

- Effectiveness of alpha-beta pruning depends on the order in which the states (nodes) are examined.
- Examine the best successors first
  - $O(b^{m/2})$ instead of $O(b^m)$
  - Branching factor: from b to $\sqrt{b}$ (chess: 35 to 6)
  - Alpha-beta pruning can solve a tree twice as deep as minimax in the same amout of time
- Examine in random order
  - $O(b^{3m/4})$ for moderate b

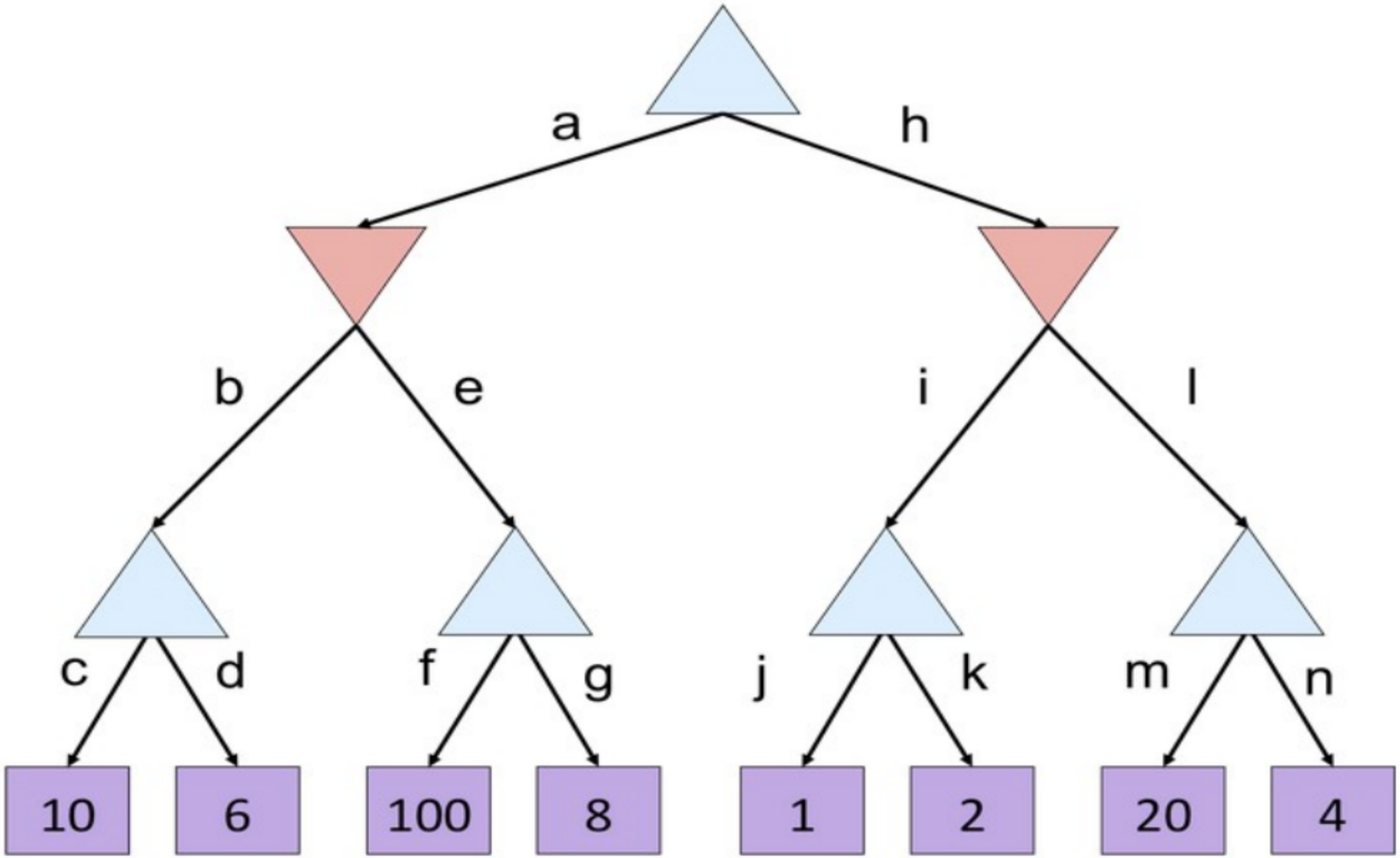
Alpha-Beta Pruning Trace



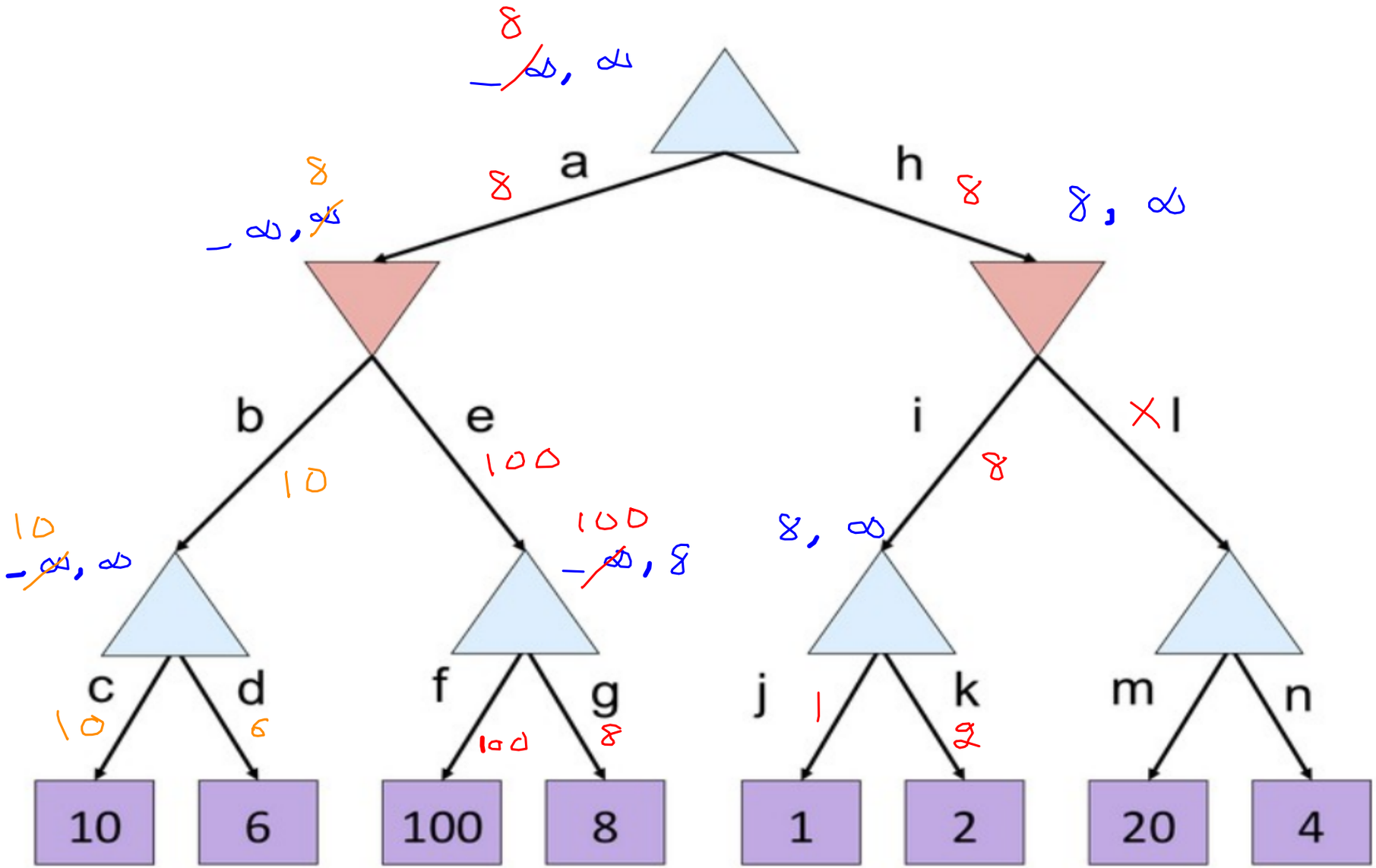
Alpha-Beta Question 1



Alpha-Beta Question 2



Alpha-Beta Solution



Killer Moves

- Dynamic move ordering
- Try first the best moves of the past
- Iterative deepening search: gain information from the current move
  - Search 1 ply first, record the best path of move
  - Search 1 ply deeper, use the recoreded best path of moves to inform the move ordering
  - Adds a constant fraction to search time
  - Made from better move ordering
- Best moves – killer moves
- Try best moves first – killer move heuristic

Transposition Table

- Repeated states in the search tree – exponential search time
- Transpositions – different permutations of the move sequence end up in the same position
- Transposition table
  - Store the evaluation of resulting positions in a hash table – look up
  - explored set in Graph-Search
  - May double the reachable search depth in chess

End